



Python-разработчик в обучающих мирах Minecraft

Модуль 1 - Уровень «Выживание» (Junior)

Урок 1 - Создаём новый мир — установка Python и print()

Шаг 01:

Миссия 1

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Ответ:

- `python --version`
- `check python`
- `install python`
- `python run`

Шаг 02:

Миссия 2

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Ответ:

- `print()`
- `echo()`
- `say()`
- `output()`

🧭 Шаг 03:

Миссия 3

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Ответ:

- **Произойдет ошибка синтаксиса**
- Сообщение всё равно выведется
- Будет пустая строка
- Python проигнорирует кавычки

🧭 Шаг 04:

Миссия 4

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Ответ:

- `print('Minecraft')` — Minecraft
- `print(5+3)` — 8
- `print('5+3')` — 5+3
- `print('gRPC rocks!')` — gRPC rocks!

🧭 Шаг 05:

Миссия 5

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Ответ:

- `print(42)` — Выводит число 42
- `print('42')` — Выводит строку '42'
- `print('Stone'+'Pickaxe')` — Склеивает строки и выводит StonePickaxe
- `print(2*3)` — Вычисляет и выводит число 6

🧭 Шаг 06:

Миссия 6

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Ответ:

- `print('Creeper!')` — Выводит Creeper!
- `print(10-3)` — Выводит 7
- `print('10-3')` — Выводит строку 10-3
- `print(2*'Hi')` — Выводит HiHi

🔮 Шаг 07:

Миссия 7

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Ответ:

- Открыть интерпретатор Python
- Написать команду `print('Hello')`
- Нажать Enter для выполнения

🔮 Шаг 08:

Миссия 8

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Ответ:

- Скачать установщик с python.org
- Запустить установщик и завершить установку
- Проверить установку командой `python --version`

🔮 Шаг 09:

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Ответ:

- Открыть среду разработки или редактор
- Написать `print('Добро пожаловать в мир PythonCraft!')`
- Запустить программу

🔮 Шаг 10:

Миссия 9

Команда программистов-героев Minecraft решила построить новый мир для тестов с gRPC. Чтобы начать, им нужен Python. Что нужно сделать сначала?

Подсказка: Вспомните: без самого Python ни одна команда не запустится.

Ответ:

- **Скачать и установить Python с официального сайта**
- Написать код в блокноте
- Установить Java

- Сразу вызвать команду `print()`

🔗 Шаг 11:

Миссия 10

Герой Алекс проверяет, установился ли Python, чтобы начать писать ботов для своего сервера. Какую команду нужно ввести в терминале?

Подсказка: Проверьте версию — если вы её увидите, значит Python готов.

Ответ:

- **`python --version`**
- `print(version)`
- `run python`
- `py start`

🔗 Шаг 12:

Миссия 11

Стив запускает первый файл для генерации деревни. Каким расширением должен заканчиваться файл с Python-кодом?

Подсказка: Этот формат понимает интерпретатор Python.

Ответ:

- **`.py`**
- `.exe`
- `.txt`
- `.mc`

🔗 Шаг 13:

Миссия 12

Команда программистов в Minecraft хочет вывести сообщение «Добро пожаловать в новый мир!». Как правильно использовать функцию `print`?

Подсказка: Обратите внимание на кавычки.

Ответ:

- **`print("Добро пожаловать в новый мир!")`**
- `print[Добро пожаловать в новый мир!]`
- `echo("Добро пожаловать в новый мир!")`
- `print: Добро пожаловать в новый мир!`

🔗 Шаг 14:

Миссия 13

Программисты делят роли в команде: кто-то настраивает Python, кто-то проверяет установку, кто-то пишет код. Сопоставьте действия и команды.

Подсказка: Подумайте, какая команда нужна для проверки, а какая — для вывода.

Ответ:

- **Установка Python** — Скачивание и установка с python.org
- **Проверка версии** — `python --version`
- **Вывод текста** — `print("Привет, Minecraft!")`

Шаг 15:

Миссия 14

Команда программистов настраивает окружение. Сопоставьте шаг и его результат.

Подсказка: Вспомните: что даёт каждый этап?

Ответ:

- **Установка Python** — Интерпретатор доступен
- **Создание .py файла** — Можно писать код
- **Команда print** — Вывод информации на экран

Шаг 16:

Миссия 15

Стив хочет пошагово подготовить сервер для запуска Python-ботов. Расставьте шаги в правильном порядке.

Подсказка: Сначала нужно подготовить Python, потом проверить, затем уже писать код.

Ответ:

- Скачать и установить Python
- Проверить `python --version`
- Создать .py файл
- Написать `print("Старт")` и запустить

Урок 2 - Первые ресурсы — переменные и типы данных

Шаг 01:

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Ответ:

- **int**
- str
- float
- bool

Шаг 02:

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Ответ:

- **str**
- int
- float
- bool

🔮 Шаг 03:

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Ответ:

- **bool**
- str
- int
- list

🔮 Шаг 04:

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Ответ:

- **int**
- float
- str
- bool

🔮 Шаг 05:

Миссия 5

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Ответ:

- **'Emerald'** — str
- **42** — int
- **True** — bool
- **3.14** — float

🔮 Шаг 06:

Миссия 6

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Ответ:

- Объявить переменные
- Присвоить значения
- Проверить типы данных с помощью type()

🔧 Шаг 07:

Миссия 7

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Ответ:

- Создать переменные для каждого типа урожая
- Присвоить количество ресурсов
- Сложить все значения и сохранить в переменной `total_harvest`

🔧 Шаг 08:

Миссия 8

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Ответ:

- Объявить переменные для ресурсов
- Присвоить каждому ресурсу количество или название
- Вывести значения на экран с помощью `print()`

🔧 Шаг 09:

Миссия 8

Команда программистов-героев Minecraft создала новый сервер с gRPC. Но чтобы начать работу, нужно объявить первую переменную. Как правильно создать переменную для хранения количества алмазов?

Подсказка: Имя переменной должно быть понятным, а значение — числом.

Ответ:

- **`diamonds = 10`**
- `10 = diamonds`
- `var diamonds: 10`
- `let diamonds == 10`

🔧 Шаг 10:

Миссия 9

В лагере программистов-строителей нужно подсчитать блоки камня. Какой тип данных будет у переменной `stone = 64`?

Подсказка: Вспомните: целые числа — это особый тип в Python.

Ответ:

- **`int`**
- `str`
- `float`
- `bool`

🔧 Шаг 11:

Миссия 10

В деревне программистов нашли секретный рецепт зелий. Нужно записать температуру варки зелья: 36.5. Какой тип данных будет у этой переменной?

Подсказка: Вещественные числа содержат точку.

Ответ:

- int
- str
- **float**
- bool

Шаг 12:

Миссия 11

Команда программистов готовит новый сервер. Расположите шаги по созданию переменной в правильном порядке.

Подсказка: Сначала выбираем имя, затем задаём значение и сохраняем в коде.

Ответ:

- Придумать имя переменной
- Присвоить значение
- Записать в коде через =

Урок 3 - Верстак и инструменты — арифметические операции

Шаг 01:

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Ответ:

- **Сложение (+)**
- Вычитание (-)
- Умножение (*)
- Деление (/)

Шаг 02:

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Ответ:

- **Деление (/)**
- Сложение (+)
- Вычитание (-)
- Умножение (*)

Шаг 03:

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Ответ:

- **Вычитание (-)**
- Сложение (+)
- Деление (/)
- Умножение (*)

Шаг 04:

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Ответ:

- **Умножение (*)**
- Сложение (+)
- Деление (/)
- Вычитание (-)

Шаг 05:

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Ответ:

- **Сложение (+)** — Объединяет два числа в одно
- **Вычитание (-)** — Находит разницу между числами
- **Умножение (*)** — Увеличивает число в несколько раз
- **Деление (/)** — Показывает, сколько раз одно число помещается в другое

Шаг 06:

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Ответ:

- **Посчитать все блоки дерева и камня** — Сложение (+)
- **Узнать, сколько блоков осталось после постройки** — Вычитание (-)
- **Удвоить количество кирок** — Умножение (*)
- **Определить, сколько инструментов можно сделать из ресурсов** — Деление (/)

Шаг 07:

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Ответ:

- **Сложение** — Объединение ресурсов
- **Вычитание** — Использование ресурсов на постройку
- **Умножение** — Крафт нескольких инструментов одновременно
- **Деление** — Разделение ресурсов на равные части

Шаг 08:

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Ответ:

- Определить количество каждого ресурса
- Применить арифметические операции
- Сохранить результат в переменной

Шаг 09:

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Ответ:

- Задать переменные с количеством блоков
- Сложить и вычесть блоки по необходимости
- Вывести результат на экран с помощью print()

Шаг 10:

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Ответ:

- Посчитать доступные ресурсы
- Выполнить необходимые арифметические операции
- Сохранить результат или вывести на экран

Урок 4 - Первое жилище — строки и конкатенация

Шаг 01:

Миссия 1

Сопоставьте действия Minecraft с операциями Python для строк:

Подсказка: Подумайте, что делает каждая операция с текстом.

Ответ:

- **Соединить имя игрока с приветствием** — + (конкатенация)
- **Повторить название дома 3 раза** — * (умножение строк)
- **Проверить длину строки** — len()

Шаг 02:

Миссия 2

Расположите шаги для создания приветствия игрока в правильном порядке:

Подсказка: Сначала нужно определить имя, потом объединить строки, затем вывести результат.

Ответ:

- Определить имя игрока
- Соединить строки с помощью + или f-string
- Вывести приветствие на экран

Шаг 03:

Миссия 3

Расположите шаги для создания названия дома с этажами:

Подсказка: Сначала основа, потом этаж, затем вывод.

Ответ:

- Создать основу названия дома
- Добавить номер этажа через + или f-string
- Вывести полное название

Шаг 04:

Миссия 4

Расположите шаги для повторения приветствия 3 раза:

Подсказка: Сначала создаем текст, затем применяем умножение, затем выводим.

Ответ:

- Создать строку приветствия
- Умножить строку на 3
- Вывести результат

Шаг 05:

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Ответ:

- **Объединить имя игрока и приветствие** — Конкатенация (+)
- **Повторить строку несколько раз** — Умножение (*)
- **Разделить текст на части** — Методы split() — только для понимания, не выбираем здесь

- **Сохранить текст без изменений** — Присвоение переменной

Шаг 06:

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Ответ:

- **Соединить название сундука и его содержимое** — Конкатенация (+)
- **Повторить надпись несколько раз** — Умножение (*)
- **Просто присвоить текст переменной** — Присвоение

Шаг 07:

Миссия 7

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Ответ:

- Определить базовые строки (название, имя игрока)
- Объединить строки или повторить при необходимости
- Сохранить результат в переменной или вывести на экран

Шаг 08:

Миссия 8

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Ответ:

- Присвоить переменные с текстом
- Применить конкатенацию или повторение
- Вывести надпись с помощью `print()`

Шаг 09:

Миссия 9

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Ответ:

- Определить тексты для вывесок
- Применить конкатенацию или повторение
- Сохранить или вывести результат

Шаг 10:

Миссия 9

Формулировка: Сопоставьте типы строковых литералов в Python с их примерами: команда Minecraft использует разные виды строк.

Понятия:

Ответ:

- Одинарные кавычки
- Двойные кавычки
- Тройные кавычки
- Сырые строки (raw strings)

Примеры:

1. "Hello from grpc"
2. 'Дом из кирпича'
3. '''Многострочный текст жилища'''
4. r"C:\\Minecraft\\Home"

 **Подсказка:**

Обратите внимание на кавычки вокруг строк и префикс r для сырых строк.

- **Одинарные кавычки** — 'Дом из кирпича'
- **Двойные кавычки** — "Hello from grpc"
- **Тройные кавычки** — '''Многострочный текст жилища'''
- **Сырые строки (raw strings)** — r"C:\\Minecraft\\Home"

 **Шаг 11:**

Миссия 10

Формулировка: Сопоставьте методы строк с их поведением: герои из команды Minecraft рассматривают разные операции со строками.

Методы:

Ответ:

- upper()
- lower()
- strip()
- replace(old, new)

Что делают:

1. заменяет все вхождения одной подстроки на другую
2. удаляет пробелы в начале и в конце строки
3. преобразует все символы в верхний регистр
4. преобразует все символы в нижний регистр

 **Подсказка:**

Методы относятся к строковым объектам, и каждый метод делает именно одно из перечисленных действий.

- **upper()** — преобразует все символы в верхний регистр
- **lower()** — преобразует все символы в нижний регистр
- **strip()** — удаляет пробелы в начале и в конце строки
- **replace(old, new)** — заменяет все вхождения одной подстроки на другую

 **Шаг 12:**

Миссия 11

Формулировка: Сопоставьте шаблоны форматирования с их примерами: команда Minecraft хочет форматировать координаты своего дома.

Шаблоны форматирования:

Ответ:

- конкатенация с +
- f-строки
- метод .format()
- старый стиль %

Примеры:

1. "Дом находится в x=%d, y=%d" % (x, y)
2. "Координаты: x=" + str(x) + ", y=" + str(y)
3. f"Координаты: x={x}, y={y}"
4. "Координаты: x={}, y={}".format(x, y)

 **Подсказка:**

Обратите внимание, как именно вставляются переменные внутри строк.

- **конкатенация с +** — "Координаты: x=" + str(x) + ", y=" + str(y)
- **f-строки** — f"Координаты: x={x}, y={y}"
- **.format()** — "Координаты: x={}, y={}".format(x, y)
- **старый стиль %** — "Дом находится в x=%d, y=%d" % (x, y)

 **Шаг 13:**

Миссия 12

Вопрос: Расположите шаги для безопасной конкатенации строк, чтобы программа gRPC-протокола Minecraft не падала, если переменные None или числа.

1. Привести значение к строке с помощью str() или аналогичного метода
2. Проверить, что значение не None
3. Конкатенировать с другими строками
4. Обработать возможное исключение (если ввод некорректен)

 **Подсказка:**

Думайте: сначала проверка, потом преобразование, затем объединение, и в конце — защита.

Ответ:

- Проверить, что значение не None
- Привести значение к строке с помощью str() или аналогичного метода
- Конкатенировать с другими строками
- Обработать возможное исключение (если ввод некорректен)

 **Шаг 14:**

Миссия 13

Вопрос: Расположите этапы создания первого жилища в Minecraft с точки зрения gRPC-сервиса, который формирует приветственные сообщения. От фазы разработки до вывода сообщения.

1. Определение шаблона строки
2. Сбор нужных переменных (имя, материал, координаты)
3. Конкатенация или форматирование
4. Использование gRPC-метода для отправки сообщения клиенту

Подсказка:

Подумайте, какая часть должна быть после сбора данных, а какая — до отправки по сети.

Ответ:

- Определение шаблона строки
- Сбор нужных переменных (имя, материал, координаты)
- Конкатенация или форматирование
- Использование gprc-метода для отправки сообщения клиенту

Шаг 15:

Миссия 14

Вопрос: Команда Minecraft хочет логировать события строительства с помощью gprc. Расположите этапы логирования сообщений с помощью строк:

1. Составление текста события (например, "Построен дом из кожи")
2. Конвертация необходимых значений в строки (материал, имя строителя)
3. Добавление временной метки в начало или конец сообщения
4. Отправка или сохранение лога

Подсказка:

Сначала собираем содержимое, затем дополняем контекстом, и в конце сохраняем или отправляем.

Ответ:

- Конвертация необходимых значений в строки (материал, имя строителя)
- Составление текста события (например, "Построен дом из кожи")
- Добавление временной метки в начало или конец сообщения
- Отправка или сохранение лога

Шаг 16:

Миссия 15

Вопрос: Расположите шаги, которые программист Minecraft-группы должен выполнить, чтобы защитить gprc-сервис от SQL-инъекций при формировании строки запроса, используя только конкатенацию (что не рекомендуется, но возможно):

1. Шаблон запроса с заполнителями
2. Экранирование специальных символов в пользовательском вводе
3. Проверка допустимых символов (в whitelist)
4. Сборка финального SQL-запроса
5. Логирование или уведомление при обнаружении недопустимых символов

Подсказка:

Безопасность — сначала проверка ввода, потом экранирование, и лишь затем формирование строки и отправка.

Ответ:

- Проверка допустимых символов (в whitelist)
- Экранирование специальных символов в пользовательском вводе
- Шаблон запроса с заполнителями
- Сборка финального SQL-запроса
- Логирование или уведомление при обнаружении недопустимых символов

Урок 5 - Первая ночь — условия (if/else)

Шаг 01:

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Ответ:

- **if/else**
- for
- while
- def

Шаг 02:

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Ответ:

- **if/else**
- while
- for
- try/except

Шаг 03:

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Ответ:

- **if здоровье < 5: использовать_зелье else: продолжать_игру**
- while здоровье < 5: использовать_зелье
- for здоровье in диапазон: использовать_зелье
- def здоровье(): использовать_зелье

Шаг 04:

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Ответ:

- **if/else**
- for
- while
- try/except

🔮 Шаг 05:

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Ответ:

- Ставим факел, если темно — if
- Продолжаем копать, если нет монстров — else
- Проверяем инвентарь — if/else

🔮 Шаг 06:

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Ответ:

- Определить состояние (например, здоровье или время суток)
- Проверить условие с помощью if
- Выполнить действие для True или False

🔮 Шаг 07:

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Ответ:

- Проверить количество досок
- if досок ≥ 3 : строим кровать
- else: добыть еще досок

🔮 Шаг 08:

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Ответ:

- Определить, есть ли моб рядом
- if есть моб: атаковать
- else: продолжать строить стену

🔮 Шаг 09:

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Ответ:

- `if/else`
- `for`
- `while`
- `def`

Шаг 10:

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Ответ:

- if/else
- while
- for
- try/except

Урок 6 - Ферма пшеницы — циклы for и while

Шаг 01:

Миссия 1

Стив хочет собрать пшеницу с 5 грядок. Какой цикл лучше использовать, если количество грядок известно заранее?

Подсказка: Подумайте, какой цикл повторяется заданное количество раз.

Ответ:

- **for**
- while
- if
- switch

Шаг 02:

Миссия 2

Алекс хочет поливать пшеницу до тех пор, пока она не вырастет. Какой цикл лучше использовать, если условие роста неизвестно заранее?

Подсказка: Подумайте, какой цикл зависит от условия, а не от количества повторений.

Ответ:

- **while**
- for
- if
- switch

Шаг 03:

Миссия 3

Стив хочет подсчитать количество собранных пшеничных блоков из 3 рядов, по 4 блока в каждом. Какой цикл поможет это сделать проще всего?

Подсказка: Счётчик нужен для фиксированного числа повторений.

Ответ:

- **for**
- while
- if
- switch

Шаг 04:

Миссия 4

Алекс заметил, что некоторые пшеницы ещё не выросли. Какой цикл поможет поливать каждую грядку до роста?

Подсказка: Нужно повторять действие, пока условие не выполнено.

Ответ:

- **while**
- for
- if
- switch

🔗 Шаг 05:

Миссия 6

Сопоставьте описание с правильным примером цикла:

Подсказка: Обратите внимание на известное количество повторений и условие окончания.

Ответ:

- **Собрать пшеницу с 5 рядов** — `for row in range(5):`
- **Поливать, пока рост не завершён** — `while not grown:`
- **Подсчитать блоки в 3 рядах** — `for i in range(3):`

🔗 Шаг 06:

Миссия 7

Сопоставьте операции с результатом цикла:

Подсказка: Смотрите, что делает каждая конструкция с блоками пшеницы.

Ответ:

- **`for i in range(3):` собрать блок** — 3 блока собрано
- **`while not grown:` поливать** — Полив продолжается до роста
- **`for i in range(5):` подсчитать ряды** — 5 рядов

🔗 Шаг 07:

Миссия 8

Расположите шаги для сбора пшеницы с известным количеством рядов:

Подсказка: Сначала инициализация цикла, потом действия, потом завершение.

Ответ:

- `for row in range(5):`
- Собрать пшеницу в текущем ряду
- Перейти к следующему ряду

🔗 Шаг 08:

Миссия 9

Расположите шаги для полива пшеницы до роста:

Подсказка: Сначала проверяем состояние, потом поливаем, затем снова проверяем.

Ответ:

- `while not grown:`
- Полить грядку
- Проверить рост

🔗 Шаг 09:

Миссия 10

Расположите шаги для подсчёта блоков в 3 рядах:

Подсказка: Используйте цикл для каждого ряда и суммируйте блоки.

Ответ:

- `total = 0`
- `for i in range(3):`

- `total += blocks_in_row[i]`

Шаг 10:

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 5

Сопоставьте действия Minecraft с операциями Python для строк:

Подсказка: Подумайте, что делает каждая операция с текстом.

Миссия 8

Расположите шаги для создания приветствия игрока в правильном порядке:

Подсказка: Сначала нужно определить имя, потом объединить строки, затем вывести результат.

Миссия 9

Расположите шаги для создания названия дома с этажами:

Подсказка: Сначала основа, потом этаж, затем вывод.

Миссия 10

Расположите шаги для повторения приветствия 3 раза:

Подсказка: Сначала создаем текст, затем применяем умножение, затем выводим.

Миссия 1

Стив хочет собрать пшеницу с 5 грядок. Какой цикл лучше использовать, если количество грядок известно заранее?

Подсказка: Подумайте, какой цикл повторяется заданное количество раз.

Миссия 2

Алекс хочет поливать пшеницу до тех пор, пока она не вырастет. Какой цикл лучше использовать, если условие роста неизвестно заранее?

Подсказка: Подумайте, какой цикл зависит от условия, а не от количества повторений.

Миссия 3

Стив хочет подсчитать количество собранных пшеничных блоков из 3 рядов, по 4 блока в каждом. Какой цикл поможет это сделать проще всего?

Подсказка: Счётчик нужен для фиксированного числа повторений.

Миссия 4

Алекс заметил, что некоторые пшеницы ещё не выросли. Какой цикл поможет поливать каждую грядку до роста?

Подсказка: Нужно повторять действие, пока условие не выполнено.

Миссия 6

Сопоставьте описание с правильным примером цикла:

Подсказка: Обратите внимание на известное количество повторений и условие окончания.

Миссия 7

Сопоставьте операции с результатом цикла:

Подсказка: Смотрите, что делает каждая конструкция с блоками пшеницы.

Миссия 8

Расположите шаги для сбора пшеницы с известным количеством рядов:

Подсказка: Сначала инициализация цикла, потом действия, потом завершение.

Миссия 9

Расположите шаги для полива пшеницы до роста:

Подсказка: Сначала проверяем состояние, потом поливаем, затем снова проверяем.

Миссия 10

Расположите шаги для подсчёта блоков в 3 рядах:

Подсказка: Используйте цикл для каждого ряда и суммируйте блоки.

Ответ:

- **while not all_grown:**
- while all_grown:
- for not all_grown:
- if not all_grown:

Шаг 11:

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 5

Сопоставьте действия Minecraft с операциями Python для строк:

Подсказка: Подумайте, что делает каждая операция с текстом.

Миссия 8

Расположите шаги для создания приветствия игрока в правильном порядке:

Подсказка: Сначала нужно определить имя, потом объединить строки, затем вывести результат.

Миссия 9

Расположите шаги для создания названия дома с этажами:

Подсказка: Сначала основа, потом этаж, затем вывод.

Миссия 10

Расположите шаги для повторения приветствия 3 раза:

Подсказка: Сначала создаем текст, затем применяем умножение, затем выводим.

Миссия 1

Стив хочет собрать пшеницу с 5 грядок. Какой цикл лучше использовать, если количество грядок известно заранее?

Подсказка: Подумайте, какой цикл повторяется заданное количество раз.

Миссия 2

Алекс хочет поливать пшеницу до тех пор, пока она не вырастет. Какой цикл лучше использовать, если условие роста неизвестно заранее?

Подсказка: Подумайте, какой цикл зависит от условия, а не от количества повторений.

Миссия 3

Стив хочет подсчитать количество собранных пшеничных блоков из 3 рядов, по 4 блока в каждом. Какой цикл поможет это сделать проще всего?

Подсказка: Счётчик нужен для фиксированного числа повторений.

Миссия 4

Алекс заметил, что некоторые пшеницы ещё не выросли. Какой цикл поможет поливать каждую грядку до роста?

Подсказка: Нужно повторять действие, пока условие не выполнено.

Миссия 6

Сопоставьте описание с правильным примером цикла:

Подсказка: Обратите внимание на известное количество повторений и условие окончания.

Миссия 7

Сопоставьте операции с результатом цикла:

Подсказка: Смотрите, что делает каждая конструкция с блоками пшеницы.

Миссия 8

Расположите шаги для сбора пшеницы с известным количеством рядов:

Подсказка: Сначала инициализация цикла, потом действия, потом завершение.

Миссия 9

Расположите шаги для полива пшеницы до роста:

Подсказка: Сначала проверяем состояние, потом поливаем, затем снова проверяем.

Миссия 10

Расположите шаги для подсчёта блоков в 3 рядах:

Подсказка: Используйте цикл для каждого ряда и суммируйте блоки.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')`. Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 2

Стив запустил терминал в новой пещере и хочет проверить, что Python установлен. Какую команду он должен ввести?

Подсказка: Вспомни: программисты всегда начинают с проверки версии.

Миссия 3

Алекс строит базу данных в мире Minecraft и хочет вывести на экран приветствие для своей команды. Какую функцию Python он должен использовать?

Подсказка: Эта функция самая первая, с неё начинают все программисты.

Миссия 4

Программисты команды открыли портал и написали код: `print('Hello World')` . Но один из игроков использовал вместо одинарных кавычек — квадратные скобки. Что произойдет?

Подсказка: Python очень строг к синтаксису.

Миссия 5

Команда в шахте распределяет задачи: одни пишут команды в Python, другие проверяют их работу. Сопоставь команду с её результатом:

Подсказка: Подумай, что именно будет выведено на экран.

Миссия 6

Стив и Алекс в пещере тестируют разные команды. Соедини команду Python и её описание:

Подсказка: Вспомни разницу между числами и строками.

Миссия 7

Команда программистов построила базу и хочет понять разницу между командами. Соедини команду и то, что произойдет:

Подсказка: Внимательно смотри на кавычки и знаки.

Миссия 8

Игроки запускают сервер и хотят вывести сообщение на экран. Расставь шаги в правильном порядке:

Подсказка: Сначала открываем Python, затем пишем код, и только потом запускаем.

Миссия 9

Программисты строят пайплайн gRPC в шахте. Расставь шаги по установке Python в правильном порядке:

Подсказка: Скачивание → установка → проверка.

Миссия 10

На собрании программистов Minecraft решили: каждый новичок должен вывести первую строку на Python. Расставь действия в правильном порядке:

Подсказка: Подумай о логике: открываем IDE → пишем код → запускаем.

Миссия 1

Стив в Minecraft нашёл первые ресурсы для своего сервера gRPC и хочет сохранить их в переменной. Какой тип данных лучше всего использовать для хранения количества железа?

Подсказка: Подумай, что количество ресурсов — это целое число, а не текст.

Миссия 2

Алекс хочет назвать своё хранилище алмазов и сохранить название в переменной. Какой тип данных нужно использовать?

Подсказка: Подумай, что название — это текст.

Миссия 3

Команда программистов Minecraft собирает ресурсы: камень, дерево, еду. Они хотят узнать, есть ли достаточно ресурсов для постройки базы. Какой тип данных лучше использовать для логической проверки?

Подсказка: Вспомни: True или False помогут понять, хватает ли ресурсов.

Миссия 4

Стив подсчитывает количество блоков в сундуке: 10 камней, 5 золота и 3 алмаза. Какой тип данных нужно использовать для переменной, чтобы сохранить их общее количество?

Подсказка: Общее количество блоков — это число без десятичных знаков.

Миссия 7

Стив ведёт журнал ресурсов. Сопоставьте значение переменной с его типом данных:

Подсказка: Внимательно смотри на кавычки и цифры.

Миссия 8

Алекс готовит сундук с ресурсами. Расставь шаги в правильном порядке:

Подсказка: Сначала создаём переменные, потом присваиваем им значения, и только потом проверяем их типы.

Миссия 9

Стив строит ферму и считает урожай. Расставь шаги вычисления общей суммы:

Подсказка: Сначала узнаём количество каждого ресурса, потом складываем.

Миссия 10

Команда Minecraft готовит отчёт о своих ресурсах. Расставь действия в правильном порядке:

Подсказка: Логика: подготовка переменных → заполнение значений → вывод на экран.

Миссия 1

Стив в Minecraft хочет посчитать, сколько блоков камня и дерева у него в сундуках: камень = 12, дерево = 7. Какую арифметическую операцию он должен использовать, чтобы узнать общее количество блоков?

Подсказка: Подумай, что нужно объединить два количества в одно.

Миссия 2

Алекс крафтит кирку и хочет узнать, сколько инструментов он сможет сделать из 10 железных слитков, если на один инструмент нужно 2 слитка. Какой оператор ему поможет?

Подсказка: Подумай, какая операция покажет, сколько раз число помещается в другое число.

Миссия 3

Команда Minecraft строит башню. На нижнем этаже 5 блоков, на верхнем 3 блока. Какой оператор поможет узнать разницу в количестве блоков между этажами?

Подсказка: Разница — это результат одной величины минус другой.

Миссия 4

Стив делает сундуки. Он хочет удвоить количество дерева, чтобы сделать больше сундуков. Какой оператор поможет ему?

Подсказка: Подумай, как увеличить число в несколько раз.

Миссия 5

Команда Minecraft использует арифметические операции для расчёта ресурсов. Сопоставьте операцию с её описанием:

Подсказка: Подумайте, какая операция объединяет, какая делит, какая уменьшает и какая увеличивает.

Миссия 6

Алекс проверяет свои инструменты. Сопоставьте действие с арифметической операцией:

Подсказка: Вспомните, как считать ресурсы при крафте.

Миссия 7

Команда Minecraft строит ферму. Сопоставьте арифметическую операцию с её практическим использованием:

Подсказка: Подумай, что делают с блоками и инструментами в игре.

Миссия 8

Стив готовит свои ресурсы для крафта. Расставьте шаги в правильном порядке:

Подсказка: Логика: сначала узнаём количество, потом применяем операции, затем сохраняем результат.

Миссия 9

Алекс строит башню и считает блоки. Расставьте действия по порядку:

Подсказка: Сначала узнаём входные данные, затем выполняем операции, потом выводим результат.

Миссия 10

Команда Minecraft крафтит сундуки и инструменты. Расставьте действия в правильном порядке:

Подсказка: Логика: определить ресурсы → применить операции → сохранить или вывести результат.

Миссия 5

Команда Minecraft использует строки для подписей и вывода сообщений. Сопоставьте действие с правильной операцией:

Подсказка: Подумай, что делает строки длиннее, что повторяет текст, а что оставляет неизменным.

Миссия 6

Алекс пишет таблички для сундуков. Сопоставьте, что он хочет сделать, с операцией:

Подсказка: Подумай, как объединять тексты и повторять их для оформления.

Миссия 8

Стив готовит надписи для разных домов. Расставьте шаги в правильном порядке:

Подсказка: Сначала определяем текст, затем объединяем или повторяем, потом сохраняем результат.

Миссия 9

Алекс готовит таблички для фермы. Расставьте действия по порядку:

Подсказка: Сначала готовим текст, потом применяем операции, затем выводим результат.

Миссия 10

Команда Minecraft создаёт вывески для всех строений. Расставьте действия в правильном порядке:

Подсказка: Сначала подготовка текста, затем операции с текстом, потом сохранение или вывод.

Миссия 1

С наступлением первой ночи в Minecraft Стив видит, что за окном темно. Ему нужно решить, ставить ли факел. Какую конструкцию Python использовать, чтобы принять решение?

Подсказка: Подумай, как программа выбирает один путь из двух возможных.

Миссия 2

Алекс хочет проверить, есть ли у него достаточно дерева для постройки кровати. Если досок меньше 3, нужно добыть еще. Какой тип условия использовать?

Подсказка: Подумай, что делать, когда есть два возможных исхода.

Миссия 3

Стив проверяет уровень здоровья. Если здоровье ниже 5, нужно использовать зелье. Как написать условие в Python?

Подсказка: Сначала проверка, потом действие для каждого исхода.

Миссия 4

Алекс решает: если враг близко, атаковать; иначе, строить стену. Какую конструкцию Python использовать?

Подсказка: Условие выбирает один из двух вариантов действий.

Миссия 6

Сопоставьте действия персонажей с логикой условий:

Подсказка: Обдумайте результат каждого условия.

Миссия 8

Расположите шаги принятия решения для Стива в правильном порядке:

Подсказка: Сначала проверяем условие, потом выполняем нужное действие.

Миссия 9

Расположите действия для проверки ресурсов и постройки кровати:

Подсказка: Сначала проверяем количество ресурсов, затем действуем по условию.

Миссия 10

Расположите шаги для защиты дома от мобов ночью:

Подсказка: Проверяем угрозу, потом выбираем действия.

Миссия 5

Сопоставьте действия Minecraft с операциями Python для строк:

Подсказка: Подумайте, что делает каждая операция с текстом.

Миссия 8

Расположите шаги для создания приветствия игрока в правильном порядке:

Подсказка: Сначала нужно определить имя, потом объединить строки, затем вывести результат.

Миссия 9

Расположите шаги для создания названия дома с этажами:

Подсказка: Сначала основа, потом этаж, затем вывод.

Миссия 10

Расположите шаги для повторения приветствия 3 раза:

Подсказка: Сначала создаем текст, затем применяем умножение, затем выводим.

Миссия 1

Стив хочет собрать пшеницу с 5 грядок. Какой цикл лучше использовать, если количество грядок известно заранее?

Подсказка: Подумайте, какой цикл повторяется заданное количество раз.

Миссия 2

Алекс хочет поливать пшеницу до тех пор, пока она не вырастет. Какой цикл лучше использовать, если условие роста неизвестно заранее?

Подсказка: Подумайте, какой цикл зависит от условия, а не от количества повторений.

Миссия 3

Стив хочет подсчитать количество собранных пшеничных блоков из 3 рядов, по 4 блока в каждом. Какой цикл поможет это сделать проще всего?

Подсказка: Счётчик нужен для фиксированного числа повторений.

Миссия 4

Алекс заметил, что некоторые пшеницы ещё не выросли. Какой цикл поможет поливать каждую грядку до роста?

Подсказка: Нужно повторять действие, пока условие не выполнено.

Миссия 6

Сопоставьте описание с правильным примером цикла:

Подсказка: Обратите внимание на известное количество повторений и условие окончания.

Миссия 7

Сопоставьте операции с результатом цикла:

Подсказка: Смотрите, что делает каждая конструкция с блоками пшеницы.

Миссия 8

Расположите шаги для сбора пшеницы с известным количеством рядов:

Подсказка: Сначала инициализация цикла, потом действия, потом завершение.

Миссия 9

Расположите шаги для полива пшеницы до роста:

Подсказка: Сначала проверяем состояние, потом поливаем, затем снова проверяем.

Миссия 10

Расположите шаги для подсчёта блоков в 3 рядах:

Подсказка: Используйте цикл для каждого ряда и суммируйте блоки.

Ответ:

- `for i in range(len(rows))`
- `while rows:`
- `if rows:`
- `for row in rows:`

Урок 7 - Сундук с предметами — списки (list)

Шаг 01:

Миссия 1

Стив открыл сундук и хочет увидеть, сколько предметов в нём лежит. Какой тип данных лучше использовать для хранения предметов?

Подсказка: Списки позволяют хранить несколько элементов в порядке добавления.

Ответ:

- `list`
- `int`
- `str`
- `dict`

Шаг 02:

Миссия 2

Алекс хочет добавить новый меч в сундук. Какой метод списка лучше использовать?

Подсказка: Нужно добавить один элемент в конец списка.

Ответ:

- **append()**
- pop()
- remove()
- clear()

Шаг 03:

Миссия 3

Стив хочет достать первый предмет из сундука без удаления остальных. Как это сделать?

Подсказка: Используйте индекс элемента в списке.

Ответ:

- **items[0]**
- items.pop(0)
- items.remove(0)
- len(items)

Шаг 04:

Миссия 4

Алекс хочет удалить лук из сундука. Какой метод списка лучше использовать?

Подсказка: Метод должен удалять конкретный элемент по значению.

Ответ:

- **remove()**
- append()
- pop()
- clear()

Шаг 05:

Миссия 5

Сопоставьте действия с методами списка:

Подсказка: Подумайте, какой метод добавляет, какой удаляет, а какой возвращает элемент.

Ответ:

- **Добавить новый предмет в конец** — append()
- **Удалить предмет по имени** — remove()
- **Достать предмет по индексу без удаления** — items[0]

Шаг 06:

Миссия 6

Сопоставьте операцию с результатом:

Подсказка: Обратите внимание, изменяется ли список или просто читается элемент.

Ответ:

- **items.append('меч')** — Меч добавлен в конец списка
- **items.remove('лук')** — Лук удалён из списка
- **items[0]** — Возвращает первый предмет, список не меняется

Шаг 07:

Миссия 7

Сопоставьте действие с Python-кодом:

Подсказка: Подумайте, что делает каждый метод и индекс.

Ответ:

- Проверить количество предметов — `len(items)`
- Удалить последний предмет — `items.pop()`
- Добавить стрелы — `items.append('стрелы')`

Шаг 08:

Миссия 8

Расположите шаги для добавления нового предмета в сундук:

Подсказка: Сначала подготовка, потом добавление, потом проверка.

Ответ:

- Подготовить список предметов
- `items.append('меч')`
- Проверить, что предмет добавлен

Шаг 09:

Миссия 9

Расположите шаги для удаления предмета по имени:

Подсказка: Найдите элемент, удалите, проверьте список.

Ответ:

- Проверить список предметов
- `items.remove('лук')`
- Убедиться, что лук удалён

Шаг 10:

Миссия 10

Расположите шаги для проверки первого предмета в сундуке:

Подсказка: Сначала проверить длину, потом достать элемент, потом использовать его.

Ответ:

- `len(items)`
- `items[0]`
- Использовать первый предмет

Урок 8 - Крафт факелов — словари (dict)

Шаг 01:

Миссия 1

Сопоставьте действие с кодом:

Подсказка: Обратите внимание на ключи и значения.

Ответ:

- **Добавить факелы** — `torches['факелы'] = 20`
- **Удалить факелы** — `del torches['факелы']`
- **Посмотреть количество факелов** — `torches['факелы']`

Шаг 02:

Миссия 2

Расположите шаги для добавления нового ресурса в словарь:

Подсказка: Сначала определить ключ, затем присвоить значение, затем проверить словарь.

Ответ:

- Определить ключ для нового ресурса
- Присвоить значение: `torches['уголь'] = 10`
- Проверить, что значение добавлено

Шаг 03:

Миссия 3

Расположите шаги для удаления ресурса из словаря:

Подсказка: Найдите ключ, удалите его, затем проверьте словарь.

Ответ:

- Найти ключ для удаления
- `del torches['дерево']`
- Проверить, что ключ удалён

Шаг 04:

Миссия 4

Стив хочет хранить количество каждого ресурса для крафта факелов. Какой тип данных в Python лучше использовать?

Подсказка: Словарь позволяет связывать ключи с соответствующими значениями.

Ответ:

- **dict**
- list
- set
- tuple

Шаг 05:

Миссия 5

Алекс хочет добавить 10 единиц угля в словарь ресурсов. Как это сделать?

Подсказка: Используйте ключ для ресурса и присвойте значение.

Ответ:

- **`resources['уголь'] = 10`**
- `resources.append('уголь', 10)`
- `resources.add('уголь', 10)`
- `resources.update(10)`

🔗 Шаг 06:

Миссия 6

Стив хочет узнать, сколько факелов можно создать из имеющихся угля и дерева. Как получить значение по ключу?

Подсказка: Используйте имя ключа в квадратных скобках или метод `get()`.

Ответ:

- `resources['уголь']`
- `resources.keys('уголь')`
- `resources.values('уголь')`
- `resources.items('уголь')`

🔗 Шаг 07:

Миссия 7

Сопоставьте действия с методами словаря:

Подсказка: Подумайте, какой метод возвращает пары, а какой удаляет элемент.

Ответ:

- **Получить все ключи** — `keys()`
- **Получить все значения** — `values()`
- **Удалить ресурс по ключу** — `pop()`
- **Получить пары ключ-значение** — `items()`

🔗 Шаг 08:

Миссия 8

Сопоставьте операции с результатом:

Подсказка: Обратите внимание, изменяет ли метод словарь или просто возвращает данные.

Ответ:

- `resources['уголь'] = 20` — Количество угля обновлено
- `resources.pop('дерево')` — Дерево удалено из словаря
- `resources.keys()` — Возвращает список ключей, словарь не меняется

🔗 Шаг 09:

Миссия 9

Сопоставьте действие с Python-кодом:

Подсказка: Подумайте, как проверить ключ, как добавить и как удалить ресурс.

Ответ:

- **Проверить наличие 'уголь'** — `'уголь' in resources`
- **Добавить факелы** — `resources['факел'] = 5`
- **Удалить дерево** — `resources.pop('дерево')`

🔗 Шаг 10:

Миссия 10

Расположите шаги для добавления нового ресурса в словарь:

Подсказка: Сначала выберите ключ, потом присвойте значение, потом проверьте словарь.

Ответ:

- Выбрать ключ для ресурса
- Присвоить значение для ключа
- Проверить, что ресурс добавлен

Шаг 11:

Миссия 11

Расположите шаги для удаления ресурса из словаря:

Подсказка: Сначала убедитесь, что ключ существует, потом удалите, потом проверьте.

Ответ:

- Проверить наличие ключа
- `resources.pop('уголь')`
- Убедиться, что ключ удалён

Шаг 12:

Миссия 12

Стив хочет хранить количество угля и дерева для крафта факелов. Какой тип данных лучше использовать?

Подсказка: Словарь позволяет связывать ключи с соответствующими значениями.

Ответ:

- `dict`
- `list`
- `tuple`
- `set`

Шаг 13:

Миссия 13

Алекс хочет добавить в словарь новое количество факелов. Как правильно это сделать?

Подсказка: Используйте ключ словаря для указания значения.

Ответ:

- `torches['количество'] = 64`
- `torches.append(64)`
- `torches.add(64)`
- `torches.push(64)`

Шаг 14:

Миссия 14

Стив хочет узнать, сколько угля у него есть для крафта. Как это сделать?

Подсказка: Используйте ключ для получения значения из словаря.

Ответ:

- `torches['уголь']`
- `torches.get('дерево')`
- `torches.keys()`
- `len(torches)`

Шаг 15:

Миссия 15

Алекс хочет удалить запись о дереве из словаря. Как это сделать?

Подсказка: Метод должен удалять ключ и его значение.

Ответ:

- **del torches['дерево']**
- torches.remove('дерево')
- torches.popitem()
- torches.clear()

Урок 9 - Случайный спавн мобов — модуль random

Шаг 01:

Миссия 1

Сопоставьте методы модуля random с их назначением:

Методы:

Ответ:

- randint
- choice
- shuffle
- uniform

Назначение:

1. Перемешивает список случайным образом
2. Возвращает случайное целое число между двумя границами
3. Выбирает случайный элемент из последовательности
4. Возвращает случайное число с плавающей точкой между двумя границами

Подсказка: Подумайте, какой метод выбирает элемент, какой перемешивает и какой генерирует числа.

- **randint** — Возвращает случайное целое число между двумя границами
- **choice** — Выбирает случайный элемент из последовательности
- **shuffle** — Перемешивает список случайным образом
- **uniform** — Возвращает случайное число с плавающей точкой между двумя границами

Шаг 02:

Миссия 2

Сопоставьте функции модуля random с их назначением:

Ответ:

- choice()
- randint()
- shuffle()
- sample()

1. Перемешивает элементы списка случайным образом
2. Возвращает случайный элемент из списка

3. Возвращает случайное целое число в диапазоне
4. Возвращает несколько случайных элементов из последовательности

Подсказка: Смотрите на название функции и на смысл слова.

- **choice()** — Возвращает случайный элемент из списка
- **randint()** — Возвращает случайное целое число в диапазоне
- **shuffle()** — Перемешивает элементы списка случайным образом
- **sample()** — Возвращает несколько случайных элементов из последовательности

🔮 Шаг 03:

Миссия 3

Сопоставьте типы мобов с их вероятностью появления:

Ответ:

- Зомби
 - Скелет
 - Крипер
 - Паук
1. 50%
 2. 25%
 3. 15%
 4. 10%

Подсказка: Распределение может быть неравномерным, популярные mobs встречаются чаще.

- **Зомби** — 50%
- **Скелет** — 25%
- **Крипер** — 15%
- **Паук** — 10%

🔮 Шаг 04:

Миссия 4

Сопоставьте функции random с их типом результата:

Ответ:

- randint()
 - uniform()
 - choice()
 - sample()
1. Целое число
 2. Число с плавающей точкой
 3. Элемент последовательности
 4. Список элементов последовательности

Подсказка: Подумайте, что возвращает каждая функция: число, элемент или набор элементов.

- **randint()** — Целое число
- **uniform()** — Число с плавающей точкой
- **choice()** — Элемент последовательности
- **sample()** — Список элементов последовательности

Шаг 05:

Миссия 5

Расположите шаги случайного спавна зомби:

1. Определяем локацию спавна
2. Выбираем тип моба с помощью `random.choice()`
3. Выбираем количество мобов с помощью `random.randint()`
4. Создаём объекты мобов
5. Добавляем их в игровую локацию

Подсказка: Сначала место, потом что и сколько, затем создание и добавление.

Ответ:

- Определяем локацию спавна
- Выбираем тип моба с помощью `random.choice()`
- Выбираем количество мобов с помощью `random.randint()`
- Создаём объекты мобов
- Добавляем их в игровую локацию

Шаг 06:

Миссия 6

Расположите шаги для перемешивания списка мобов перед спавном:

1. Создаём список мобов
2. Вызываем `random.shuffle()` для списка
3. Спавним мобов по порядку
4. Обновляем состояние локации

Подсказка: Сначала список, потом перемешивание, затем спавн и обновление состояния.

Ответ:

- Создаём список мобов
- Вызываем `random.shuffle()` для списка
- Спавним мобов по порядку
- Обновляем состояние локации

Шаг 07:

Миссия 7

Расположите шаги для случайного выбора нескольких мобов для рейда:

1. Создаём список всех мобов
2. Выбираем случайные 3 моба с помощью `random.sample()`
3. Отображаем выбранных мобов игроку
4. Запускаем их действия в игровом цикле

Подсказка: Сначала список, потом выбор, отображение и запуск действий.

Ответ:

- Создаём список всех мобов
- Выбираем случайные 3 моба с помощью `random.sample()`
- Отображаем выбранных мобов игроку
- Запускаем их действия в игровом цикле

Шаг 08:

Миссия 8

Расположите шаги генерации случайного моба с вероятностью редкости:

1. Определяем шанс редкого моба
2. Генерируем случайное число с помощью `random.random()`
3. Сравниваем число с шансом
4. Если число меньше шанса, выбираем редкого моба, иначе обычного
5. Добавляем моба в локацию

Подсказка: Сначала шанс, затем генерация числа, проверка условия и добавление.

Ответ:

- Определяем шанс редкого моба
- Генерируем случайное число с помощью `random.random()`
- Сравниваем число с шансом
- Если число меньше шанса, выбираем редкого моба, иначе обычного
- Добавляем моба в локацию

Шаг 09:

Миссия 9

Сопоставьте типы мобов с их особенностями в Minecraft:

Ответ:

- Зомби
 - Скелет
 - Крипер
 - Паук
1. Взрывается при приближении игрока
 2. Стреляет из лука
 3. Преследует игрока и атакует вблизи
 4. Лезет по стенам и может прыгать на игрока

Подсказка: Вспомните особенности поведения каждого моба в игре.

- **Зомби** — Преследует игрока и атакует вблизи
- **Скелет** — Стреляет из лука
- **Крипер** — Взрывается при приближении игрока
- **Паук** — Лезет по стенам и может прыгать на игрока

Шаг 10:

Миссия 10

Сопоставьте функции модуля `random` с результатами:

Ответ:

- `randint(1,6)`
 - `uniform(0,1)`
 - `choice(['золото', 'железо', 'алмаз'])`
 - `shuffle(['a','б','в'])`
1. Случайный элемент из списка

2. Перемешивание списка
3. Случайное целое число от 1 до 6
4. Случайное число с плавающей точкой между 0 и 1

Подсказка: Смотрите на диапазоны и тип возвращаемого значения.

- **randint(1,6)** — Случайное целое число от 1 до 6
- **uniform(0,1)** — Случайное число с плавающей точкой между 0 и 1
- **choice(['золото', 'железо', 'алмаз'])** — Случайный элемент из списка
- **shuffle(['a','б','в'])** — Перемешивание списка

Шаг 11:

Миссия 11

Расположите шаги случайного спавна мобов в правильном порядке:

1. Определить диапазон координат для спавна
2. Выбрать случайный тип моба
3. Сгенерировать случайные координаты
4. Создать объект моба в мире

Подсказка: Подумайте, сначала где спавн возможен, потом выбираем кого спавнить, затем координаты, и в конце — появление в мире.

Ответ:

- Определить диапазон координат для спавна
- Выбрать случайный тип моба
- Сгенерировать случайные координаты
- Создать объект моба в мире

Шаг 12:

Миссия 12

Расположите шаги случайного выпадения лута из моба:

1. Определяем список возможного лута
2. Перемешиваем список или выбираем случайный предмет
3. Определяем количество предметов
4. Выдаём предметы игроку

Подсказка: Сначала что есть в луте, потом случайный выбор, затем количество и выдача.

Ответ:

- Определяем список возможного лута
- Перемешиваем список или выбираем случайный предмет
- Определяем количество предметов
- Выдаём предметы игроку

Шаг 13:

Миссия 13

Расположите шаги создания случайного события в мире Minecraft:

1. Определяем тип события (спавн моба, выпадение лута, изменение погоды)
2. Выбираем случайное время или координаты для события

3. Используем random для генерации чисел
4. Запускаем событие в мире

Подсказка: Сначала планируем событие, потом случайные параметры, генерация и запуск.

Ответ:

- Определяем тип события (спавн моба, выпадение лута, изменение погоды)
- Выбираем случайное время или координаты для события
- Используем random для генерации чисел
- Запускаем событие в мире

Шаг 14:

Миссия 14

Расположите этапы генерации случайного мини-босса с уникальными свойствами:

1. Выбираем уникальный тип моба
2. Генерируем случайные координаты появления
3. Присваиваем случайные свойства (здоровье, урон, эффекты)
4. Создаём объект босса в мире
5. Запускаем событие и отображаем его игрокам

Подсказка: Сначала выбор типа, потом место, свойства, создание и показ игрокам.

Ответ:

- Выбираем уникальный тип моба
- Генерируем случайные координаты появления
- Присваиваем случайные свойства (здоровье, урон, эффекты)
- Создаём объект босса в мире
- Запускаем событие и отображаем его игрокам

Урок 10 - Сражение с зомби — функции

Шаг 01:

Миссия 1

Сопоставьте действия функции с её результатом при атаке зомби:

Ответ:

- Функция attack(damage)
 - Функция heal(amount)
 - Функция flee()
 - Функция critical_hit()
1. Уменьшает здоровье зомби на указанное количество
 2. Увеличивает здоровье игрока или моба
 3. Увеличивает урон в случае критического удара
 4. Позволяет игроку уйти от боя

Подсказка: Подумайте, что делает каждая функция исходя из её имени.

- **Функция attack(damage)** — Уменьшает здоровье зомби на указанное количество
- **Функция heal(amount)** — Увеличивает здоровье игрока или моба
- **Функция flee()** — Позволяет игроку уйти от боя
- **Функция critical_hit()** — Увеличивает урон в случае критического удара

Шаг 02:

Миссия 2

Сопоставьте термины функций с их назначением:

Ответ:

- def
- return
- параметр
- аргумент

1. Объявление функции
2. Возврат значения из функции
3. Переменная внутри определения функции
4. Значение, передаваемое функции при вызове

Подсказка: Вспомните синтаксис объявления и вызова функции.

- **def** — Объявление функции
- **return** — Возврат значения из функции
- **параметр** — Переменная внутри определения функции
- **аргумент** — Значение, передаваемое функции при вызове

Шаг 03:

Миссия 3

Сопоставьте типы функций с их примерами:

Ответ:

- Функция без параметров
- Функция с параметрами
- Функция, возвращающая значение
- Функция, ничего не возвращающая

1. `def attack(): ...`
2. `def attack(mob_type): ...`
3. `def damage(mob): return mob.hp`
4. `def print_attack(): print("Атака!")`

Подсказка: Смотрите на наличие параметров и return.

- **Функция без параметров** — `def attack(): ...`
- **Функция с параметрами** — `def attack(mob_type): ...`
- **Функция, возвращающая значение** — `def damage(mob): return mob.hp`
- **Функция, ничего не возвращающая** — `def print_attack(): print("Атака!")`

Шаг 04:

Миссия 4

Сопоставьте этапы вызова функции с их описанием:

Ответ:

- Определение функции
- Передача аргументов
- Выполнение тела функции

- Возврат результата

1. Создаём код функции с def
2. Передаём значения параметрам
3. Происходит выполнение команд внутри функции
4. Возвращаемое значение можно использовать вне функции

Подсказка: Сначала определяем, затем вызываем с аргументами, выполняем и возвращаем результат.

- **Определение функции** — Создаём код функции с def
- **Передача аргументов** — Передаём значения параметрам
- **Выполнение тела функции** — Происходит выполнение команд внутри функции
- **Возврат результата** — Возвращаемое значение можно использовать вне функции

Шаг 05:

Миссия 5

Расположите шаги написания функции атаки зомби:

1. Определяем функцию с def
2. Добавляем параметры для типа моба и урона
3. Пишем тело функции с действиями
4. Возвращаем результат урона
5. Вызываем функцию в игровом цикле

Подсказка: Сначала объявление, параметры, тело, return, вызов.

Ответ:

- Определяем функцию с def
- Добавляем параметры для типа моба и урона
- Пишем тело функции с действиями
- Возвращаем результат урона
- Вызываем функцию в игровом цикле

Шаг 06:

Миссия 6

Расположите шаги от простого вызова функции до использования её результата для атаки:

1. Определяем функцию calculate_damage(mob)
2. Вызов функции с конкретным мобом
3. Получаем результат урона
4. Применяем урон к мобу

Подсказка: Сначала определение, потом вызов, результат и применение.

Ответ:

- Определяем функцию calculate_damage(mob)
- Вызов функции с конкретным мобом
- Получаем результат урона
- Применяем урон к мобу

Шаг 07:

Миссия 7

Расположите шаги для создания функции массовой атаки на группу зомби:

1. Определяем функцию `attack_group(zombies, damage)`
2. Проходим по списку зомби
3. Вычисляем урон каждому зомби с помощью `calculate_damage()`
4. Применяем урон
5. Возвращаем общий урон нанесённый группе

Подсказка: Сначала объявление, затем цикл, вычисления, применение и результат.

Ответ:

- Определяем функцию `attack_group(zombies, damage)`
- Проходим по списку зомби
- Вычисляем урон каждому зомби с помощью `calculate_damage()`
- Применяем урон
- Возвращаем общий урон нанесённый группе

Шаг 08:

Миссия 8

Расположите шаги от создания функции атаки до использования в игровом цикле:

1. Определяем функцию с параметрами типа и силы атаки
2. Пишем тело функции с логикой нанесения урона
3. Возвращаем значение урона
4. В игровом цикле выбираем моба
5. Вызываем функцию и применяем результат

Подсказка: Сначала функция полностью, затем вызов и применение в цикле.

Ответ:

- Определяем функцию с параметрами типа и силы атаки
- Пишем тело функции с логикой нанесения урона
- Возвращаем значение урона
- В игровом цикле выбираем моба
- Вызываем функцию и применяем результат

Шаг 09:

Миссия 9

Сопоставьте аргументы функции с их описанием:

Ответ:

- `damage`
 - `health`
 - `weapon`
 - `critical`
1. Тип оружия, которым атакует игрок
 2. Количество здоровья, которое есть у моба
 3. Наносимый урон при атаке

4. Флаг критического удара

Подсказка: Имена аргументов обычно отражают их назначение.

- **damage** — Наносимый урон при атаке
- **health** — Количество здоровья, которое есть у моба
- **weapon** — Тип оружия, которым атакует игрок
- **critical** — Флаг критического удара

Шаг 10:

Миссия 10

Сопоставьте типы функций с их примером:

Ответ:

- Функция без параметров
- Функция с параметрами
- Функция с возвращаемым значением
- Функция с побочным эффектом

1. `print('Бросаю зелье!')`
2. `def attack(damage): ...`
3. `def calculate_damage(): return 10`
4. `def heal(player): player.health += 10`

Подсказка: Смотрите на наличие параметров и возвращаемого значения.

- **Функция без параметров** — `print('Бросаю зелье!')`
- **Функция с параметрами** — `def attack(damage): ...`
- **Функция с возвращаемым значением** — `def calculate_damage(): return 10`
- **Функция с побочным эффектом** — `def heal(player): player.health += 10`

Шаг 11:

Миссия 11

Расположите шаги вызова функции атаки зомби:

1. Определяем количество урона
2. Вызываем функцию `attack` с этим параметром
3. Функция уменьшает здоровье зомби
4. Отображаем результат игроку

Подсказка: Сначала готовим параметры, затем вызываем функцию, потом изменения и вывод результата.

Ответ:

- Определяем количество урона
- Вызываем функцию `attack` с этим параметром
- Функция уменьшает здоровье зомби
- Отображаем результат игроку

Шаг 12:

Миссия 12

Расположите шаги критического удара в правильном порядке:

1. Определяем вероятность критического удара
2. Проверяем, сработал ли критический удар
3. Если да, умножаем урон на коэффициент
4. Применяем урон к здоровью зомби
5. Выводим результат на экран

Подсказка: Сначала проверяем шанс, затем изменения урона и применяем эффект.

Ответ:

- Определяем вероятность критического удара
- Проверяем, сработал ли критический удар
- Если да, умножаем урон на коэффициент
- Применяем урон к здоровью зомби
- Выводим результат на экран

Шаг 13:

Миссия 13

Расположите шаги функции лечения игрока:

1. Игрок использует зелье
2. Функция heal получает объект игрока
3. Увеличиваем здоровье на определённое значение
4. Проверяем, не превышено ли максимальное здоровье
5. Отображаем новое здоровье игрока

Подсказка: Сначала действие игрока, затем функция, изменение состояния и проверка ограничения, вывод результата.

Ответ:

- Игрок использует зелье
- Функция heal получает объект игрока
- Увеличиваем здоровье на определённое значение
- Проверяем, не превышено ли максимальное здоровье
- Отображаем новое здоровье игрока

Шаг 14:

Миссия 14

Расположите шаги вызова функции атаки нескольких зомби подряд:

1. Создаём список всех зомби в локации
2. Для каждого зомби определяем наносимый урон
3. Вызываем функцию attack для каждого зомби
4. Обновляем состояние каждого зомби
5. Отображаем результаты игроку

Подсказка: Сначала список объектов, потом цикл с вычислением урона, применение функции, обновление состояния и вывод.

Ответ:

- Создаём список всех зомби в локации
- Для каждого зомби определяем наносимый урон

- Вызываем функцию attack для каждого зомби
- Обновляем состояние каждого зомби
- Отображаем результаты игроку

Урок 11 - Поход в пещеру — ввод пользователя (input)

Шаг 01:

Миссия 1

Сопоставьте действия с функциями ввода и обработки данных:

Ответ:

- Получение текста от игрока
- Преобразование текста в число
- Вывод результата
- Сохранение ввода в переменную

1. input()
2. int()
3. print()
4. =

Подсказка: Думайте о порядке действий: запрос → преобразование → сохранение → вывод.

- **Получение текста от игрока** — input()
- **Преобразование текста в число** — int()
- **Вывод результата** — print()
- **Сохранение ввода в переменную** — =

Шаг 02:

Миссия 2

Сопоставьте типы ввода с примером:

Ответ:

- Строка
- Целое число
- Дробное число
- Булев тип

1. input()
2. int(input())
3. float(input())
4. bool(input())

Подсказка: Вспомните, как преобразовать ввод для разных типов данных.

- **Строка** — input()
- **Целое число** — int(input())
- **Дробное число** — float(input())
- **Булев тип** — bool(input())

Шаг 03:

Миссия 3

Сопоставьте действия игрока в пещере с обработкой ввода:

Ответ:

- Выбирает оружие
- Указывает количество факелов
- Отвечает на вопрос о направлении
- Выводит информацию о состоянии здоровья

1. input() + сохранение в переменную
2. int(input())
3. input()
4. print()

Подсказка: Определите тип данных для каждого действия игрока.

- **Выбирает оружие** — input() + сохранение в переменную
- **Указывает количество факелов** — int(input())
- **Отвечает на вопрос о направлении** — input()
- **Выводит информацию о состоянии здоровья** — print()

Шаг 04:

Миссия 4

Расположите шаги обработки ввода игрока о количестве факелов:

1. Запросить ввод с помощью input()
2. Преобразовать ввод в число с int()
3. Сохранить результат в переменную
4. Использовать значение для расчета длительности похода

Подсказка: Сначала запрос, затем преобразование, сохранение и применение.

Ответ:

- Запросить ввод с помощью input()
- Преобразовать ввод в число с int()
- Сохранить результат в переменную
- Использовать значение для расчета длительности похода

Шаг 05:

Миссия 5

Расположите шаги, чтобы игрок выбрал оружие и получил уведомление:

1. Выводим список оружия через print()
2. Запрашиваем выбор через input()
3. Сохраняем выбор в переменную
4. Выводим подтверждение выбора через print()

Подсказка: Сначала показываем варианты, затем ввод, сохранение и подтверждение.

Ответ:

- Выводим список оружия через print()
- Запрашиваем выбор через input()
- Сохраняем выбор в переменную
- Выводим подтверждение выбора через print()

Шаг 06:

Миссия 6

Расположите шаги для безопасного ввода координат для похода:

1. Запросить координаты через `input()`
2. Разделить строку на части с `split()`
3. Преобразовать каждую часть в число с `int()`
4. Сохранить координаты в список или кортеж
5. Использовать координаты для передвижения игрока

Подсказка: Сначала ввод, потом разбор строки, преобразование, сохранение и использование.

Ответ:

- Запросить координаты через `input()`
- Разделить строку на части с `split()`
- Преобразовать каждую часть в число с `int()`
- Сохранить координаты в список или кортеж
- Использовать координаты для передвижения игрока

Шаг 07:

Миссия 7

Расположите шаги для обработки ввода нескольких игроков для похода в пещеру:

1. Запросить имена игроков через `input()`
2. Сохранить их в список
3. Вывести список участников через `print()`
4. Проверить корректность имен (не пустые)
5. Начать поход с корректными участниками

Подсказка: Сначала ввод, сохранение, проверка и вывод, затем использование.

Ответ:

- Запросить имена игроков через `input()`
- Сохранить их в список
- Вывести список участников через `print()`
- Проверить корректность имен (не пустые)
- Начать поход с корректными участниками

Шаг 08:

Миссия 8

Сопоставьте термины с их объяснением в контексте ввода данных в пещере:

Ответ:

- `input()`
 - `int()`
 - `str()`
 - `float()`
1. Позволяет пользователю ввести данные с клавиатуры
 2. Преобразует строку в целое число
 3. Преобразует данные в строку

4. Преобразует данные в число с плавающей точкой

Подсказка: Смотрите, какие типы данных нужны для различных случаев.

- **input()** — Позволяет пользователю ввести данные с клавиатуры
- **int()** — Преобразует строку в целое число
- **str()** — Преобразует данные в строку
- **float()** — Преобразует данные в число с плавающей точкой

Шаг 09:

Миссия 9

Сопоставьте примеры ввода с их результатом:

Ответ:

- `name = input("Имя героя: ")`
- `gold = int(input("Золото: "))`
- `height = float(input("Рост: "))`
- `description = str(input("Описание: "))`

1. Строка с именем героя
2. Целое число с количеством золота
3. Число с плавающей точкой с ростом
4. Строка с описанием

Подсказка: Посмотрите на тип данных, который нужно получить из `input()`.

- **`name = input("Имя героя: ")`** — Строка с именем героя
- **`gold = int(input("Золото: "))`** — Целое число с количеством золота
- **`height = float(input("Рост: "))`** — Число с плавающей точкой с ростом
- **`description = str(input("Описание: "))`** — Строка с описанием

Шаг 10:

Миссия 10

Сопоставьте варианты обработки ошибок при вводе данных:

Ответ:

- `try / except`
- Проверка с `isnumeric()`
- Проверка с `isalpha()`
- Использование `int()` с проверкой

1. Ловим исключения при неправильном вводе
2. Проверяем, что строка состоит только из цифр
3. Проверяем, что строка состоит только из букв
4. Преобразуем строку в число с безопасной проверкой

Подсказка: Подумайте, как можно безопасно получить число от пользователя.

- **`try / except`** — Ловим исключения при неправильном вводе
- **Проверка с `isnumeric()`** — Проверяем, что строка состоит только из цифр
- **Проверка с `isalpha()`** — Проверяем, что строка состоит только из букв
- **Использование `int()` с проверкой** — Преобразуем строку в число с безопасной проверкой

Шаг 11:

Миссия 11

Расположите шаги обработки числового ввода от игрока:

1. Спросить количество золота с помощью `input()`
2. Проверить, что введённая строка состоит из цифр
3. Преобразовать строку в `int`
4. Использовать число в игре

Подсказка: Сначала получаем данные, затем проверяем, потом преобразуем и используем.

Ответ:

- Спросить количество золота с помощью `input()`
- Проверить, что введённая строка состоит из цифр
- Преобразовать строку в `int`
- Использовать число в игре

Шаг 12:

Миссия 12

Расположите шаги для ввода и обработки роста героя:

1. Получаем строку с помощью `input()`
2. Проверяем, что строка может быть преобразована в `float`
3. Преобразуем строку в `float`
4. Используем значение роста в игре

Подсказка: Сначала получаем данные, проверяем корректность, преобразуем и используем.

Ответ:

- Получаем строку с помощью `input()`
- Проверяем, что строка может быть преобразована в `float`
- Преобразуем строку в `float`
- Используем значение роста в игре

Шаг 13:

Миссия 13

Расположите шаги для безопасного ввода количества зельев:

1. Спросить число с помощью `input()`
2. Проверить, что строка состоит из цифр
3. Преобразовать в `int`
4. Ловим исключения через `try/except`
5. Использовать число в игровом цикле

Подсказка: Сначала запрашиваем данные, проверяем, преобразуем и защищаемся от ошибок.

Ответ:

- Спросить число с помощью `input()`
- Проверить, что строка состоит из цифр
- Преобразовать в `int`
- Ловим исключения через `try/except`
- Используем число в игровом цикле

Шаг 14:

Миссия 14

Расположите шаги для ввода описания найденного артефакта и использования его в игре:

1. Запросить описание с помощью `input()`
2. Проверить, что введенные данные не пустые
3. Преобразовать при необходимости в `str()`
4. Вывести описание на экран или сохранить в переменную

Подсказка: Сначала получаем данные, проверяем корректность, преобразуем и используем.

Ответ:

- Запросить описание с помощью `input()`
- Проверить, что введенные данные не пустые
- Преобразовать при необходимости в `str()`
- Вывести описание на экран или сохранить в переменную

Урок 12 - Мини-босс «Паук» — логические операции

Шаг 01:

Миссия 1

Сопоставьте логические операторы с их действием в бою с пауком:

Ответ:

- `and`
 - `or`
 - `not`
 - `==`
1. Оба условия должны быть истинными
 2. Хотя бы одно условие истинно
 3. Отрицает условие
 4. Проверяет равенство

Подсказка: Вспомните правила логики, которые помогут принимать решения в бою.

- **`and`** — Оба условия должны быть истинными
- **`or`** — Хотя бы одно условие истинно
- **`not`** — Отрицает условие
- **`==`** — Проверяет равенство

Шаг 02:

Миссия 2

Сопоставьте примеры условий с их смыслом:

Ответ:

- `health > 50 and arrows > 0`
 - `health <= 50 or shield == True`
 - `not under_attack`
 - `mana == 100`
1. Оба условия для безопасной атаки

2. Одно из условий для защиты или слабого здоровья
3. Герой не под атакой
4. Полная мана

Подсказка: Вспомните, как работают and, or, not и == в боевых сценариях.

- **health > 50 and arrows > 0** — Оба условия для безопасной атаки
- **health <= 50 or shield == True** — Одно из условий для защиты или слабого здоровья
- **not under_attack** — Герой не под атакой
- **mana == 100** — Полная мана

Шаг 03:

Миссия 3

Сопоставьте логические комбинации с возможной тактикой в бою:

Ответ:

- (health > 70) and (arrows >= 5)
- (health < 30) or (mana < 20)
- not stunned
- (shield_active == True) and (enemy_near == False)

1. Атака с гарантией успеха
2. Использовать зелье лечения или ману
3. Можно двигаться или атаковать
4. Безопасно стоять и готовиться к следующей атаке

Подсказка: Логика помогает принимать стратегические решения.

- **(health > 70) and (arrows >= 5)** — Атака с гарантией успеха
- **(health < 30) or (mana < 20)** — Использовать зелье лечения или ману
- **not stunned** — Можно двигаться или атаковать
- **(shield_active == True) and (enemy_near == False)** — Безопасно стоять и готовиться к следующей атаке

Шаг 04:

Миссия 4

Расположите шаги логической проверки перед атакой мини-босса:

1. Проверить здоровье героя
2. Проверить наличие стрел
3. Проверить, что враг в зоне атаки
4. Принять решение атаковать или использовать защиту

Подсказка: Сначала собираем данные, потом принимаем решение.

Ответ:

- Проверить здоровье героя
- Проверить наличие стрел
- Проверить, что враг в зоне атаки
- Принять решение атаковать или использовать защиту

Шаг 05:

Миссия 5

Расположите шаги для безопасного обхода паука:

1. Проверить наличие ловушек
2. Проверить направление движения паука
3. Использовать логические условия для выбора пути
4. Двигаться безопасно

Подсказка: Логика и проверка условий помогают избегать урона.

Ответ:

- Проверить наличие ловушек
- Проверить направление движения паука
- Использовать логические условия для выбора пути
- Двигаться безопасно

Шаг 06:

Миссия 6

Расположите шаги для комбинированной атаки с проверкой условий:

1. Проверить здоровье героя
2. Проверить наличие стрел и зелья силы
3. Проверить, что паук не вблизи героя
4. Атаковать, если все условия выполнены

Подсказка: Проверка всех условий перед атакой увеличивает шанс победы.

Ответ:

- Проверить здоровье героя
- Проверить наличие стрел и зелья силы
- Проверить, что паук не вблизи героя
- Атаковать, если все условия выполнены

Шаг 07:

Миссия 7

Расположите шаги для безопасного использования зелья и атаки паука:

1. Проверить, что здоровье ниже 50
2. Проверить наличие зелья лечения
3. Принять решение использовать зелье или атаковать
4. Выполнить выбранное действие

Подсказка: Сначала проверяем условия, потом действуем.

Ответ:

- Проверить, что здоровье ниже 50
- Проверить наличие зелья лечения
- Принять решение использовать зелье или атаковать
- Выполнить выбранное действие

Шаг 08:

Миссия 8

Сопоставьте логические операторы с их описанием:

Ответ:

- and
 - or
 - not
1. Возвращает True, если оба условия истинны
 2. Возвращает True, если хотя бы одно условие истинно
 3. Меняет значение на противоположное

Подсказка: Подумайте, когда вам нужно все условия, а когда хотя бы одно.

- **and** — Возвращает True, если оба условия истинны
- **or** — Возвращает True, если хотя бы одно условие истинно
- **not** — Меняет значение на противоположное

Шаг 09:

Миссия 9

Сопоставьте сценарии боя с правильными логическими условиями:

Ответ:

- Атаковать только если паук агрессивный и ядовитый
 - Атаковать если паук агрессивный или ядовитый
 - Отступить, если паук не агрессивный
1. and
 2. or
 3. not

Подсказка: Подумайте, когда требуется соблюсти оба условия, а когда достаточно одного.

- **Атаковать только если паук агрессивный и ядовитый** — and
- **Атаковать если паук агрессивный или ядовитый** — or
- **Отступить, если паук не агрессивный** — not

Шаг 10:

Миссия 10

Расположите шаги логической проверки перед атакой паука:

1. Проверить, ядовитый ли паук
2. Проверить, агрессивный ли паук
3. Объединить условия с помощью and или or
4. Решить атаковать или отступить

Подсказка: Сначала проверяем характеристики, затем объединяем и принимаем решение.

Ответ:

- Проверить, ядовитый ли паук
- Проверить, агрессивный ли паук
- Объединить условия с помощью and или or

- Решить атаковать или отступить

Шаг 11:

Миссия 11

Расположите действия для безопасного боя с пауком:

1. Считать статус агрессии и ядовитости
2. Применить not для безопасных условий
3. Объединить условия с and/or
4. Принять решение атаковать или отступить

Подсказка: Сначала получаем информацию, проверяем на опасность, объединяем и действуем.

Ответ:

- Считать статус агрессии и ядовитости
- Применить not для безопасных условий
- Объединить условия с and/or
- Принять решение атаковать или отступить

Шаг 12:

Миссия 12

Расположите шаги для использования нескольких логических проверок в бою:

1. Проверить, есть ли щит у героя
2. Проверить, агрессивен ли паук
3. Использовать not для проверки угрозы
4. Объединить условия через and/or
5. Принять решение об атаке

Подсказка: Сначала проверяем характеристики героя и врага, затем объединяем и действуем.

Ответ:

- Проверить, есть ли щит у героя
- Проверить, агрессивен ли паук
- Использовать not для проверки угрозы
- Объединить условия через and/or
- Принять решение об атаке

Шаг 13:

Миссия 13

Расположите шаги для комплексного боя с несколькими пауками:

1. Проверить статус агрессии каждого паука
2. Объединить условия через or для атакующих
3. Использовать not для безопасных зон
4. Принять решение атаковать или отступить
5. Атаковать выбранного паука

Подсказка: Сначала оцениваем угрозу всех пауков, затем действуем безопасно и эффективно.

Ответ:

- Проверить статус агрессии каждого паука

- Объединить условия через `or` для атакующих
- Использовать `not` для безопасных зон
- Принять решение атаковать или отступить
- Атаковать выбранного паука

Модуль 2 - Уровень «Nether» (Middle)

Урок 1 - Первая шахта — работа с файлами (`open`, `read`, `write`)

Шаг 01:

Миссия 1

Сопоставьте режимы открытия файла с их описанием:

Ответ:

- `'r'`
- `'w'`
- `'a'`
- `'x'`

1. Чтение содержимого
2. Запись с удалением старого содержимого
3. Добавление данных в конец файла
4. Создать файл, если его нет, иначе ошибка

Подсказка: Подумайте, когда используете `read`, `write`, `append` и `create`.

- `'r'` — Чтение содержимого
- `'w'` — Запись с удалением старого содержимого
- `'a'` — Добавление данных в конец файла
- `'x'` — Создать файл, если его нет, иначе ошибка

Шаг 02:

Миссия 2

Сопоставьте методы работы с файлами с их действием:

Ответ:

- `read()`
- `readline()`
- `write()`
- `close()`

1. Читает весь файл целиком
2. Читает одну строку
3. Записывает строку в файл
4. Закрывает файл

Подсказка: Подумайте, что делает каждый метод и когда он нужен.

- **`read()`** — Читает весь файл целиком
- **`readline()`** — Читает одну строку
- **`write()`** — Записывает строку в файл
- **`close()`** — Закрывает файл

Шаг 03:

Миссия 3

Сопоставьте ситуации в шахте с правильным действием с файлами:

Ответ:

- Сохранить найденные материалы
- Прочитать карту шахты
- Добавить новые заметки к журналу
- Создать файл для дневника исследований

1. 'w'
2. 'r'
3. 'a'
4. 'x'

Подсказка: Подумайте, когда вы хотите перезаписать, когда просто читать или добавлять.

- **Сохранить найденные материалы** — 'w'
- **Прочитать карту шахты** — 'r'
- **Добавить новые заметки к журналу** — 'a'
- **Создать файл для дневника исследований** — 'x'

Шаг 04:

Миссия 4

Расположите шаги чтения файла:

1. Открыть файл в режиме чтения
2. Прочитать данные с помощью read() или readlines()
3. Обработать данные
4. Закрыть файл

Подсказка: Сначала откройте файл, потом читайте, затем используйте данные и закрывайте.

Ответ:

- Открыть файл в режиме чтения
- Прочитать данные с помощью read() или readlines()
- Обработать данные
- Закрыть файл

Шаг 05:

Миссия 5

Расположите шаги записи данных в файл:

1. Открыть файл в режиме 'w' или 'a'
2. Записать данные с помощью write()
3. Обработать данные для записи
4. Закрыть файл

Подсказка: Подготовка данных идет перед записью, затем закрытие.

Ответ:

- Открыть файл в режиме 'w' или 'a'

- Обработать данные для записи
- Записать данные с помощью write()
- Закрывать файл

Шаг 06:

Миссия 6

Расположите шаги безопасного обновления журнала шахты:

1. Открыть файл в режиме 'r' и прочитать старые записи
2. Обработать прочитанные данные
3. Открыть файл в режиме 'a' и добавить новые записи
4. Закрывать файл

Подсказка: Сначала прочитайте, затем добавьте новые записи и закройте файл.

Ответ:

- Открыть файл в режиме 'r' и прочитать старые записи
- Обработать прочитанные данные
- Открыть файл в режиме 'a' и добавить новые записи
- Закрывать файл

Шаг 07:

Миссия 7

Расположите шаги комплексной работы с файлами в шахте:

1. Создать файл дневника исследований ('x')
2. Открыть файл для добавления заметок ('a')
3. Записать новые заметки
4. Закрывать файл
5. Открыть файл для чтения ('r') и проверить содержимое

Подсказка: Сначала создаем, затем добавляем данные, потом проверяем их.

Ответ:

- Создать файл дневника исследований ('x')
- Открыть файл для добавления заметок ('a')
- Записать новые заметки
- Закрывать файл
- Открыть файл для чтения ('r') и проверить содержимое

Шаг 08:

Миссия 8

Сопоставьте режимы открытия файлов с их назначением:

Ответ:

- 'r'
 - 'w'
 - 'a'
 - 'x'
1. Открыть для чтения, файл должен существовать
 2. Открыть для записи, перезаписывает существующий

3. Открыть для добавления, новые данные дописываются в конец

4. Создать новый файл, ошибка если файл существует

Подсказка: Подумайте, какой режим используется для чтения, записи или добавления.

- **'r'** — Открыть для чтения, файл должен существовать
- **'w'** — Открыть для записи, перезаписывает существующий
- **'a'** — Открыть для добавления, новые данные дописываются в конец
- **'x'** — Создать новый файл, ошибка если файл существует

Шаг 09:

Миссия 9

Сопоставьте методы файлов с их действием:

Ответ:

- `read()`
 - `readline()`
 - `write()`
 - `close()`
1. Читает весь файл целиком
 2. Читает одну строку файла
 3. Записывает строку в файл
 4. Закрывает файл

Подсказка: Помните, что после работы с файлом его всегда нужно закрывать.

- **`read()`** — Читает весь файл целиком
- **`readline()`** — Читает одну строку файла
- **`write()`** — Записывает строку в файл
- **`close()`** — Закрывает файл

Шаг 10:

Миссия 10

Сопоставьте ситуации с действиями в Python:

Ответ:

- Нужно сохранить список ресурсов
 - Прочитать карту шахты
 - Добавить новую находку к существующему файлу
 - Создать файл отчёта только если его нет
1. `write()`
 2. `read()`
 3. `open` с `'a'`
 4. `open` с `'x'`

Подсказка: Сначала подумайте о режиме открытия файла, затем о методах.

- **Нужно сохранить список ресурсов** — `write()`
- **Прочитать карту шахты** — `read()`
- **Добавить новую находку к существующему файлу** — `open` с `'a'`
- **Создать файл отчёта только если его нет** — `open` с `'x'`

Шаг 11:

Миссия 11

Расположите шаги записи новой карты шахты в файл:

1. Открыть файл для записи
2. Записать данные методом `write()`
3. Заккрыть файл

Подсказка: Сначала открываем, затем записываем, потом закрываем.

Ответ:

- Открыть файл для записи
- Записать данные методом `write()`
- Заккрыть файл

Шаг 12:

Миссия 12

Расположите шаги чтения файла с картой шахты по строкам:

1. Открыть файл для чтения
2. Считать строку методом `readline()`
3. Обработать данные
4. Заккрыть файл

Подсказка: Чтение по строкам требует цикла, но базовые шаги остаются: открыть, читать, обработать, закрыть.

Ответ:

- Открыть файл для чтения
- Считать строку методом `readline()`
- Обработать данные
- Заккрыть файл

Шаг 13:

Миссия 13

Расположите действия при добавлении новых ресурсов в существующий файл:

1. Открыть файл в режиме 'a'
2. Добавить новые данные с `write()`
3. Заккрыть файл

Подсказка: Добавление похоже на запись, но в конец файла.

Ответ:

- Открыть файл в режиме 'a'
- Добавить новые данные с `write()`
- Заккрыть файл

Шаг 14:

Миссия 14

Расположите шаги для безопасного создания отчёта только если файла ещё нет:

1. Открыть файл с 'x'
2. Записать информацию с write()
3. Закрыть файл

Подсказка: Режим 'x' предотвращает перезапись существующего файла.

Ответ:

- Открыть файл с 'x'
- Записать информацию с write()
- Закрыть файл

Урок 2 - Железные инструменты — обработка ошибок (tryexcept)

Шаг 01:

Миссия 1

Сопоставьте ситуации с правильным использованием try/except:

Ответ:

- Деление двух чисел
 - Открытие файла для чтения
 - Доступ к элементу словаря по ключу
 - Доступ к элементу списка по индексу
1. ZeroDivisionError
 2. FileNotFoundError
 3. KeyError
 4. IndexError

Подсказка: Каждое действие может вызвать конкретную ошибку, которую стоит поймать.

- **Деление двух чисел** — ZeroDivisionError
- **Открытие файла для чтения** — FileNotFoundError
- **Доступ к элементу словаря по ключу** — KeyError
- **Доступ к элементу списка по индексу** — IndexError

Шаг 02:

Миссия 2

Расположите шаги безопасного деления чисел с обработкой исключений:

1. Выполнить деление внутри try
2. Обработать ZeroDivisionError в except
3. Вывести результат, если ошибки не произошло
4. Выполнить блок finally (например, очистка ресурсов)

Подсказка: Сначала пытаемся выполнить действие, затем обрабатываем возможные ошибки, и всегда завершаем finally.

Ответ:

- Выполнить деление внутри try
- Обработать ZeroDivisionError в except
- Вывести результат, если ошибки не произошло

- Выполнить блок `finally` (например, очистка ресурсов)

Шаг 03:

Миссия 3

Расположите шаги открытия файла с обработкой ошибок:

1. Попытка открыть файл в `try`
2. Обработка `FileNotFoundError` в `except`
3. Использование файла, если нет ошибки
4. Закрытие файла в `finally`

Подсказка: Безопасная работа с файлами требует `try/except/finally`.

Ответ:

- Попытка открыть файл в `try`
- Обработка `FileNotFoundError` в `except`
- Использование файла, если нет ошибки
- Закрытие файла в `finally`

Шаг 04:

Миссия 4

Расположите шаги доступа к элементу словаря с обработкой `KeyError`:

1. Попытка получить значение по ключу в `try`
2. Обработка `KeyError` в `except`
3. Использование значения, если ключ существует
4. Очистка ресурсов или вывод сообщения в `finally`

Подсказка: Блок `finally` выполняется всегда, даже если была ошибка.

Ответ:

- Попытка получить значение по ключу в `try`
- Обработка `KeyError` в `except`
- Использование значения, если ключ существует
- Очистка ресурсов или вывод сообщения в `finally`

Шаг 05:

Миссия 5

Расположите шаги безопасного доступа к элементу списка по индексу:

1. Попытка доступа к элементу списка в `try`
2. Обработка `IndexError` в `except`
3. Использование элемента, если индекс корректный
4. Вывод сообщения или очистка ресурсов в `finally`

Подсказка: Всегда сначала `try`, потом `except`, далее `else` и `finally`.

Ответ:

- Попытка доступа к элементу списка в `try`
- Обработка `IndexError` в `except`
- Использование элемента, если индекс корректный
- Вывод сообщения или очистка ресурсов в `finally`

Шаг 06:

Миссия 6

Сопоставьте тип ошибки с её ситуацией:

Ответ:

- `FileNotFoundError`
 - `ZeroDivisionError`
 - `IndexError`
 - `KeyError`
1. Попытка открыть отсутствующий файл
 2. Деление на ноль
 3. Доступ к несуществующему индексу списка
 4. Доступ к несуществующему ключу словаря

Подсказка: Подумайте, какая ошибка связана с файлом, какая с числами, а какая с коллекциями.

- **`FileNotFoundError`** — Попытка открыть отсутствующий файл
- **`ZeroDivisionError`** — Деление на ноль
- **`IndexError`** — Доступ к несуществующему индексу списка
- **`KeyError`** — Доступ к несуществующему ключу словаря

Шаг 07:

Миссия 7

Сопоставьте блоки try/except с их предназначением:

Ответ:

- `try`
 - `except`
 - `finally`
 - `else`
1. Выполняется, если не возникло исключений
 2. Выполняет основной код, который может вызвать ошибку
 3. Обрабатывает исключение
 4. Выполняется всегда после `try`

Подсказка: Вспомните порядок блоков: `try` → `except` → `else` → `finally`.

- **`try`** — Выполняет основной код, который может вызвать ошибку
- **`except`** — Обрабатывает исключение
- **`finally`** — Выполняется всегда после `try`
- **`else`** — Выполняется, если не возникло исключений

Шаг 08:

Миссия 8

Сопоставьте ошибки с примерами кода:

Ответ:

- `FileNotFoundError`
- `ZeroDivisionError`

- `KeyError`
- `IndexError`

1. `open('map.txt', 'r')` когда файла нет
2. `value = 5 / 0`
3. `dictionary['diamond']` если ключа нет
4. `mylist[10]` если длина списка меньше 10

Подсказка: Каждая ошибка появляется в своём контексте: файл, деление, словарь или список.

- **`FileNotFoundError`** — `open('map.txt', 'r')` когда файла нет
- **`ZeroDivisionError`** — `value = 5 / 0`
- **`KeyError`** — `dictionary['diamond']` если ключа нет
- **`IndexError`** — `mylist[10]` если длина списка меньше 10

Шаг 09:

Миссия 9

Расположите шаги обработки ошибки при делении прочности кирки:

1. Попытка разделить прочность на количество ударов
2. Если ошибка `ZeroDivisionError`, вывести предупреждение
3. Использовать `finally` для вывода итогов

Подсказка: Сначала пробуем, потом обрабатываем, потом завершаем.

Ответ:

- Попытка разделить прочность на количество ударов
- Если ошибка `ZeroDivisionError`, вывести предупреждение
- Использовать `finally` для вывода итогов

Шаг 10:

Миссия 10

Расположите шаги безопасного чтения файла с картой шахты:

1. `try` открыть файл
2. `except` обработать `FileNotFoundError`
3. прочитать содержимое
4. `finally` закрыть файл

Подсказка: Сначала пробуем открыть, потом читаем, обрабатываем ошибки, в конце закрываем файл.

Ответ:

- `try` открыть файл
- прочитать содержимое
- `except` обработать `FileNotFoundError`
- `finally` закрыть файл

Шаг 11:

Миссия 11

Расположите шаги для безопасного получения ресурса из словаря шахты:

1. `try` получить значение по ключу

2. except обработать KeyError
3. else вывести найденный ресурс
4. finally вывести сообщение о завершении

Подсказка: Блок else выполняется только если исключение не возникло.

Ответ:

- try получить значение по ключу
- except обработать KeyError
- else вывести найденный ресурс
- finally вывести сообщение о завершении

Шаг 12:

Миссия 12

Расположите шаги безопасного сложного расчёта прочности инструментов:

1. try выполнить все деления и вычисления
2. except отловить ZeroDivisionError и ValueError
3. else вывести итоговый результат
4. finally вывести сообщение о завершении расчёта

Подсказка: try → except → else → finally — классическая схема.

Ответ:

- try выполнить все деления и вычисления
- except отловить ZeroDivisionError и ValueError
- else вывести итоговый результат
- finally вывести сообщение о завершении расчёта

Шаг 13:

Миссия 13

Сопоставьте тип исключения с его описанием:

Ответ:

- ZeroDivisionError
- FileNotFoundError
- IndexError
- KeyError

1. Попытка деления на ноль
2. Файл не найден
3. Выход за пределы списка
4. Нет ключа в словаре

Подсказка: Подумайте, когда возникает каждая ошибка в коде.

- **ZeroDivisionError** — Попытка деления на ноль
- **FileNotFoundError** — Файл не найден
- **IndexError** — Выход за пределы списка
- **KeyError** — Нет ключа в словаре

Шаг 14:

Миссия 14

Сопоставьте конструкции с их назначением:

Ответ:

- try
- except
- finally
- else

1. Блок для кода, который может вызвать исключение
2. Обработка конкретного исключения
3. Выполняется всегда после try, даже если было исключение
4. Выполняется если исключение не произошло

Подсказка: Вспомните порядок и смысл блоков try/except/else/finally.

- **try** — Блок для кода, который может вызвать исключение
- **except** — Обработка конкретного исключения
- **finally** — Выполняется всегда после try, даже если было исключение
- **else** — Выполняется если исключение не произошло

Урок 3 - Хранение ресурсов в сундуках — JSON

Шаг 01:

Миссия 1

Сопоставьте методы JSON с их назначением:

Ответ:

- json.load()
- json.loads()
- json.dump()
- json.dumps()

1. Чтение JSON из файла
2. Чтение JSON из строки
3. Запись JSON в файл
4. Преобразование объекта Python в JSON строку

Подсказка: Методы с 's' работают со строками, остальные — с файлами.

- **json.load()** — Чтение JSON из файла
- **json.loads()** — Чтение JSON из строки
- **json.dump()** — Запись JSON в файл
- **json.dumps()** — Преобразование объекта Python в JSON строку

Шаг 02:

Миссия 2

Сопоставьте типы данных Python с их JSON-эквивалентами:

Ответ:

- dict

- list
- str
- int

1. object
2. array
3. string
4. number

Подсказка: JSON использует универсальные типы, похожие на Python.

- **dict** — object
- **list** — array
- **str** — string
- **int** — number

Шаг 03:

Миссия 3

Сопоставьте операции с правильным методом JSON:

Ответ:

- Записать список ресурсов в файл
- Прочитать словарь ресурсов из строки
- Сохранить объект Python как строку JSON
- Загрузить JSON из файла в переменную

1. json.dump()
2. json.loads()
3. json.dumps()
4. json.load()

Подсказка: Файлы используют dump/load, строки — dumps/loads.

- **Записать список ресурсов в файл** — json.dump()
- **Прочитать словарь ресурсов из строки** — json.loads()
- **Сохранить объект Python как строку JSON** — json.dumps()
- **Загрузить JSON из файла в переменную** — json.load()

Шаг 04:

Миссия 4

Расположите шаги безопасного сохранения ресурсов в JSON-файл:

1. Открыть файл для записи
2. Вызвать json.dump() для объекта
3. Обработать исключения, если они возникнут
4. Закрыть файл

Подсказка: Сначала открываем файл, затем записываем и обрабатываем ошибки.

Ответ:

- Открыть файл для записи
- Вызвать json.dump() для объекта
- Обработать исключения, если они возникнут

- Заккрыть файл

Шаг 05:

Миссия 5

Расположите шаги загрузки JSON с ресурсами из файла с обработкой ошибок:

1. Открыть файл в try
2. Вызвать `json.load()` для чтения данных
3. Обработать `JSONDecodeError` в except
4. Заккрыть файл в finally

Подсказка: Безопасная работа с JSON требует try/except/finally.

Ответ:

- Открыть файл в try
- Вызвать `json.load()` для чтения данных
- Обработать `JSONDecodeError` в except
- Заккрыть файл в finally

Шаг 06:

Миссия 6

Расположите шаги проверки структуры JSON-файла с ресурсами:

1. Загрузить данные из файла
2. Проверить наличие ключей 'id', 'name', 'quantity'
3. **Если ключи отсутствуют** — обработать исключение
4. Использовать данные для обновления сундука

Подсказка: Сначала загрузка, потом проверка, потом использование.

Ответ:

- Загрузить данные из файла
- Проверить наличие ключей 'id', 'name', 'quantity'
- **Если ключи отсутствуют** — обработать исключение
- Использовать данные для обновления сундука

Шаг 07:

Миссия 7

Расположите шаги сериализации ресурсов для передачи по сети:

1. Подготовить объект Python с ресурсами
2. Вызвать `json.dumps()` для сериализации в строку
3. Обработать исключения `TypeError`
4. Отправить строку через gRPC

Подсказка: Сначала формируем объект, затем сериализуем, проверяем ошибки и отправляем.

Ответ:

- Подготовить объект Python с ресурсами
- Вызвать `json.dumps()` для сериализации в строку
- Обработать исключения `TypeError`
- Отправить строку через gRPC

Шаг 08:

Миссия 8

Сопоставьте типы данных Python с JSON типами:

Ответ:

- dict
 - list
 - str
 - int
1. object
 2. array
 3. string
 4. number

Подсказка: Вспомните, как Python-тип конвертируется в JSON.

- **dict** — object
- **list** — array
- **str** — string
- **int** — number

Шаг 09:

Миссия 9

Сопоставьте функции json с их назначением:

Ответ:

- json.load
 - json.loads
 - json.dump
 - json.dumps
1. Считать JSON из файла
 2. Считать JSON из строки
 3. Записать JSON в файл
 4. Преобразовать объект в JSON строку

Подсказка: load/loads — чтение, dump/dumps — запись.

- **json.load** — Считать JSON из файла
- **json.loads** — Считать JSON из строки
- **json.dump** — Записать JSON в файл
- **json.dumps** — Преобразовать объект в JSON строку

Шаг 10:

Миссия 10

Сопоставьте ситуации с нужным методом JSON:

Ответ:

- Прочитать сундук из файла
- Получить данные из строки JSON
- Сохранить сундук в файл

- Создать строку JSON для передачи по сети

1. json.load
2. json.loads
3. json.dump
4. json.dumps

Подсказка: Файл и строка требуют разных методов.

- **Прочитать сундук из файла** — json.load
- **Получить данные из строки JSON** — json.loads
- **Сохранить сундук в файл** — json.dump
- **Создать строку JSON для передачи по сети** — json.dumps

Шаг 11:

Миссия 11

Расположите шаги записи ресурсов в JSON файл:

1. Открыть файл для записи
2. Вызвать json.dump для объекта с ресурсами
3. Закрыть файл
4. Проверить корректность записи

Подсказка: Сначала подготовка файла, потом запись, закрытие и проверка.

Ответ:

- Открыть файл для записи
- Вызвать json.dump для объекта с ресурсами
- Закрыть файл
- Проверить корректность записи

Шаг 12:

Миссия 12

Расположите шаги чтения ресурсов из JSON файла:

1. Открыть файл для чтения
2. Вызвать json.load для чтения данных
3. Закрыть файл
4. Использовать данные в программе

Подсказка: Файл открывается, данные читаются, закрывается и применяются.

Ответ:

- Открыть файл для чтения
- Вызвать json.load для чтения данных
- Закрыть файл
- Использовать данные в программе

Шаг 13:

Миссия 13

Расположите шаги создания строки JSON для сетевого запроса:

1. Подготовить Python-объект с ресурсами

2. Вызвать `json.dumps` для преобразования в строку
3. Отправить строку по сети
4. Обработать ответ сервера

Подсказка: Сначала объект, затем сериализация и отправка.

Ответ:

- Подготовить Python-объект с ресурсами
- Вызвать `json.dumps` для преобразования в строку
- Отправить строку по сети
- Обработать ответ сервера

Шаг 14:

Миссия 14

Расположите шаги обработки данных из JSON строки:

1. Получить JSON строку от сервера
2. Вызвать `json.loads` для преобразования в Python-объект
3. Обработать данные
4. Сохранить необходимые ресурсы в файл

Подсказка: Сначала строка, затем десериализация и работа с данными.

Ответ:

- Получить JSON строку от сервера
- Вызвать `json.loads` для преобразования в Python-объект
- Обработать данные
- Сохранить необходимые ресурсы в файл

Урок 4 - Лавовая ловушка — работа с исключениями

Шаг 01:

Миссия 1

Сопоставьте тип исключения с примером ошибки:

Ответ:

- `ZeroDivisionError`
 - `IndexError`
 - `KeyError`
 - `ValueError`
1. Деление на ноль
 2. Доступ к несуществующему индексу списка
 3. Обращение к отсутствующему ключу в словаре
 4. Преобразование строки 'abc' в число

Подсказка: Вспомните, какой тип ошибки возникает в каждой ситуации.

- **`ZeroDivisionError`** — Деление на ноль
- **`IndexError`** — Доступ к несуществующему индексу списка
- **`KeyError`** — Обращение к отсутствующему ключу в словаре
- **`ValueError`** — Преобразование строки 'abc' в число

Шаг 02:

Миссия 2

Сопоставьте блоки try/except/else/finally с их поведением:

Ответ:

- try
- except
- else
- finally

1. Блок, который выполняется, если исключения нет
2. Блок, где размещается основной код
3. Блок, который перехватывает ошибки
4. Блок, который выполняется всегда

Подсказка: Порядок выполнения зависит от того, произошло исключение или нет.

- **try** — Блок, где размещается основной код
- **except** — Блок, который перехватывает ошибки
- **else** — Блок, который выполняется, если исключения нет
- **finally** — Блок, который выполняется всегда

Шаг 03:

Миссия 3

Сопоставьте ситуации в лавовой ловушке с нужным типом исключения:

Ответ:

- Игрок делит ресурсы на ноль
- Попытка взять сундук по несуществующему индексу
- Обращение к несуществующему ключу сундука
- Попытка преобразовать 'lava' в число

1. ZeroDivisionError
2. IndexError
3. KeyError
4. ValueError

Подсказка: Каждый тип ошибки соответствует конкретной ситуации.

- **Игрок делит ресурсы на ноль** — ZeroDivisionError
- **Попытка взять сундук по несуществующему индексу** — IndexError
- **Обращение к несуществующему ключу сундука** — KeyError
- **Попытка преобразовать 'lava' в число** — ValueError

Шаг 04:

Миссия 4

Расположите шаги обработки ошибки деления ресурсов на ноль:

1. Попытка разделить ресурсы
2. Перехват ZeroDivisionError
3. Вывести сообщение об ошибке
4. Продолжить выполнение программы

Подсказка: Сначала действие, потом обработка ошибки и продолжение работы.

Ответ:

- Попытка разделить ресурсы
- Перехват `ZeroDivisionError`
- Вывести сообщение об ошибке
- Продолжить выполнение программы

Шаг 05:

Миссия 5

Расположите шаги использования `try/except/else/finally`:

1. Выполнить основной код
2. Если ошибка, обработать её
3. Если ошибки нет, выполнить дополнительный код
4. Выполнить финальные действия всегда

Подсказка: Логика блоков зависит от наличия исключений.

Ответ:

- Выполнить основной код
- Если ошибка, обработать её
- Если ошибки нет, выполнить дополнительный код
- Выполнить финальные действия всегда

Шаг 06:

Миссия 6

Расположите шаги безопасного доступа к сундукам в лавовой ловушке:

1. Попытка получить ресурс по ключу
2. Перехват `KeyError`
3. Использование ресурса, если ключ есть
4. Вывод сообщения о недоступном ресурсе

Подсказка: Сначала доступ к данным, потом обработка ошибок и использование ресурсов.

Ответ:

- Попытка получить ресурс по ключу
- Перехват `KeyError`
- Использование ресурса, если ключ есть
- Вывод сообщения о недоступном ресурсе

Шаг 07:

Миссия 7

Расположите шаги обработки нескольких исключений одновременно:

1. Попытка выполнить несколько действий с ресурсами
2. Перехват `ZeroDivisionError`
3. Перехват `KeyError`
4. Выполнить `finally` блок
5. Продолжить работу программы

Подсказка: Основной код, разные except, finally и продолжение работы.

Ответ:

- Попытка выполнить несколько действий с ресурсами
- Перехват ZeroDivisionError
- Перехват KeyError
- Выполнить finally блок
- Продолжить работу программы

Шаг 08:

Миссия 8

Сопоставьте блоки try/except/else/finally с их назначением:

Ответ:

- try
 - except
 - else
 - finally
1. Содержит код, который может вызвать исключение
 2. Выполняется при возникновении указанного исключения
 3. Выполняется, если исключение не возникло
 4. Выполняется всегда, независимо от ошибок

Подсказка: Каждое ключевое слово имеет своё уникальное поведение.

- **try** — Содержит код, который может вызвать исключение
- **except** — Выполняется при возникновении указанного исключения
- **else** — Выполняется, если исключение не возникло
- **finally** — Выполняется всегда, независимо от ошибок

Шаг 09:

Миссия 9

Сопоставьте исключения с ситуациями Minecraft:

Ответ:

- ZeroDivisionError
 - FileNotFoundError
 - KeyError
 - TypeError
1. Вы делите ресурсы на ноль
 2. Файл сундука отсутствует
 3. Вы обращаетесь к несуществующему ключу в сундуке
 4. Вы пытаетесь сложить число и строку

Подсказка: Подумайте, какая ошибка возникает при каждой операции.

- **ZeroDivisionError** — Вы делите ресурсы на ноль
- **FileNotFoundError** — Файл сундука отсутствует
- **KeyError** — Вы обращаетесь к несуществующему ключу в сундуке
- **TypeError** — Вы пытаетесь сложить число и строку

Шаг 10:

Миссия 10

Сопоставьте методы обработки ошибок с их описанием:

Ответ:

- raise
 - try/except
 - assert
 - finally
1. Вызывает исключение вручную
 2. Перехватывает и обрабатывает ошибки
 3. Проверяет условие и возбуждает AssertionError при ложном
 4. Гарантирует выполнение кода в любом случае

Подсказка: Каждая конструкция используется для своей цели.

- **raise** — Вызывает исключение вручную
- **try/except** — Перехватывает и обрабатывает ошибки
- **assert** — Проверяет условие и возбуждает AssertionError при ложном
- **finally** — Гарантирует выполнение кода в любом случае

Шаг 11:

Миссия 11

Расположите шаги безопасного открытия сундука с ловушкой:

1. Попытка открыть сундук
2. Перехват возможных исключений
3. Выполнение действий при отсутствии ошибок
4. Заккрытие сундука независимо от ошибок

Подсказка: Сначала пробуем, потом ловим ошибки, далее блок else/finally.

Ответ:

- Попытка открыть сундук
- Перехват возможных исключений
- Выполнение действий при отсутствии ошибок
- Заккрытие сундука независимо от ошибок

Шаг 12:

Миссия 12

Расположите шаги обработки ошибок при делении ресурсов:

1. Попытка деления ресурсов
2. Проверка на ZeroDivisionError
3. Вывод предупреждения о лаве
4. Продолжение работы программы

Подсказка: Сначала выполняем код, затем ловим ошибку и продолжаем.

Ответ:

- Попытка деления ресурсов

- Проверка на ZeroDivisionError
- Вывод предупреждения о лаве
- Продолжение работы программы

Шаг 13:

Миссия 13

Расположите шаги обработки файлов сундука с ловушкой:

1. Открыть файл сундука
2. Попытка прочитать данные
3. Перехват FileNotFoundError
4. Закрытие файла

Подсказка: Открытие, попытка чтения, обработка ошибки, закрытие.

Ответ:

- Открыть файл сундука
- Попытка прочитать данные
- Перехват FileNotFoundError
- Закрытие файла

Шаг 14:

Миссия 14

Расположите шаги работы с ловушкой и несколькими исключениями:

1. Попытка открыть сундук с ловушкой
2. Перехват ZeroDivisionError и KeyError
3. Выполнение действий при отсутствии ошибок
4. Закрытие сундука

Подсказка: Сначала try, потом except, else и finally.

Ответ:

- Попытка открыть сундук с ловушкой
- Перехват ZeroDivisionError и KeyError
- Выполнение действий при отсутствии ошибок
- Закрытие сундука

Урок 5 - Редстоун-механизмы — классы и объекты

Шаг 01:

Миссия 1

Сопоставьте элементы класса с их назначением:

Ответ:

- `__init__`
 - Метод
 - Атрибут
 - Экземпляр класса
1. Инициализирует объект с заданными параметрами
 2. Функция, принадлежащая классу

3. Переменная, принадлежащая объекту
4. Конкретный объект класса

Подсказка: Каждый термин имеет своё конкретное значение в ООП.

- **`__init__`** — Инициализирует объект с заданными параметрами
- **Метод** — Функция, принадлежащая классу
- **Атрибут** — Переменная, принадлежащая объекту
- **Экземпляр класса** — Конкретный объект класса

🔮 Шаг 02:

Миссия 2

Сопоставьте объекты редстоун-механизмов с их действиями:

Ответ:

- Лавовая ловушка
 - Дверь с редстоун-сигналом
 - Световой индикатор
 - Поршень
1. Выстрел лавой при активации
 2. Открывает или закрывает проход
 3. Светится при подаче сигнала
 4. Выдвигает или втягивает блок

Подсказка: Подумайте о типичном поведении каждого механизма в Minecraft.

- **Лавовая ловушка** — Выстрел лавой при активации
- **Дверь с редстоун-сигналом** — Открывает или закрывает проход
- **Световой индикатор** — Светится при подаче сигнала
- **Поршень** — Выдвигает или втягивает блок

🔮 Шаг 03:

Миссия 3

Сопоставьте виды методов класса с их назначением:

Ответ:

- Метод экземпляра
 - Статический метод
 - Классовый метод
1. Принимает `self`, действует над конкретным объектом
 2. Не требует `self`, не зависит от объекта
 3. Принимает `cls`, действует на сам класс

Подсказка: Подумайте о том, на что направлен каждый метод: на объект или на класс.

- **Метод экземпляра** — Принимает `self`, действует над конкретным объектом
- **Статический метод** — Не требует `self`, не зависит от объекта
- **Классовый метод** — Принимает `cls`, действует на сам класс

🔮 Шаг 04:

Миссия 4

Расположите шаги создания объекта редстоун-механизма:

1. Определение класса с `__init__`
2. Создание экземпляра класса
3. Добавление методов для действий
4. Использование объекта в игре

Подсказка: Сначала класс, потом методы, потом объект, потом применение.

Ответ:

- Определение класса с `__init__`
- Добавление методов для действий
- Создание экземпляра класса
- Использование объекта в игре

Шаг 05:

Миссия 5

Расположите шаги вызова метода объекта:

1. Создание объекта
2. Определение метода в классе
3. Вызов метода через объект
4. Обработка результата метода

Подсказка: Сначала определяем метод, затем объект, потом вызываем метод, и обрабатываем результат.

Ответ:

- Определение метода в классе
- Создание объекта
- Вызов метода через объект
- Обработка результата метода

Шаг 06:

Миссия 6

Расположите шаги расширения класса редстоун-механизма:

1. Создание базового класса `Trap`
2. Создание подкласса `LavaTrap(Trap)`
3. Переопределение метода `activate`
4. Использование нового объекта в игре

Подсказка: Сначала базовый класс, потом наследование, потом изменение метода, потом использование.

Ответ:

- Создание базового класса `Trap`
- Создание подкласса `LavaTrap(Trap)`
- Переопределение метода `activate`
- Использование нового объекта в игре

Шаг 07:

Миссия 7

Расположите шаги комплексного использования объектов редстоун-механизмов:

1. Определение всех классов объектов
2. Создание экземпляров для каждого механизма
3. Вызов методов для активации действий
4. Обработка и проверка результатов взаимодействий

Подсказка: Сначала классы, потом объекты, действия и проверки.

Ответ:

- Определение всех классов объектов
- Создание экземпляров для каждого механизма
- Вызов методов для активации действий
- Обработка и проверка результатов взаимодействий

Шаг 08:

Миссия 8

Сопоставьте понятия классов и объектов:

Ответ:

- Класс
 - Объект
 - Метод
 - Атрибут
1. Шаблон для создания объектов
 2. Экземпляр класса
 3. Функция внутри класса
 4. Переменная внутри объекта

Подсказка: Подумайте, что создаёт объекты и что хранит данные внутри них.

- **Класс** — Шаблон для создания объектов
- **Объект** — Экземпляр класса
- **Метод** — Функция внутри класса
- **Атрибут** — Переменная внутри объекта

Шаг 09:

Миссия 9

Сопоставьте типы атрибутов с их описанием:

Ответ:

- Экземплярный атрибут
 - Классовый атрибут
 - Публичный атрибут
 - Приватный атрибут
1. Относится к конкретному объекту
 2. Общий для всех объектов класса
 3. Доступен извне класса

4. Доступен только внутри класса

Подсказка: Помните, что есть атрибуты объекта и класса, а также ограничения доступа.

- **Экземплярный атрибут** — Относится к конкретному объекту
- **Классовый атрибут** — Общий для всех объектов класса
- **Публичный атрибут** — Доступен извне класса
- **Приватный атрибут** — Доступен только внутри класса

Шаг 10:

Миссия 10

Сопоставьте специальные методы с их назначением:

Ответ:

- `__init__`
 - `__str__`
 - `__repr__`
 - `__del__`
1. Инициализация нового объекта
 2. Строковое представление для пользователя
 3. Строковое представление для разработчика
 4. Удаление объекта

Подсказка: Специальные методы начинаются и заканчиваются двойным подчёркиванием.

- `__init__` — Инициализация нового объекта
- `__str__` — Строковое представление для пользователя
- `__repr__` — Строковое представление для разработчика
- `__del__` — Удаление объекта

Шаг 11:

Миссия 11

Расположите шаги создания объекта редстоун-механизма:

1. Объявление класса
2. Добавление методов и атрибутов
3. Создание экземпляра объекта
4. Вызов методов объекта

Подсказка: Сначала планируем класс, затем используем объект.

Ответ:

- Объявление класса
- Добавление методов и атрибутов
- Создание экземпляра объекта
- Вызов методов объекта

Шаг 12:

Миссия 12

Расположите шаги работы с атрибутами класса и экземпляра:

1. Определение атрибутов класса

2. Создание экземпляров
3. Изменение экземплярных атрибутов
4. Доступ к атрибутам через объект

Подсказка: Сначала создаем общие атрибуты, затем работаем с объектами.

Ответ:

- Определение атрибутов класса
- Создание экземпляров
- Изменение экземплярных атрибутов
- Доступ к атрибутам через объект

Шаг 13:

Миссия 13

Расположите шаги наследования для редстоун-механизма:

1. Создать базовый класс
2. Создать подкласс с наследованием
3. Добавить новые методы в подкласс
4. Создать объект подкласса

Подсказка: Сначала создаем основу, затем расширяем её.

Ответ:

- Создать базовый класс
- Создать подкласс с наследованием
- Добавить новые методы в подкласс
- Создать объект подкласса

Шаг 14:

Миссия 14

Расположите шаги для полиморфного вызова метода у разных механизмов:

1. Определить базовый метод в классе родителя
2. Переопределить метод в подклассе
3. Создать объекты разных подклассов
4. Вызвать метод через объект

Подсказка: Сначала создаём базу, затем расширяем и используем.

Ответ:

- Определить базовый метод в классе родителя
- Переопределить метод в подклассе
- Создать объекты разных подклассов
- Вызвать метод через объект

Урок 6 - Автоматическая дверь — наследование

Шаг 01:

Миссия 1

Расположите шаги работы с методами подкласса:

1. Создать метод в базовом классе
2. Переопределить метод в подклассе при необходимости
3. Создать объект подкласса
4. Вызвать метод объекта

Подсказка: Сначала создаём основу, затем работаем с объектом.

Ответ:

- Создать метод в базовом классе
- Переопределить метод в подклассе при необходимости
- Создать объект подкласса
- Вызвать метод объекта

Шаг 02:

Миссия 2

Расположите шаги полиморфного использования дверей:

1. Создать базовый класс Door с методом open()
2. Создать подклассы с разной реализацией open()
3. Создать объекты разных подклассов
4. Вызвать метод open() у каждого объекта

Подсказка: Разные двери открываются по-своему.

Ответ:

- Создать базовый класс Door с методом open()
- Создать подклассы с разной реализацией open()
- Создать объекты разных подклассов
- Вызвать метод open() у каждого объекта

Шаг 03:

Миссия 3

Расположите шаги вызова методов с использованием super():

1. Создать метод в базовом классе
2. Создать подкласс и переопределить метод
3. Вызвать super() внутри метода подкласса
4. Вызвать метод объекта подкласса

Подсказка: super() позволяет использовать логику родителя внутри подкласса.

Ответ:

- Создать метод в базовом классе
- Создать подкласс и переопределить метод
- Вызвать super() внутри метода подкласса
- Вызвать метод объекта подкласса

Шаг 04:

Миссия 4

Сопоставьте понятия наследования с их описанием:

Ответ:

- Базовый класс
 - Подкласс
 - Переопределение
 - `super()`
1. Класс, от которого наследуются свойства
 2. Класс, который наследует свойства базового
 3. Создание нового поведения для метода базового класса
 4. Вызов метода родителя из подкласса

Подсказка: Подумайте, кто даёт свойства, а кто их получает и изменяет.

- **Базовый класс** — Класс, от которого наследуются свойства
- **Подкласс** — Класс, который наследует свойства базового
- **Переопределение** — Создание нового поведения для метода базового класса
- **`super()`** — Вызов метода родителя из подкласса

 Шаг 05:

Миссия 5

Сопоставьте типы наследования с их описанием:

Ответ:

- Одиночное наследование
 - Множественное наследование
 - Иерархическое наследование
 - Многоуровневое наследование
1. Подкласс наследует только один базовый класс
 2. Подкласс наследует несколько базовых классов
 3. Несколько подклассов наследуют один базовый класс
 4. Наследование через несколько уровней

Подсказка: Подумайте о том, как соединяются классы и кто наследует от кого.

- **Одиночное наследование** — Подкласс наследует только один базовый класс
- **Множественное наследование** — Подкласс наследует несколько базовых классов
- **Иерархическое наследование** — Несколько подклассов наследуют один базовый класс
- **Многоуровневое наследование** — Наследование через несколько уровней

 Шаг 06:

Миссия 6

Сопоставьте термины с примерами в контексте автоматической двери:

Ответ:

- Базовый метод `open`
 - Метод `open` в подклассе
 - Атрибут состояния двери
 - Конструктор `__init__`
1. Определяет общее открытие двери
 2. Добавляет сенсорное поведение

3. Хранит состояние 'открыта' или 'закрыта'
4. Инициализация атрибутов объекта двери

Подсказка: Смотрите, что делает метод, а что хранит данные.

- **Базовый метод open** — Определяет общее открытие двери
- **Метод open в подклассе** — Добавляет сенсорное поведение
- **Атрибут состояния двери** — Хранит состояние 'открыта' или 'закрыта'
- **Конструктор __init__** — Инициализация атрибутов объекта двери

Шаг 07:

Миссия 7

Расположите шаги создания автоматической двери с наследованием:

1. Создать базовый класс Door
2. Добавить методы и атрибуты базового класса
3. Создать подкласс AutomaticDoor
4. Переопределить метод open в подклассе
5. Создать объект подкласса

Подсказка: Сначала создаём основу, потом расширяем и используем объект.

Ответ:

- Создать базовый класс Door
- Добавить методы и атрибуты базового класса
- Создать подкласс AutomaticDoor
- Переопределить метод open в подклассе
- Создать объект подкласса

Шаг 08:

Миссия 8

Расположите шаги для использования super() в подклассе:

1. Определить метод в подклассе с тем же именем
2. Вызвать super().метод() внутри нового метода
3. Создать объект подкласса
4. Вызвать метод через объект

Подсказка: Сначала пишем метод, потом вызываем метод родителя.

Ответ:

- Определить метод в подклассе с тем же именем
- Вызвать super().метод() внутри нового метода
- Создать объект подкласса
- Вызвать метод через объект

Шаг 09:

Миссия 9

Расположите шаги многоуровневого наследования для дверей:

1. Создать базовый класс Door
2. Создать подкласс AutomaticDoor

3. Создать подкласс SmartDoor, наследующий AutomaticDoor
4. Добавить новые методы SmartDoor
5. Создать объект SmartDoor

Подсказка: Каждый следующий класс расширяет предыдущий.

Ответ:

- Создать базовый класс Door
- Создать подкласс AutomaticDoor
- Создать подкласс SmartDoor, наследующий AutomaticDoor
- Добавить новые методы SmartDoor
- Создать объект SmartDoor

Шаг 10:

Миссия 10

Расположите шаги для полиморфного вызова метода open у разных дверей:

1. Определить метод open в базовом классе
2. Переопределить метод open в подклассах
3. Создать объекты разных дверей
4. Вызвать метод open через объекты

Подсказка: Сначала создаём основу, затем расширяем её и используем через объекты.

Ответ:

- Определить метод open в базовом классе
- Переопределить метод open в подклассах
- Создать объекты разных дверей
- Вызвать метод open через объекты

Шаг 11:

Миссия 11

Сопоставьте термины наследования с определением:

Ответ:

- Базовый класс
 - Подкласс
 - Метод подкласса
 - Атрибут подкласса
1. Класс, от которого наследуются другие
 2. Класс, который наследует свойства и методы базового
 3. Функция внутри подкласса
 4. Переменная внутри подкласса

Подсказка: Подумайте, кто что наследует и кто что добавляет.

- **Базовый класс** — Класс, от которого наследуются другие
- **Подкласс** — Класс, который наследует свойства и методы базового
- **Метод подкласса** — Функция внутри подкласса
- **Атрибут подкласса** — Переменная внутри подкласса

Шаг 12:

Миссия 12

Сопоставьте виды методов с их назначением:

Ответ:

- Метод базового класса
 - Переопределённый метод
 - Метод подкласса
 - Статический метод
1. Определён в базовом классе и доступен в подклассе
 2. Изменяет поведение метода базового класса в подклассе
 3. Определён только в подклассе
 4. Метод, не использующий экземпляр

Подсказка: Подумайте, какие методы общие, а какие специфические.

- **Метод базового класса** — Определён в базовом классе и доступен в подклассе
- **Переопределённый метод** — Изменяет поведение метода базового класса в подклассе
- **Метод подкласса** — Определён только в подклассе
- **Статический метод** — Метод, не использующий экземпляр

Шаг 13:

Миссия 13

Сопоставьте термины наследования и полиморфизма с примерами:

Ответ:

- Наследование
 - Полиморфизм
 - `super()`
 - Переопределение метода
1. Подкласс получает методы и атрибуты базового класса
 2. Разные объекты реагируют по-разному на один и тот же вызов метода
 3. Вызов метода базового класса из подкласса
 4. Изменение метода базового класса в подклассе

Подсказка: Подумайте, что позволяет расширять функциональность.

- **Наследование** — Подкласс получает методы и атрибуты базового класса
- **Полиморфизм** — Разные объекты реагируют по-разному на один и тот же вызов метода
- **`super()`** — Вызов метода базового класса из подкласса
- **Переопределение метода** — Изменение метода базового класса в подклассе

Шаг 14:

Миссия 14

Расположите шаги создания подкласса автоматической двери:

1. Объявить базовый класс
2. Создать подкласс, указав базовый класс
3. Добавить уникальные методы или атрибуты подкласса
4. Создать объект подкласса и использовать методы

Подсказка: Сначала планируем базу, затем расширяем её.

Ответ:

- Объявить базовый класс
- Создать подкласс, указав базовый класс
- Добавить уникальные методы или атрибуты подкласса
- Создать объект подкласса и использовать методы

Урок 7 - Создание мобов — магические методы (`__init__`, `__str__`)

Шаг 01:

Миссия 1

Сопоставьте магические методы с их назначением:

Ответ:

- `__init__`
 - `__str__`
 - `__repr__`
 - `__del__`
1. Инициализация нового объекта
 2. Возвращает строковое представление объекта для пользователя
 3. Возвращает строковое представление объекта для разработчика
 4. Вызывается при удалении объекта

Подсказка: Подумайте, какой метод срабатывает при создании, какой при выводе.

- `__init__` — Инициализация нового объекта
- `__str__` — Возвращает строковое представление объекта для пользователя
- `__repr__` — Возвращает строковое представление объекта для разработчика
- `__del__` — Вызывается при удалении объекта

Шаг 02:

Миссия 2

Сопоставьте параметры `__init__` с их ролью:

Ответ:

- `self`
 - `name`
 - `health`
 - `level`
1. Ссылка на создаваемый объект
 2. Имя моба
 3. Количество здоровья
 4. Уровень силы моба

Подсказка: Первый параметр всегда ссылается на объект.

- **`self`** — Ссылка на создаваемый объект
- **`name`** — Имя моба
- **`health`** — Количество здоровья
- **`level`** — Уровень силы моба

Шаг 03:

Миссия 3

Расположите шаги создания моба с использованием `__init__`:

1. Объявить класс моба
2. Определить метод `__init__` с нужными параметрами
3. Создать объект моба с аргументами
4. Использовать методы объекта

Подсказка: Сначала создаём класс, затем инициализируем объект.

Ответ:

- Объявить класс моба
- Определить метод `__init__` с нужными параметрами
- Создать объект моба с аргументами
- Использовать методы объекта

Шаг 04:

Миссия 4

Расположите шаги для вывода информации о мобе с `__str__`:

1. Определить метод `__str__` в классе
2. Создать объект моба
3. Вызвать `print(объект)`
4. Получить читаемое строковое представление

Подсказка: `__str__` преобразует объект в строку для пользователя.

Ответ:

- Определить метод `__str__` в классе
- Создать объект моба
- Вызвать `print(объект)`
- Получить читаемое строковое представление

Шаг 05:

Миссия 5

Расположите шаги создания нескольких мобов с разными параметрами:

1. Определить класс `Mob` с `__init__`
2. Создать объект `mob1 = Mob("Zombie", 20)`
3. Создать объект `mob2 = Mob("Skeleton", 15)`
4. Использовать методы и `__str__` для каждого объекта

Подсказка: Каждый объект получает свои уникальные параметры.

Ответ:

- Определить класс `Mob` с `__init__`
- Создать объект `mob1 = Mob("Zombie", 20)`
- Создать объект `mob2 = Mob("Skeleton", 15)`
- Использовать методы и `__str__` для каждого объекта

Шаг 06:

Миссия 6

Расположите шаги переопределения `__str__` для кастомного вывода:

1. Определить метод `__str__` в классе
2. Использовать `self` для доступа к атрибутам
3. Вернуть строку с нужной информацией
4. Вызвать `print(объект)` для проверки

Подсказка: Метод `__str__` должен возвращать строку, которая будет показана игроку.

Ответ:

- Определить метод `__str__` в классе
- Использовать `self` для доступа к атрибутам
- Вернуть строку с нужной информацией
- Вызвать `print(объект)` для проверки

Шаг 07:

Миссия 7

Сопоставьте магические методы с их функцией:

Ответ:

- `__init__`
 - `__str__`
 - `__repr__`
 - `__del__`
1. Инициализация объекта при создании
 2. Строковое представление объекта для пользователя
 3. Строковое представление объекта для разработчика
 4. Удаление объекта

Подсказка: Подумайте, когда вызываются эти методы: при создании, удалении или выводе.

- **`__init__`** — Инициализация объекта при создании
- **`__str__`** — Строковое представление объекта для пользователя
- **`__repr__`** — Строковое представление объекта для разработчика
- **`__del__`** — Удаление объекта

Шаг 08:

Миссия 8

Сопоставьте действия моба с соответствующими методами:

Ответ:

- Создать моба
 - Вывести имя моба
 - Удалить моба
 - Представление для разработчика
1. `__init__`
 2. `__str__`
 3. `__del__`

4. `__repr__`

Подсказка: Подумайте, какой метод вызывается автоматически при каждой операции.

- **Создать моба** — `__init__`
- **Вывести имя моба** — `__str__`
- **Удалить моба** — `__del__`
- **Представление для разработчика** — `__repr__`

Шаг 09:

Миссия 9

Сопоставьте части конструктора с их назначением:

Ответ:

- `self`
 - `name`
 - `health`
 - Атрибуты класса
1. Ссылка на текущий объект
 2. Имя моба
 3. Здоровье моба
 4. Переменные, хранящие состояние объекта

Подсказка: `self` — это всегда объект, остальные — данные.

- **`self`** — Ссылка на текущий объект
- **`name`** — Имя моба
- **`health`** — Здоровье моба
- **Атрибуты класса** — Переменные, хранящие состояние объекта

Шаг 10:

Миссия 10

Расположите шаги создания моба с магическими методами:

1. Определить класс моба
2. Создать конструктор `__init__`
3. Переопределить `__str__` для красивого вывода
4. Создать объект моба и использовать методы

Подсказка: Сначала определяем структуру, потом создаём экземпляр.

Ответ:

- Определить класс моба
- Создать конструктор `__init__`
- Переопределить `__str__` для красивого вывода
- Создать объект моба и использовать методы

Шаг 11:

Миссия 11

Расположите шаги использования магических методов:

1. Создать объект моба

2. Вывести объект (вызывается `__str__`)
3. Использовать объект в отладке (`__repr__`)
4. Удалить объект (`__del__`)

Подсказка: Подумайте, что вызывается автоматически при каждом действии.

Ответ:

- Создать объект моба
- Вывести объект (вызывается `__str__`)
- Использовать объект в отладке (`__repr__`)
- Удалить объект (`__del__`)

Шаг 12:

Миссия 12

Расположите шаги работы с несколькими мобами:

1. Создать классы разных мобов
2. Переопределить `__str__` для каждого класса
3. Создать объекты всех мобов
4. Вывести всех мобов на экран

Подсказка: Сначала создаём классы, потом экземпляры, потом выводим.

Ответ:

- Создать классы разных мобов
- Переопределить `__str__` для каждого класса
- Создать объекты всех мобов
- Вывести всех мобов на экран

Шаг 13:

Миссия 13

Расположите шаги использования `__init__` и `__str__` при создании группы мобов:

1. Определить класс Mob с `__init__` и `__str__`
2. Создать несколько объектов с разными параметрами
3. Вывести информацию о каждом мобе
4. Удалить объекты при завершении работы

Подсказка: Сначала определяем класс, потом создаём и используем объекты.

Ответ:

- Определить класс Mob с `__init__` и `__str__`
- Создать несколько объектов с разными параметрами
- Вывести информацию о каждом мобе
- Удалить объекты при завершении работы

Урок 8 - Армия скелетов — модули и пакеты

Шаг 01:

Миссия 1

Сопоставьте тип Python-компонента с его описанием:

Ответ:

- Модуль
 - Пакет
 - Подмодуль
 - `__init__.py`
1. Одиночный `.py` файл с кодом
 2. Папка с `__init__.py` и подмодулями
 3. Модуль внутри пакета
 4. Файл, который делает папку пакетом

Подсказка: Подумайте, какой компонент содержит другие модули, а какой просто файл.

- **Модуль** — Одиночный `.py` файл с кодом
- **Пакет** — Папка с `__init__.py` и подмодулями
- **Подмодуль** — Модуль внутри пакета
- **`__init__.py`** — Файл, который делает папку пакетом

Шаг 02:

Миссия 2

Сопоставьте способ импорта с результатом:

Ответ:

- `import skeletons`
 - `from skeletons import create_skeleton`
 - `from skeletons import *`
 - `import skeletons as sk`
1. Доступ к функциям через `skeletons.function()`
 2. Доступ к функции напрямую `create_skeleton()`
 3. Импорт всех функций и классов модуля
 4. Импорт модуля с псевдонимом `sk`

Подсказка: Подумайте, как будет вызываться функция после каждого типа импорта.

- **`import skeletons`** — Доступ к функциям через `skeletons.function()`
- **`from skeletons import create_skeleton`** — Доступ к функции напрямую `create_skeleton()`
- **`from skeletons import *`** — Импорт всех функций и классов модуля
- **`import skeletons as sk`** — Импорт модуля с псевдонимом `sk`

Шаг 03:

Миссия 3

Сопоставьте команды импорта с их особенностью:

Ответ:

- `import math`
 - `from math import sqrt`
 - `import os as operating_system`
 - `from os import path`
1. Импортирует весь модуль
 2. Импортирует только функцию `sqrt`

3. Импортирует модуль с псевдонимом
4. Импортирует только подмодуль path

Подсказка: Псевдонимы меняют имя модуля при вызове.

- **import math** — Импортирует весь модуль
- **from math import sqrt** — Импортирует только функцию sqrt
- **import os as operating_system** — Импортирует модуль с псевдонимом
- **from os import path** — Импортирует только подмодуль path

🔮 Шаг 04:

Миссия 4

Расположите шаги создания пакета для армии скелетов:

1. Создать папку с названием пакета
2. Создать файл `__init__.py` внутри папки
3. Создать модули внутри папки
4. Импортировать пакет в основной код

Подсказка: Сначала создаём структуру, потом используем пакет в коде.

Ответ:

- Создать папку с названием пакета
- Создать файл `__init__.py` внутри папки
- Создать модули внутри папки
- Импортировать пакет в основной код

🔮 Шаг 05:

Миссия 5

Расположите шаги импорта функции из подмодуля:

1. Создать подмодуль `file_ops.py`
2. Определить функцию `create_skeleton()`
3. В основном коде написать `from skeleton_army.file_ops import create_skeleton`
4. Вызвать функцию `create_skeleton()`

Подсказка: Функция должна существовать до её импорта.

Ответ:

- Создать подмодуль `file_ops.py`
- Определить функцию `create_skeleton()`
- В основном коде написать `from skeleton_army.file_ops import create_skeleton`
- Вызвать функцию `create_skeleton()`

🔮 Шаг 06:

Миссия 6

Расположите шаги для переиспользования функций из модуля:

1. Создать модуль с функциями управления скелетами
2. Импортировать модуль в другой файл
3. Вызвать нужную функцию через имя модуля
4. Проверить результат работы функции

Подсказка: Сначала создаём, потом импортируем, затем используем.

Ответ:

- Создать модуль с функциями управления скелетами
- Импортировать модуль в другой файл
- Вызвать нужную функцию через имя модуля
- Проверить результат работы функции

Шаг 07:

Миссия 7

Расположите шаги для создания пакета с подмодулями для разных типов скелетов:

1. Создать папку `skeleton_army`
2. Создать файл `__init__.py` внутри папки
3. Создать подмодули `melee_skeleton.py` и `archer_skeleton.py`
4. Импортировать пакет и использовать функции из подмодулей

Подсказка: Пакет собирает все подмодули вместе для удобного импорта.

Ответ:

- Создать папку `skeleton_army`
- Создать файл `__init__.py` внутри папки
- Создать подмодули `melee_skeleton.py` и `archer_skeleton.py`
- Импортировать пакет и использовать функции из подмодулей

Шаг 08:

Миссия 8

Сопоставьте тип импорта с его примером:

Ответ:

- Импорт всего модуля
 - Импорт конкретной функции
 - Импорт с псевдонимом
 - Импорт всего содержимого
1. `import math`
 2. `from math import sqrt`
 3. `import math as m`
 4. `from math import *`

Подсказка: Обратите внимание, когда используется `*`, когда `as` и когда конкретная функция.

- **Импорт всего модуля** — `import math`
- **Импорт конкретной функции** — `from math import sqrt`
- **Импорт с псевдонимом** — `import math as m`
- **Импорт всего содержимого** — `from math import *`

Шаг 09:

Миссия 9

Сопоставьте элементы пакета с их назначением:

Ответ:

- `__init__.py`
 - `module.py`
 - `subpackage/`
 - `README.md`
1. Обозначает папку как пакет
 2. Содержит функции и классы модуля
 3. Подпакет с дополнительными модулями
 4. Документация пакета

Подсказка: Первый файл делает папку пакетом, подпапки содержат дополнительные модули.

- **`__init__.py`** — Обозначает папку как пакет
- **`module.py`** — Содержит функции и классы модуля
- **`subpackage/`** — Подпакет с дополнительными модулями
- **`README.md`** — Документация пакета

Шаг 10:

Миссия 10

Сопоставьте типы импортов с примером их применения в армии скелетов:

Ответ:

- Импорт модуля `random`
 - Импорт функции `randint`
 - Импорт с псевдонимом
 - Импорт всего содержимого модуля
1. `import random`
 2. `from random import randint`
 3. `import random as rnd`
 4. `from random import *`

Подсказка: Используйте `as` для сокращения, `*` для полного импорта.

- **Импорт модуля `random`** — `import random`
- **Импорт функции `randint`** — `from random import randint`
- **Импорт с псевдонимом** — `import random as rnd`
- **Импорт всего содержимого модуля** — `from random import *`

Шаг 11:

Миссия 11

Расположите шаги создания пакета армии скелетов:

1. Создать папку `army`
2. Добавить `__init__.py`
3. Создать модули `skeleton.py` и `archer.py`
4. Импортировать модуль в основной скрипт

Подсказка: Порядок отражает этапы разработки пакета.

Ответ:

- Создать папку `army`

- Добавить `__init__.py`
- Создать модули `skeleton.py` и `archer.py`
- Импортировать модуль в основной скрипт

Шаг 12:

Миссия 12

Расположите шаги подключения стандартного модуля и использования функций:

1. `import random`
2. Создать функцию, которая выбирает случайного скелета
3. Вызвать функцию для генерации случайного объекта
4. Использовать результат в армии

Подсказка: Сначала импортируем, затем используем функции модуля.

Ответ:

- `import random`
- Создать функцию, которая выбирает случайного скелета
- Вызвать функцию для генерации случайного объекта
- Использовать результат в армии

Шаг 13:

Миссия 13

Расположите шаги добавления подпакета `archer` в пакет `army`:

1. Создать папку `archer` внутри `army`
2. Добавить `__init__.py` в `archer`
3. Создать модули `archer_unit.py` и `crossbow.py`
4. Импортировать `archer` в основной скрипт армии

Подсказка: Подпакеты создаются по аналогии с основным пакетом.

Ответ:

- Создать папку `archer` внутри `army`
- Добавить `__init__.py` в `archer`
- Создать модули `archer_unit.py` и `crossbow.py`
- Импортировать `archer` в основной скрипт армии

Шаг 14:

Миссия 14

Расположите шаги использования пакета и модулей в бою армии скелетов:

1. Импортировать пакет `army` и подпакет `archer`
2. Создать объекты скелетов и лучников
3. Вызвать функции модулей для атаки
4. Обработать результаты боя

Подсказка: Сначала импортируем, затем создаём объекты и вызываем функции.

Ответ:

- Импортировать пакет `army` и подпакет `archer`
- Создать объекты скелетов и лучников

- Вызвать функции модулей для атаки
- Обработать результаты боя

Урок 9 - Алхимия зелий — лямбды, map и filter

Шаг 01:

Миссия 1

Сопоставьте функцию с её назначением в алхимии зелий:

Ответ:

- lambda
 - map
 - filter
 - list comprehension
1. Анонимная функция в одну строку
 2. Применяет функцию ко всем элементам списка
 3. Выбирает элементы списка по условию
 4. Создаёт новый список через выражение

Подсказка: lambda — функция без имени, map и filter часто используют вместе с ней.

- **lambda** — Анонимная функция в одну строку
- **map** — Применяет функцию ко всем элементам списка
- **filter** — Выбирает элементы списка по условию
- **list comprehension** — Создаёт новый список через выражение

Шаг 02:

Миссия 2

Сопоставьте пример с его действием:

Ответ:

- lambda x: x*2
 - map(lambda x: x+5, potions)
 - filter(lambda x: x>10, potions)
 - [x**2 for x in potions]
1. Удваивает значение одного элемента
 2. Применяет функцию ко всем элементам списка
 3. Фильтрует элементы больше 10
 4. Создаёт новый список с квадратами

Подсказка: Обратите внимание, где применяется к каждому элементу, а где — к одному.

- **lambda x: x*2** — Удваивает значение одного элемента
- **map(lambda x: x+5, potions)** — Применяет функцию ко всем элементам списка
- **filter(lambda x: x>10, potions)** — Фильтрует элементы больше 10
- **[x**2 for x in potions]** — Создаёт новый список с квадратами

Шаг 03:

Миссия 3

Сопоставьте комбинацию lambda с map/filter с результатом:

Ответ:

- `map(lambda x: x*3, potions)`
- `filter(lambda x: x%2==0, potions)`
- `map(lambda x: x+1, filter(lambda x: x>5, potions))`
- `list(filter(lambda x: x<10, potions))`

1. Умножение каждого зелья на 3
2. Выбирает чётные зелья
3. Выбирает зелья >5 и увеличивает их на 1
4. Выбирает зелья меньше 10

Подсказка: Читайте вложенность функций внимательно.

- **`map(lambda x: x*3, potions)`** — Умножение каждого зелья на 3
- **`filter(lambda x: x%2==0, potions)`** — Выбирает чётные зелья
- **`map(lambda x: x+1, filter(lambda x: x>5, potions))`** — Выбирает зелья >5 и увеличивает их на 1
- **`list(filter(lambda x: x<10, potions))`** — Выбирает зелья меньше 10

 Шаг 04:

Миссия 4

Расположите шаги для создания списка усиленных зелий:

1. Создать список исходных зелий
2. Написать lambda-функцию для усиления силы
3. Применить `map` к списку зелий
4. Преобразовать результат в список

Подсказка: Сначала создаём данные, затем функцию, потом применяем `map`, и в конце преобразуем в `list`.

Ответ:

- Создать список исходных зелий
- Написать lambda-функцию для усиления силы
- Применить `map` к списку зелий
- Преобразовать результат в список

 Шаг 05:

Миссия 5

Расположите шаги для фильтрации только мощных зелий и их увеличения:

1. Создай список зелий
2. Использовать `filter` для выбора зелий >50
3. Применить `map` с `lambda` для увеличения силы
4. Преобразовать результат в `list`

Подсказка: Фильтрация должна идти перед применением `map`.

Ответ:

- Создать список зелий
- Использовать `filter` для выбора зелий >50
- Применить `map` с `lambda` для увеличения силы

- Преобразовать результат в list

Шаг 06:

Миссия 6

Расположите шаги создания комбинированного списка зелий: сначала усилить все, затем выбрать только чётные значения.

1. Создать список зелий
2. Применить map с lambda для усиления силы
3. Использовать filter для выбора чётных зелий
4. Преобразовать результат в list

Подсказка: map идёт перед filter.

Ответ:

- Создать список зелий
- Применить map с lambda для усиления силы
- Использовать filter для выбора чётных зелий
- Преобразовать результат в list

Шаг 07:

Миссия 7

Расположите шаги для создания списка квадратов зелий только выше 10:

1. Создать список зелий
2. Использовать filter для выбора >10
3. Применить map с lambda для возведения в квадрат
4. Преобразовать результат в list

Подсказка: Сначала фильтрация, потом map, затем list.

Ответ:

- Создать список зелий
- Использовать filter для выбора >10
- Применить map с lambda для возведения в квадрат
- Преобразовать результат в list

Шаг 08:

Миссия 8

Сопоставьте инструмент Python с его описанием:

Ответ:

- map
 - filter
 - lambda
1. Применяет функцию ко всем элементам последовательности
 2. Отбирает элементы по условию
 3. Анонимная функция, используемая для кратких вычислений

Подсказка: map — для преобразования, filter — для отбора, lambda — для краткой функции.

- **map** — Применяет функцию ко всем элементам последовательности

- **filter** — Отбирает элементы по условию
- **lambda** — Анонимная функция, используемая для кратких вычислений

🧙 Шаг 09:

Миссия 9

Сопоставьте примеры с их результатом:

Ответ:

- `map(lambda x: x+1, [1,2,3])`
- `filter(lambda x: x>1, [1,2,3])`
- `lambda x: x*2`

1. [2,3,4]
2. [2,3]
3. Функция, удваивающая число

Подсказка: `map` изменяет все элементы, `filter` отбирает, `lambda` — это сама функция.

- **`map(lambda x: x+1, [1,2,3])` — [2,3,4]**
- **`filter(lambda x: x>1, [1,2,3])` — [2,3]**
- **`lambda x: x*2` — Функция, удваивающая число**

🧙 Шаг 10:

Миссия 10

Сопоставьте операцию с эффектом на зелье:

Ответ:

- `map(lambda x: x**2, [1,2,3])`
- `filter(lambda x: x%2==0, [1,2,3,4])`
- `lambda x: x-1`

1. Возводит каждое число в квадрат
2. Выбирает только чётные числа
3. Создаёт функцию, уменьшающую число на 1

Подсказка: `map` — изменение всех элементов, `filter` — фильтрация, `lambda` — сама функция.

- **`map(lambda x: x**2, [1,2,3])` — Возводит каждое число в квадрат**
- **`filter(lambda x: x%2==0, [1,2,3,4])` — Выбирает только чётные числа**
- **`lambda x: x-1` — Создаёт функцию, уменьшающую число на 1**

🧙 Шаг 11:

Миссия 11

Расположите шаги создания списка мощных зелий с использованием `map` и `lambda`:

1. Создать список базовых зелий: [1,2,3]
2. Создать лямбда-функцию для усиления силы
3. Применить `map` к списку зелий
4. Преобразовать результат в список

Подсказка: Сначала есть исходные данные, потом функция, затем `map` и результат.

Ответ:

- Создать список базовых зелий: [1,2,3]
- Создать лямбда-функцию для усиления силы
- Применить map к списку зелий
- Преобразовать результат в список

 Шаг 12:

Миссия 12

Расположите шаги фильтрации ингредиентов с помощью filter и lambda:

1. Создать список ингредиентов с мощностью [1,2,3,4]
2. Создать лямбда-функцию для проверки силы>2
3. Применить filter к списку ингредиентов
4. Преобразовать результат в список

Подсказка: filter оставляет только подходящие элементы.

Ответ:

- Создать список ингредиентов с мощностью [1,2,3,4]
- Создать лямбда-функцию для проверки силы>2
- Применить filter к списку ингредиентов
- Преобразовать результат в список

 Шаг 13:

Миссия 13

Расположите шаги объединения map и filter для зелий:

1. Создать список зелий [1,2,3,4]
2. Создать лямбда-функцию для усиления силы
3. Применить map для усиления всех зелий
4. Применить filter для отбора только сильных зелий
5. Преобразовать результат в список

Подсказка: Сначала map усиливает, затем filter отбирает лучшие.

Ответ:

- Создать список зелий [1,2,3,4]
- Создать лямбда-функцию для усиления силы
- Применить map для усиления всех зелий
- Применить filter для отбора только сильных зелий
- Преобразовать результат в список

 Шаг 14:

Миссия 14

Расположите шаги создания цепочки трансформации зелий:

1. Список базовых зелий [1,2,3,4]
2. Применить map(lambda x: x*2)
3. Применить filter(lambda x: x>4)
4. Преобразовать в список

Подсказка: Сначала данные, затем map для усиления, filter для отбора, результат — список.

Ответ:

- Список базовых зелий [1,2,3,4]
- Применить `map(lambda x: x*2)`
- Применить `filter(lambda x: x>4)`
- Преобразовать в список

Урок 10 - Торговля с жителем — итераторы и генераторы

Шаг 01:

Миссия 1

Расположите шаги объединения итератора и генератора для обработки всех товаров:

1. Создать список товаров
2. Создать генератор для каждого товара
3. Перебрать генераторы через итератор
4. Обработать каждый товар

Подсказка: Сначала данные, затем генератор, затем итератор, потом обработка.

Ответ:

- Создать список товаров
- Создать генератор для каждого товара
- Перебрать генераторы через итератор
- Обработать каждый товар

Шаг 02:

Миссия 2

Сопоставьте понятие с его действием при торговле с жителем:

Ответ:

- `iter()`
 - `next()`
 - генератор
 - `yield`
1. Создаёт объект-итератор
 2. Возвращает следующий элемент из итератора
 3. Функция, возвращающая элементы по одному
 4. Возвращает элемент генератора и сохраняет состояние

Подсказка: Итераторы и генераторы позволяют не хранить все элементы сразу.

- **`iter()`** — Создаёт объект-итератор
- **`next()`** — Возвращает следующий элемент из итератора
- **генератор** — Функция, возвращающая элементы по одному
- **`yield`** — Возвращает элемент генератора и сохраняет состояние

Шаг 03:

Миссия 3

Сопоставьте пример генератора с его результатом:

Ответ:

- `def gen(): yield 1`
 - `def gen(): for i in range(3): yield i`
 - `(i*2 for i in range(5))`
 - `list(gen())`
1. Выдаёт один элемент
 2. Выдаёт 0,1,2
 3. Генератор выражение, выдаёт 0,2,4,6,8
 4. Преобразует генератор в список

Подсказка: Генератор нужно итерировать или преобразовать в список, чтобы получить значения.

- **`def gen(): yield 1`** — Выдаёт один элемент
- **`def gen(): for i in range(3): yield i`** — Выдаёт 0,1,2
- **`(i*2 for i in range(5))`** — Генератор выражение, выдаёт 0,2,4,6,8
- **`list(gen())`** — Преобразует генератор в список

Шаг 04:

Миссия 4

Сопоставьте ситуацию с правильной техникой итерации:

Ответ:

- Перебор всех товаров в списке
 - Получение первого товара
 - Создание последовательности скидок
 - Пошаговая выдача каждого предмета
1. `for item in trades`
 2. `next(iter(trades))`
 3. генератор с `yield`
 4. итератор

Подсказка: Различайте, когда нужен цикл, а когда просто следующий элемент.

- **Перебор всех товаров в списке** — `for item in trades`
- **Получение первого товара** — `next(iter(trades))`
- **Создание последовательности скидок** — генератор с `yield`
- **Пошаговая выдача каждого предмета** — итератор

Шаг 05:

Миссия 5

Расположите шаги для перебора товаров с применением генератора:

1. Создать генератор для товаров
2. Получить итератор генератора
3. Использовать `next()` для получения первого товара
4. Продолжить перебор до конца

Подсказка: Генератор сначала создаётся, затем перебирается через итератор.

Ответ:

- Создать генератор для товаров
- Получить итератор генератора
- Использовать `next()` для получения первого товара
- Продолжить перебор до конца

 Шаг 06:

Миссия 6

Расположите шаги для создания генератора скидок и перебора их:

1. Определить функцию с `yield`
2. Создать генератор вызовом функции
3. Получить итератор
4. Перебирать элементы с `next()` или в цикле

Подсказка: `yield` создаёт генератор, затем его можно итерировать.

Ответ:

- Определить функцию с `yield`
- Создать генератор вызовом функции
- Получить итератор
- Перебирать элементы с `next()` или в цикле

 Шаг 07:

Миссия 7

Расположите шаги генерации и фильтрации товаров:

1. Создать генератор всех товаров
2. Использовать генераторное выражение для выбора дорогих товаров
3. Получить итератор
4. Перебирать и выводить товары

Подсказка: Сначала создаём генератор, потом фильтруем и итерируем.

Ответ:

- Создать генератор всех товаров
- Использовать генераторное выражение для выбора дорогих товаров
- Получить итератор
- Перебирать и выводить товары

 Шаг 08:

Миссия 8

Расположите шаги комбинированного перебора и применения генератора скидок:

1. Определить генератор с `yield` для скидки
2. Создать генератор товаров
3. Перебирать товары и применять генератор скидок
4. Вывести результат

Подсказка: Генераторы можно использовать вместе, шаг за шагом.

Ответ:

- Определить генератор с `yield` для скидки
- Создать генератор товаров
- Перебирать товары и применять генератор скидок
- Вывести результат

 Шаг 09:

Миссия 9

Сопоставьте понятие с определением:

Ответ:

- Итератор
 - Генератор
 - `next()`
 - `yield`
1. Объект, который возвращает элементы по одному
 2. Функция, которая позволяет создавать генератор
 3. Возвращает следующий элемент итератора
 4. Создаёт элементы на лету, экономя память

Подсказка: `yield` используется внутри функции для создания генератора.

- **Итератор** — Объект, который возвращает элементы по одному
- **Генератор** — Создаёт элементы на лету, экономя память
- **`next()`** — Возвращает следующий элемент итератора
- **`yield`** — Функция, которая позволяет создавать генератор

 Шаг 10:

Миссия 10

Сопоставьте код с его действием:

Ответ:

- `iter(goods)`
 - `next(goods_iter)`
 - `def shop(): yield item`
 - `for item in shop()`
1. Создаёт итератор из списка
 2. Получает следующий элемент из итератора
 3. Создаёт генератор для товаров
 4. Перебирает элементы генератора

Подсказка: Для генераторов используется `for`, для итераторов — `next()`.

- **`iter(goods)`** — Создаёт итератор из списка
- **`next(goods_iter)`** — Получает следующий элемент из итератора
- **`def shop(): yield item`** — Создаёт генератор для товаров
- **`for item in shop()`** — Перебирает элементы генератора

🔮 Шаг 11:

Миссия 11

Сопоставьте метод/ключевое слово с результатом:

Ответ:

- `next()`
- `iter()`
- `yield`
- `for ... in ...`

1. Возвращает следующий элемент
2. Создаёт итератор
3. Превращает функцию в генератор
4. Перебирает элементы

Подсказка: Помните, что `yield` всегда используется внутри функции.

- **`next()`** — Возвращает следующий элемент
- **`iter()`** — Создаёт итератор
- **`yield`** — Превращает функцию в генератор
- **`for ... in ...`** — Перебирает элементы

🔮 Шаг 12:

Миссия 12

Расположите шаги получения товаров у жителя с генератором:

1. Создать функцию `shop()` с `yield`
2. Вызвать генератор: `items = shop()`
3. Перебрать товары через `for item in items`
4. Обработать каждый товар (например, продать или оценить)

Подсказка: Сначала создаём генератор, затем получаем объект генератора и перебираем его.

Ответ:

- Создать функцию `shop()` с `yield`
- Вызвать генератор: `items = shop()`
- Перебрать товары через `for item in items`
- Обработать каждый товар (например, продать или оценить)

🔮 Шаг 13:

Миссия 13

Расположите шаги для создания итератора товаров и получения первого предмета:

1. Создать список товаров
2. Создать итератор: `it = iter(goods)`
3. Получить первый товар: `next(it)`
4. Использовать товар (например, показать игроку)

Подсказка: Итератор создаётся после списка, `next()` используется для получения элемента.

Ответ:

- Создать список товаров

- Создать итератор: `it = iter(goods)`
- Получить первый товар: `next(it)`
- Использовать товар (например, показать игроку)

🔮 Шаг 14:

Миссия 14

Расположите шаги для генерации товаров и фильтрации только редких:

1. Создать генератор с `yield`
2. Применить фильтр для редких товаров
3. Перебрать элементы генератора через `for`
4. Обработать редкие товары

Подсказка: Сначала генератор, потом фильтр и перебор.

Ответ:

- Создать генератор с `yield`
- Применить фильтр для редких товаров
- Перебрать элементы генератора через `for`
- Обработать редкие товары

📖 Урок 11 - Генерация деревни — списковые включения (list comprehension)

🔮 Шаг 01:

Миссия 1

Сопоставьте элементы list comprehension с их назначением:

Ответ:

- `[x for x in range(5)]`
- `[x**2 for x in range(5)]`
- `[x for x in range(10) if x%2==0]`
- `[name.upper() for name in villagers]`

1. Все числа от 0 до 4
2. Квадраты чисел от 0 до 4
3. Только чётные числа от 0 до 9
4. Имена жителей в верхнем регистре

Подсказка: Обратите внимание на операции и условия внутри списка.

- **`[x for x in range(5)]`** — Все числа от 0 до 4
- **`[x**2 for x in range(5)]`** — Квадраты чисел от 0 до 4
- **`[x for x in range(10) if x%2==0]`** — Только чётные числа от 0 до 9
- **`[name.upper() for name in villagers]`** — Имена жителей в верхнем регистре

🔮 Шаг 02:

Миссия 2

Сопоставьте код с результатом:

Ответ:

- `[x*3 for x in range(4)]`

- `[x for x in range(10) if x>5]`
- `[name.lower() for name in villagers]`
- `[x**2 for x in range(3,6)]`

1. `[0,3,6,9]`
2. `[6,7,8,9]`
3. Все имена жителей в нижнем регистре
4. `[9,16,25]`

Подсказка: Обратите внимание на диапазоны и условия.

- **`[x*3 for x in range(4)]`** — `[0,3,6,9]`
- **`[x for x in range(10) if x>5]`** — `[6,7,8,9]`
- **`[name.lower() for name in villagers]`** — Все имена жителей в нижнем регистре
- **`[x**2 for x in range(3,6)]`** — `[9,16,25]`

Шаг 03:

Миссия 3

Сопоставьте части list comprehension с функцией:

Ответ:

- Выражение
 - Цикл `for`
 - Условие `if`
 - Диапазон `range()`
1. Определяет, что добавляется в итоговый список
 2. Перебирает элементы
 3. Фильтрует элементы по условию
 4. Создает последовательность чисел

Подсказка: Подумайте, какая часть отвечает за что.

- **Выражение** — Определяет, что добавляется в итоговый список
- **Цикл `for`** — Перебирает элементы
- **Условие `if`** — Фильтрует элементы по условию
- **Диапазон `range()`** — Создает последовательность чисел

Шаг 04:

Миссия 4

Расположите шаги создания списка домов с площадью от 10 до 20 м²:

1. Создать диапазон чисел для площади
2. Применить условие `if` для фильтрации
3. Определить, что добавляется в список (например, площадь)**
4. Собрать элементы в список через list comprehension

Подсказка: Сначала диапазон, потом условие, потом выражение, затем list comprehension.

Ответ:

- Создать диапазон чисел для площади
- Применить условие `if` для фильтрации
- Определить, что добавляется в список (например, площадь)

- Собрать элементы в список через list comprehension

Шаг 05:

Миссия 5

Расположите шаги для генерации имен жителей с заглавной буквы:

1. Взять список исходных имён
2. Использовать метод .capitalize() для каждого имени
3. Создать list comprehension
4. Собрать итоговый список

Подсказка: Сначала исходные данные, потом метод, потом comprehension, потом итоговый список.

Ответ:

- Взять список исходных имён
- Использовать метод .capitalize() для каждого имени
- Создать list comprehension
- Собрать итоговый список

Шаг 06:

Миссия 6

Расположите шаги для генерации списка редких ресурсов деревни:

1. Создать список всех ресурсов
2. Применить фильтр для редких ресурсов через if
3. Определить выражение для добавления в итоговый список
4. Собрать список через list comprehension

Подсказка: Сначала исходные данные, потом фильтр, потом выражение, потом итоговый список.

Ответ:

- Создать список всех ресурсов
- Применить фильтр для редких ресурсов через if
- Определить выражение для добавления в итоговый список
- Собрать список через list comprehension

Шаг 07:

Миссия 7

Расположите шаги создания списка квадратов площадей домов больше 15 м²:

1. Создать список площадей домов
2. Применить фильтр if >15
3. Возвести каждый элемент в квадрат
4. Собрать итоговый список через list comprehension

Подсказка: Сначала исходные данные, затем фильтр, затем выражение, потом итоговый список.

Ответ:

- Создать список площадей домов
- Применить фильтр if >15
- Возвести каждый элемент в квадрат

- Собрать итоговый список через list comprehension

🔮 Шаг 08:

Миссия 8

Сопоставьте элементы списка с их правильным результатом в list comprehension:

Ответ:

- `[x*2 for x in [1,2,3]]`
- `[x for x in [1,2,3] if x>1]`
- `[x.upper() for x in ['a','b']]`
- `[x**2 for x in range(4)]`

1. `[2,4,6]`
2. `[2,3]`
3. `['A','B']`
4. `[0,1,4,9]`

Подсказка: Смотрите на выражения перед for и условие if.

- **`[x*2 for x in [1,2,3]]`** — `[2,4,6]`
- **`[x for x in [1,2,3] if x>1]`** — `[2,3]`
- **`[x.upper() for x in ['a','b']]`** — `['A','B']`
- **`[x**2 for x in range(4)]`** — `[0,1,4,9]`

🔮 Шаг 09:

Миссия 9

Сопоставьте код list comprehension с его действием:

Ответ:

- `[house.height for house in village]`
- `[villager.name for villager in villagers if villager.age>30]`
- `[shop.name.upper() for shop in shops]`
- `[x**3 for x in range(5)]`

1. Создаёт список высот домов
2. Список имён жителей старше 30
3. Список имён магазинов в верхнем регистре
4. Список кубов чисел от 0 до 4

Подсказка: Читайте внимательно, что возвращает выражение перед for.

- **`[house.height for house in village]`** — Создаёт список высот домов
- **`[villager.name for villager in villagers if villager.age>30]`** — Список имён жителей старше 30
- **`[shop.name.upper() for shop in shops]`** — Список имён магазинов в верхнем регистре
- **`[x**3 for x in range(5)]`** — Список кубов чисел от 0 до 4

🔮 Шаг 10:

Миссия 10

Сопоставьте конструкцию list comprehension с её результатом:

Ответ:

- `[x for x in range(10) if x%2==0]`
 - `[x*10 for x in range(5)]`
 - `[house.color for house in village]`
 - `[name.lower() for name in ['ALICE','BOB']]`
1. `[0,2,4,6,8]`
 2. `[0,10,20,30,40]`
 3. Список цветов домов
 4. `['alice','bob']`

Подсказка: Смотрите на условие `if` и выражение перед `for`.

- **`[x for x in range(10) if x%2==0]`** — `[0,2,4,6,8]`
- **`[x*10 for x in range(5)]`** — `[0,10,20,30,40]`
- **`[house.color for house in village]`** — Список цветов домов
- **`[name.lower() for name in ['ALICE','BOB']]`** — `['alice','bob']`

 Шаг 11:**Миссия 11**

Расположите шаги создания списка имён жителей, старше 25 лет, в верхнем регистре:

1. Взять список всех жителей
2. Применить фильтр `age>25`
3. Взять имена выбранных жителей
4. Преобразовать имена в верхний регистр

Подсказка: Сначала фильтруем, затем берём атрибуты и преобразуем.

Ответ:

- Взять список всех жителей
- Применить фильтр `age>25`
- Взять имена выбранных жителей
- Преобразовать имена в верхний регистр

 Шаг 12:**Миссия 12**

Расположите шаги для создания списка красных и синих крыш:

1. Перебрать все дома
2. Проверить цвет крыши: красный или синий
3. Добавить дом в список, если условие выполнено
4. Получить готовый список

Подсказка: Сначала перебор, затем фильтр, потом добавление и результат.

Ответ:

- Перебрать все дома
- Проверить цвет крыши: красный или синий
- Добавить дом в список, если условие выполнено
- Получить готовый список

Шаг 13:

Миссия 13

Расположите шаги создания списка всех жителей с их возрастом и профессией:

1. Перебрать всех жителей
2. Собрать tuple: (name, age, profession)
3. Добавить tuple в список
4. Получить готовый список всех жителей

Подсказка: Tuple создаётся внутри list comprehension.

Ответ:

- Перебрать всех жителей
- Собрать tuple: (name, age, profession)
- Добавить tuple в список
- Получить готовый список всех жителей

Шаг 14:

Миссия 14

Расположите шаги для создания списка всех редких предметов из деревни:

1. Перебрать все дома
2. Взять предметы из каждого дома
3. Применить фильтр на редкость
4. Получить итоговый список редких предметов

Подсказка: Сначала перебираем дома, затем предметы, потом фильтр и список.

Ответ:

- Перебрать все дома
- Взять предметы из каждого дома
- Применить фильтр на редкость
- Получить итоговый список редких предметов

Модуль 3 - Уровень «End» (Senior)

Урок 1 - Поход в Энд — асинхронность (async/await)

Шаг 01:

Миссия 1

Вы — программисты Minecraft, и вам нужно понять, что такое асинхронность. Какое определение `async/await` наиболее точное?

Подсказка: Подумайте о том, как игра может выполнять несколько действий одновременно, не блокируя главный процесс.

Ответ:

- **Механизм, позволяющий выполнять задачи параллельно без блокировки основного потока**
- Способ записывать классические циклы в коде
- Метод повышения производительности видеокарты в Minecraft

🔮 Шаг 02:

Миссия 2

Какой из этих вариантов лучше всего описывает, зачем в Python используют `async/await`?

Подсказка: Представьте, что ваш персонаж Minecraft ждет открытия сундука, но не должен просто стоять на месте.

Ответ:

- Чтобы ускорить работу процессора
- **Чтобы выполнять другие задачи во время ожидания**
- Чтобы создать новый игровой режим
- Чтобы уменьшить размер программы

🔮 Шаг 03:

Миссия 3

Ваш Minecraft-бот отправляется в поход по Энд. Какой метод из `async/await` позволит не блокировать игру, ожидая ответа сервера?

Подсказка: Обратите внимание на ключевое слово, которое приостанавливает выполнение, позволяя другим процессам идти дальше.

Ответ:

- **`await`**
- `async`
- `sync`

🔮 Шаг 04:

Миссия 4

Сопоставьте ключевые понятия асинхронности с их описаниями.

Ответ:

- `async`
- `await`
- корутина
- цикл событий

1. Позволяет приостанавливать выполнение функции до завершения асинхронной операции
2. Ключевое слово для объявления асинхронной функции
3. Объект, который можно приостанавливать и возобновлять
4. Механизм управления асинхронными операциями в Python

Подсказка: Подумайте, какое слово используется для создания, а какое — для ожидания, а что управляет процессом.

- **`async`** — Ключевое слово для объявления асинхронной функции
- **`await`** — Позволяет приостанавливать выполнение функции до завершения асинхронной операции
- **корутина** — Объект, который можно приостанавливать и возобновлять
- **цикл событий** — Механизм управления асинхронными операциями в Python

🔮 Шаг 05:

Миссия 5

Сопоставьте ситуации с правильными способами использования `async/await` в Minecraft.

Ответ:

- Обработка событий от игроков
- Загрузка данных с сервера
- Обновление интерфейса
- Проверка соединения с базой данных

1. `await`
2. `async def`
3. `asyncio.run()`
4. `callback`

Подсказка: Обратите внимание, когда нужно приостановить функцию, а когда — запустить весь цикл.

- **Обработка событий от игроков** — `callback`
- **Загрузка данных с сервера** — `await`
- **Обновление интерфейса** — `asyncio.run()`
- **Проверка соединения с базой данных** — `async def`

Шаг 06:

Миссия 6

Сопоставьте ошибки с их возможными причинами при использовании `async/await` в Minecraft-проектах.

Ответ:

- Корутину вызвали без `await`
- Использование `blocking I/O` внутри `async` функции
- Неправильный цикл событий
- Отсутствие `async` перед функцией

1. Функция не ожидает завершения корутины
2. Блокировка основного потока, несмотря на `async`
3. Некорректное управление задачами
4. Функция не является асинхронной

Подсказка: Вспомните, как важно правильно писать `async` функции и использовать `await`.

- **Корутину вызвали без `await`** — Функция не ожидает завершения корутины
- **Использование `blocking I/O` внутри `async` функции** — Блокировка основного потока, несмотря на `async`
- **Неправильный цикл событий** — Некорректное управление задачами
- **Отсутствие `async` перед функцией** — Функция не является асинхронной

Шаг 07:

Миссия 7

Программист из команды Minecraft решил изучить `async/await`. Что означает ключевое слово `async` в Python?

Подсказка: Обратите внимание, что `async` делает функцию особой — не просто запускает её, а даёт возможность работать с ожиданиями.

Ответ:

- **Определяет функцию как асинхронную, позволяющую использовать await**
- Запускает функцию в отдельном потоке
- Означает, что функция возвращает результат сразу
- Указывает, что функция не имеет параметров

🔮 **Шаг 08:**

Миссия 8

В Minecraft команда разработчиков использует await. Что делает ключевое слово await в асинхронной функции?

Подсказка: Подумайте, как ожидание результата может не блокировать остальные действия.

Ответ:

- **Ожидает завершения асинхронной операции, не блокируя поток**
- Запускает асинхронную функцию параллельно
- Блокирует программу до завершения операции
- Превращает функцию в синхронную

🔮 **Шаг 09:**

Миссия 9

Команда Minecraft использует asyncio. Что такое event loop в контексте async/await?

Подсказка: Подумайте о центральном диспетчере, который следит за выполнением задач.

Ответ:

- **Механизм, который управляет и переключает выполнение асинхронных задач**
- Новая синхронная функция в Python
- Отдельный поток для работы с сетью
- Модуль для работы с файлами

🔮 **Шаг 10:**

Миссия 10

При асинхронном программировании в Minecraft возникает проблема "блокировки". Что это означает?

Подсказка: Вспомните, что происходит, когда выполнение одного кода задерживает другой.

Ответ:

- **Когда одна операция мешает выполнению других из-за ожидания**
- Когда программа выполняется слишком быстро
- Когда операции работают параллельно без синхронизации
- Когда используется много потоков

🔮 **Шаг 11:**

Миссия 11

Сопоставьте понятия из асинхронного программирования и их объяснения, которые команда Minecraft обсуждала после похода в Энд.

Понятия:

Ответ:

- `async`
- `await`
- `event loop`
- `coroutine`

Определения:

1. Специальная функция, которая может приостанавливать выполнение
2. Ключевое слово для определения асинхронной функции
3. Объект, ожидающий завершения другой операции
4. Диспетчер, управляющий задачами

🔗 Подсказка:

Подумайте о связях между функциями и управляющими элементами.

- **`async`** — Ключевое слово для определения асинхронной функции
- **`await`** — Объект, ожидающий завершения другой операции
- **`event loop`** — Диспетчер, управляющий задачами
- **`coroutine`** — Специальная функция, которая может приостанавливать выполнение

🔗 Шаг 12:

Миссия 12

Сопоставьте ошибки и их причины, которые команда Minecraft хотела избежать при работе с `async/await`.

Ошибки:

Ответ:

- Забыли `await`
- Блокирующий вызов
- Отсутствие `event loop`
- Неправильное использование `coroutine`

Причины:

1. Функция не ждёт завершения операции
2. Код останавливает выполнение всего приложения
3. Асинхронный код не имеет диспетчера задач
4. Корутин вызван без запуска

🔗 Подсказка:

Вспомните, как важно правильно запускать и ожидать асинхронные операции.

- **Забыли `await`** — Функция не ждёт завершения операции
- **Блокирующий вызов** — Код останавливает выполнение всего приложения
- **Отсутствие `event loop`** — Асинхронный код не имеет диспетчера задач
- **Неправильное использование `coroutine`** — Корутин вызван без запуска

📖 Урок 2 - Сражение с Драконом Края — корутины и задачи

🔗 Шаг 01:

Миссия 1

Во время подготовки к сражению с Драконом Края, программист-герой Minecraft решил использовать корутины для одновременного выполнения нескольких задач. Что из списка лучше всего описывает, что такое корутина?

Подсказка: Корутины похожи на легковесные функции, которые можно приостанавливать и возобновлять.

Ответ:

- **Функция, которая может приостанавливаться и возобновляться во время выполнения**
- Параллельный поток, который всегда работает в отдельном ядре процессора
- Обычная функция без особенностей
- Многопоточная программа, использующая несколько процессоров

Шаг 02:

Миссия 2

Программист из команды Minecraft хочет запустить несколько задач одновременно для ослабления дракона. Какой из способов запуска корутин является правильным в Python?

Подсказка: Для запуска корутин обычно используют специальную функцию, которая управляет циклом событий.

Ответ:

- **`asyncio.run(имя_корутины())`**
- `threading.start(имя_функции())`
- `time.sleep(5)`
- `run(имя_функции())`

Шаг 03:

Миссия 3

В битве с Драконом Края программист создал корутину, но она не запускается. Какое ключевое слово обязательно должно стоять перед объявлением корутины?

Подсказка: Это ключевое слово указывает, что функция асинхронная и может быть приостановлена.

Ответ:

- **`async`**
- `await`
- `yield`
- `def`

Шаг 04:

Миссия 4

Сопоставьте понятия с их описаниями из сражения с Драконом Края:

Ответ:

- Корутина
- Задача (Task)
- Цикл событий (Event loop)
- `await`

Определения:

1. Объект, который запускает корутину и управляет её выполнением
2. Ключевое слово для приостановки корутины и ожидания результата другой корутины
3. Механизм для запуска и управления асинхронными операциями
4. Функция, которая может приостанавливаться и возобновляться

Подсказка: Подумайте, как корутины взаимодействуют с циклом событий и задачами.

- **Корутина** — Функция, которая может приостанавливаться и возобновляться
- **Задача (Task)** — Объект, который запускает корутину и управляет её выполнением
- **Цикл событий (Event loop)** — Механизм для запуска и управления асинхронными операциями
- **await** — Ключевое слово для приостановки корутины и ожидания результата другой корутины

Шаг 05:

Миссия 5

Сопоставьте примеры корутин с их поведением в бою с Драконом Края:

Ответ:

- `async def attack_dragon()`
- `async def heal_hero()`
- `async def recharge_shield()`
- `async def scan_for_obstacles()`

Определения:

1. Атака по дракону с задержкой между ударами
2. Восстановление здоровья героя после ранения
3. Обновление энергетического щита героя
4. Поиск препятствий на пути героя

Подсказка: Обратите внимание на названия и контекст каждой корутины.

- **`async def attack_dragon()`** — Атака по дракону с задержкой между ударами
- **`async def heal_hero()`** — Восстановление здоровья героя после ранения
- **`async def recharge_shield()`** — Обновление энергетического щита героя
- **`async def scan_for_obstacles()`** — Поиск препятствий на пути героя

Шаг 06:

Миссия 6

Сопоставьте функции `asyncio` с их назначением в сражении с Драконом Края:

Ответ:

- `asyncio.create_task()`
- `asyncio.gather()`
- `asyncio.sleep()`
- `asyncio.run()`

Определения:

1. Запускает основную корутину и цикл событий
2. Создаёт задачу для выполнения корутины параллельно

3. Ждёт выполнения сразу нескольких корутин
4. Приостанавливает выполнение корутины на заданное время

Подсказка: Подумайте, как эффективно координировать несколько корутин в бою.

- **`asyncio.run()`** — Запускает основную корутину и цикл событий
- **`asyncio.create_task()`** — Создаёт задачу для выполнения корутины параллельно
- **`asyncio.gather()`** — Ждёт выполнения сразу нескольких корутин
- **`asyncio.sleep()`** — Приостанавливает выполнение корутины на заданное время

Шаг 07:

Миссия 7

Расположите действия программиста по подготовке асинхронного сражения с Драконом Края в правильном порядке:

1. Определить корутины для разных задач (атака, лечение, защита)
2. Запустить цикл событий и создать задачи для корутин
3. Обеспечить правильное ожидание выполнения корутин через `await` или `gather`
4. Обработать исключения и ошибки, возникающие в корутинах
5. Завершить цикл событий и получить результаты сражения

Подсказка: Важно сначала определить логику, затем запустить и управлять задачами, а в конце корректно завершить работу.

Ответ:

- Определить корутины для разных задач (атака, лечение, защита)
- Запустить цикл событий и создать задачи для корутин
- Обеспечить правильное ожидание выполнения корутин через `await` или `gather`
- Обработать исключения и ошибки, возникающие в корутинах
- Завершить цикл событий и получить результаты сражения

Шаг 08:

Миссия 8

Расположите этапы отладки асинхронного кода в ходе битвы с Драконом Края:

1. Проверить, что все корутины запускаются и завершаются
2. Использовать логи для отслеживания выполнения корутин
3. Обращать внимание на исключения внутри корутин для предотвращения сбоев
4. Тестировать работу корутин с разными сценариями и задержками

Подсказка: Начинайте с проверки запуска, потом изучайте поведение, а потом улучшайте стабильность и тестируйте.

Ответ:

- Проверить, что все корутины запускаются и завершаются
- Использовать логи для отслеживания выполнения корутин
- Обращать внимание на исключения внутри корутин для предотвращения сбоев
- Тестировать работу корутин с разными сценариями и задержками

Шаг 09:

Миссия 9

Расположите в правильном порядке шаги обработки ошибок в асинхронном сражении с Драконом Края:

1. Добавить try-ехсепт блоки в корутины
2. Логировать ошибки для анализа
3. Обеспечить корректное завершение задач при ошибках
4. Перезапускать задачи, если это необходимо

Подсказка: Ошибки нужно сначала поймать, потом изучить, затем корректно обработать и при необходимости повторить задачу.

Ответ:

- Добавить try-ехсепт блоки в корутины
- Логировать ошибки для анализа
- Обеспечить корректное завершение задач при ошибках
- Перезапускать задачи, если это необходимо

Шаг 10:

Миссия 10

Расположите этапы работы с задачами (Tasks) в asyncio при сражении с Драконом Края:

1. Создание задач с помощью `asyncio.create_task()`
2. Запуск цикла событий с `asyncio.run()`
3. Ожидание завершения задач с `await` или `asyncio.gather()`
4. Обработка результатов задач

Подсказка: Важно правильно создавать, запускать, ожидать и затем обрабатывать задачи для эффективной работы.

Ответ:

- Создание задач с помощью `asyncio.create_task()`
- Запуск цикла событий с `asyncio.run()`
- Ожидание завершения задач с `await` или `asyncio.gather()`
- Обработка результатов задач

Шаг 11:

Миссия 11

Команда Minecraft-программистов запускает корутину для контроля полета стрелы дракона. Что из этого верно про корутины?

Подсказка: Подумайте, как корутины отличаются от обычных функций.

Ответ:

- **Корутину можно приостанавливать и возобновлять без блокировки программы**
- Корутину нужно запускать в отдельном процессе
- Корутину нельзя использовать с `async/await`
- Корутин всегда работает синхронно и завершает работу быстро

Шаг 12:

Миссия 12

При создании задачи для слежения за здоровьем дракона команда решила использовать `asyncio`. Что является главным преимуществом `asyncio`?

Подсказка: Подумайте, как можно выполнять несколько задач одновременно, не используя потоки.

Ответ:

- **Асинхронное выполнение множества задач без создания новых потоков**
- Автоматическое распараллеливание кода на все ядра процессора
- Обязательное использование корутин только с сетью
- Синхронное выполнение задач с очередностью

Шаг 13:

Миссия 13

В задаче «Атака дракона» нужно отменить задачу, если игрок покинул игру. Как правильно отменить `asyncio`-задачу?

Подсказка: Подумайте, какой метод позволяет корректно прервать выполнение задачи.

Ответ:

- **Вызвать метод `cancel()` у объекта задачи**
- Прервать выполнение корутины с помощью `raise Exception`
- Остановить `event loop` полностью
- Удалить объект задачи из памяти

Шаг 14:

Миссия 14

Сопоставьте термины и их определения, связанные с `asyncio` и корутинами в Minecraft-коде.

Ответ:

- Корутина
- `Task`
- `Event Loop`
- `await`

1. Объект, который запускает и управляет выполнением корутины
2. Ключевое слово для приостановки корутины до завершения другой корутины
3. Цикл, который обрабатывает все события и задачи
4. Функция, которая может быть приостановлена и возобновлена

Подсказка: Обратите внимание, что `await` — это оператор ожидания, а `Task` — это оболочка для корутины.

- **Корутина** — Функция, которая может быть приостановлена и возобновлена
- **`Task`** — Объект, который запускает и управляет выполнением корутины
- **`Event Loop`** — Цикл, который обрабатывает все события и задачи
- **`await`** — Ключевое слово для приостановки корутины до завершения другой корутины

Шаг 15:

Миссия 15

Сопоставьте состояния `asyncio`-задачи в сражении с драконом с их описаниями.

Ответ:

- `Pending`
- `Running`

- Cancelled
- Done

1. Задача завершена или отменена
2. Задача ожидает запуска или возобновления
3. Задача была прервана до завершения
4. Задача в процессе выполнения

Подсказка: Подумайте, как задачи переходят между состояниями во время выполнения.

- **Pending** — Задача ожидает запуска или возобновления
- **Running** — Задача в процессе выполнения
- **Cancelled** — Задача была прервана до завершения
- **Done** — Задача завершена или отменена

🔮 Шаг 16:

Миссия 16

Сопоставьте примеры кода с их описанием из мира Minecraft-программирования на Python asyncio.

Ответ:

- `async def fight_dragon(): ...`
 - `await asyncio.sleep(1)`
 - `task = asyncio.create_task(fight_dragon())`
 - `await task`
1. Создает корутину-функцию для боя с драконом
 2. Ожидание выполнения задачи (корутины)
 3. Запускает задачу в event loop
 4. Приостанавливает выполнение на 1 секунду

Подсказка: Внимательно следите за тем, что делают `async`, `await` и `create_task`.

- **`async def fight_dragon(): ...`** — Создает корутину-функцию для боя с драконом
- **`await asyncio.sleep(1)`** — Приостанавливает выполнение на 1 секунду
- **`task = asyncio.create_task(fight_dragon())`** — Запускает задачу в event loop
- **`await task`** — Ожидание выполнения задачи (корутины)

📖 Урок 3 - Добыча элитр — работа с API (requests)

🔮 Шаг 01:

Миссия 1

В команде Minecraft-программистов решили добыть элитры, используя API сервера. Какой HTTP-метод чаще всего используют для получения информации через API?

Подсказка: Этот метод предназначен для запроса данных без их изменения.

Ответ:

- GET
- POST
- PUT
- DELETE

Шаг 02:

Миссия 2

Программисты Minecraft пытаются подключиться к API, но сервер требует передать ключ API в заголовке запроса. Какой заголовок обычно используется для передачи токена авторизации?

Подсказка: Часто используется тип авторизации Bearer.

Ответ:

- **Authorization**
- Content-Type
- Accept
- User-Agent

Шаг 03:

Миссия 3

Для получения данных об элитрах с сервера Minecraft-программисты используют библиотеку requests в Python. Какой из вариантов правильно отправит GET-запрос к API по адресу `https://api.minecraftcraft.com/elytra`?

Подсказка: В requests для GET-запроса используется метод `requests.get`.

Ответ:

- **`requests.get('https://api.minecraftcraft.com/elytra')`**
- `requests.post('https://api.minecraftcraft.com/elytra')`
- `requests.put('https://api.minecraftcraft.com/elytra')`
- `requests.delete('https://api.minecraftcraft.com/elytra')`

Шаг 04:

Миссия 4

Сопоставьте ключевые части HTTP-запроса с их назначением при работе с API для добычи элитр:

Ответ:

- URL
- Метод
- Заголовки
- Тело запроса

Назначения:

1. Адрес ресурса на сервере
2. Тип операции (получение, отправка и др.)
3. Дополнительные параметры, например, авторизация или тип содержимого
4. Данные, отправляемые на сервер (часто для POST или PUT)

Подсказка: Подумайте, что нужно указать, чтобы запрос был понят серверу и выполнил нужное действие.

- **URL** — Адрес ресурса на сервере
- **Метод** — Тип операции (получение, отправка и др.)
- **Заголовки** — Дополнительные параметры, например, авторизация или тип содержимого
- **Тело запроса** — Данные, отправляемые на сервер (часто для POST или PUT)

Шаг 05:

Миссия 5

Команда Minecraft-программистов получает ответ от API. Сопоставьте элементы HTTP-ответа с их описанием:

Ответ:

- Status Code
- Headers
- Body

Описание:

1. Код состояния, показывающий результат запроса (например, 200, 404)
2. Метаданные, такие как тип содержимого и дата
3. Основные данные ответа, например, JSON с информацией об элитрах

Подсказка: Ответ всегда содержит код, данные и дополнительные параметры.

- **Status Code** — Код состояния, показывающий результат запроса (например, 200, 404)
- **Headers** — Метаданные, такие как тип содержимого и дата
- **Body** — Основные данные ответа, например, JSON с информацией об элитрах

Шаг 06:

Миссия 6

Программисты Minecraft определяют формат передачи данных при работе с API. Сопоставьте форматы с их описаниями:

Ответ:

- JSON
- XML
- CSV
- YAML

Описание:

1. Легковесный формат обмена данными, часто используемый с API
2. Текстовый формат с тегами, похожий на HTML
3. Табличный формат, удобный для данных в строках и столбцах
4. Читаемый формат с отступами для структурирования данных

Подсказка: Подумайте, какой формат самый популярный для веб-API и какой более человекочитаемый.

- **JSON** — Легковесный формат обмена данными, часто используемый с API
- **XML** — Текстовый формат с тегами, похожий на HTML
- **CSV** — Табличный формат, удобный для данных в строках и столбцах
- **YAML** — Читаемый формат с отступами для структурирования данных

Шаг 07:

Миссия 7

Вопрос: Расположите шаги запроса к API для получения данных об элитрах в правильном порядке:

1. Обработка ответа сервера (парсинг JSON)

2. Отправка GET-запроса на сервер
3. Подготовка URL и заголовков с авторизацией
4. Использование полученных данных в программе

Подсказка: Подумайте, сначала надо подготовиться, потом отправить, потом обработать, и только после использовать.

Ответ:

- Подготовка URL и заголовков с авторизацией
- Отправка GET-запроса на сервер
- Обработка ответа сервера (парсинг JSON)
- Использование полученных данных в программе

Шаг 08:

Миссия 8

Вопрос: Расположите этапы обработки ошибки при работе с API:

1. Логирование ошибки для анализа
2. Проверка кода состояния ответа
3. Вывод сообщения пользователю
4. Повторный запрос или переход к альтернативному сценарию

Подсказка: Сначала нужно проверить ответ, затем сообщить, сохранить данные об ошибке, и потом решить, что делать дальше.

Ответ:

- Проверка кода состояния ответа
- Вывод сообщения пользователю
- Логирование ошибки для анализа
- Повторный запрос или переход к альтернативному сценарию

Шаг 09:

Миссия 9

Вопрос: Расположите действия при обновлении кода, который взаимодействует с API добычи элитр, чтобы избежать сбоев:

1. Написание и запуск тестов для проверки новой логики
2. Анализ документации API на предмет изменений
3. Разработка и интеграция обновленного кода
4. Деплой изменений в продакшен

Подсказка: Необходимо сначала понять изменения, потом написать код, проверить и только потом внедрить.

Ответ:

- Анализ документации API на предмет изменений
- Разработка и интеграция обновленного кода
- Написание и запуск тестов для проверки новой логики
- Деплой изменений в продакшен

Шаг 10:

Миссия 10

Вопрос: Расположите этапы успешного взаимодействия Minecraft-программиста с API для добычи элитр в правильном порядке:

1. Подключение к API с использованием ключа авторизации
2. Отправка запроса с правильными параметрами
3. Получение и проверка ответа
4. Обработка и использование данных в игре

Подсказка: Сначала надо авторизоваться, затем отправить запрос, проверить ответ и использовать данные.

Ответ:

- Подключение к API с использованием ключа авторизации
- Отправка запроса с правильными параметрами
- Получение и проверка ответа
- Обработка и использование данных в игре

Шаг 11:

Миссия 1

Вы — программисты Minecraft, и вам нужно понять, что такое асинхронность. Какое определение `async/await` наиболее точное?

Подсказка: Подумайте о том, как игра может выполнять несколько действий одновременно, не блокируя главный процесс.

Миссия 2

Какой из этих вариантов лучше всего описывает, зачем в Python используют `async/await`?

Подсказка: Представьте, что ваш персонаж Minecraft ждет открытия сундука, но не должен просто стоять на месте.

Миссия 3

Ваш Minecraft-бот отправляется в поход по Энд. Какой метод из `async/await` позволит не блокировать игру, ожидая ответа сервера?

Подсказка: Обратите внимание на ключевое слово, которое приостанавливает выполнение, позволяя другим процессам идти дальше.

Миссия 4

Сопоставьте ключевые понятия асинхронности с их описаниями.

Ответ:

- `async`
 - `await`
 - корутина
 - цикл событий
1. Позволяет приостанавливать выполнение функции до завершения асинхронной операции
 2. Ключевое слово для объявления асинхронной функции
 3. Объект, который можно приостанавливать и возобновлять
 4. Механизм управления асинхронными операциями в Python

Подсказка: Подумайте, какое слово используется для создания, а какое — для ожидания, а что управляет процессом.

Миссия 5

Сопоставьте ситуации с правильными способами использования `async/await` в Minecraft.

- Обработка событий от игроков

- Загрузка данных с сервера
- Обновление интерфейса
- Проверка соединения с базой данных

1. `await`
2. `async def`
3. `asyncio.run()`
4. `callback`

Подсказка: Обратите внимание, когда нужно приостановить функцию, а когда — запустить весь цикл.

Миссия 6

Сопоставьте ошибки с их возможными причинами при использовании `async/await` в Minecraft-проектах.

- Корутину вызвали без `await`
- Использование blocking I/O внутри `async` функции
- Неправильный цикл событий
- Отсутствие `async` перед функцией

1. Функция не ожидает завершения корутины
2. Блокировка основного потока, несмотря на `async`
3. Некорректное управление задачами
4. Функция не является асинхронной

Подсказка: Вспомните, как важно правильно писать `async` функции и использовать `await`.

Миссия 7

Вы — программисты Minecraft, и вам нужно понять, что такое асинхронность. Какое определение `async/await` наиболее точное?

Подсказка: Подумайте о том, как игра может выполнять несколько действий одновременно, не блокируя главный процесс.

Миссия 8

Какой из этих вариантов лучше всего описывает, зачем в Python используют `async/await`?

Подсказка: Представьте, что ваш персонаж Minecraft ждет открытия сундука, но не должен просто стоять на месте.

Миссия 9

Ваш Minecraft-бот отправляется в поход по Энд. Какой метод из `async/await` позволит не блокировать игру, ожидая ответа сервера?

Подсказка: Обратите внимание на ключевое слово, которое приостанавливает выполнение, позволяя другим процессам идти дальше.

Миссия 10

Сопоставьте ключевые понятия асинхронности с их описаниями.

- `async`
- `await`
- корутина
- цикл событий

1. Позволяет приостанавливать выполнение функции до завершения асинхронной операции
2. Ключевое слово для объявления асинхронной функции
3. Объект, который можно приостанавливать и возобновлять
4. Механизм управления асинхронными операциями в Python

Подсказка: Подумайте, какое слово используется для создания, а какое — для ожидания, а что управляет процессом.

Миссия 11

Сопоставьте ситуации с правильными способами использования `async/await` в Minecraft.

- Обработка событий от игроков
- Загрузка данных с сервера
- Обновление интерфейса
- Проверка соединения с базой данных

1. `await`
2. `async def`
3. `asyncio.run()`
4. `callback`

Подсказка: Обратите внимание, когда нужно приостановить функцию, а когда — запустить весь цикл.

Миссия 12

Сопоставьте ошибки с их возможными причинами при использовании `async/await` в Minecraft-проектах.

- Корутину вызвали без `await`
- Использование blocking I/O внутри `async` функции
- Неправильный цикл событий
- Отсутствие `async` перед функцией

1. Функция не ожидает завершения корутины
2. Блокировка основного потока, несмотря на `async`
3. Некорректное управление задачами
4. Функция не является асинхронной

Подсказка: Вспомните, как важно правильно писать `async` функции и использовать `await`.

Миссия 7

Программист из команды Minecraft решил изучить `async/await`. Что означает ключевое слово `async` в Python?

Подсказка: Обратите внимание, что `async` делает функцию особой — не просто запускает её, а даёт возможность работать с ожиданиями.

Миссия 8

В Minecraft команда разработчиков использует `await`. Что делает ключевое слово `await` в асинхронной функции?

Подсказка: Подумайте, как ожидание результата может не блокировать остальные действия.

Миссия 9

Команда Minecraft использует `asyncio`. Что такое `event loop` в контексте `async/await`?

Подсказка: Подумайте о центральном диспетчере, который следит за выполнением задач.

Миссия 10

При асинхронном программировании в Minecraft возникает проблема "блокировки". Что это означает?

Подсказка: Вспомните, что происходит, когда выполнение одного кода задерживает другой.

Миссия 11

Сопоставьте понятия из асинхронного программирования и их объяснения, которые команда Minecraft обсуждала после похода в Энд.

Понятия:

- `async`
- `await`
- `event loop`
- `coroutine`

Определения:

1. Специальная функция, которая может приостанавливать выполнение
2. Ключевое слово для определения асинхронной функции
3. Объект, ожидающий завершения другой операции
4. Диспетчер, управляющий задачами

🔮 **Подсказка:**

Подумайте о связях между функциями и управляющими элементами.

Миссия 12

Сопоставьте ошибки и их причины, которые команда Minecraft хотела избежать при работе с `async/await`.

Ошибки:

- Забыли `await`
- Блокирующий вызов
- Отсутствие `event loop`
- Неправильное использование `coroutine`

Причины:

1. Функция не ждёт завершения операции
2. Код останавливает выполнение всего приложения
3. Асинхронный код не имеет диспетчера задач
4. Корутин вызван без запуска

🔮 **Подсказка:**

Вспомните, как важно правильно запускать и ожидать асинхронные операции.

Миссия 1

Во время подготовки к сражению с Драконом Края, программист-герой Minecraft решил использовать корутины для одновременного выполнения нескольких задач. Что из списка лучше всего описывает, что такое корутина?

Подсказка: Корутины похожи на легковесные функции, которые можно приостанавливать и возобновлять.

Миссия 2

Программист из команды Minecraft хочет запустить несколько задач одновременно для ослабления дракона. Какой из способов запуска корутин является правильным в Python?

Подсказка: Для запуска корутин обычно используют специальную функцию, которая управляет циклом событий.

Миссия 3

В битве с Драконом Края программист создал корутину, но она не запускается. Какое ключевое слово обязательно должно стоять перед объявлением корутины?

Подсказка: Это ключевое слово указывает, что функция асинхронная и может быть приостановлена.

Миссия 4

Сопоставьте понятия с их описаниями из сражения с Драконом Края:

- Корутина
- Задача (Task)
- Цикл событий (Event loop)
- await

Определения:

1. Объект, который запускает корутину и управляет её выполнением
2. Ключевое слово для приостановки корутины и ожидания результата другой корутины
3. Механизм для запуска и управления асинхронными операциями
4. Функция, которая может приостанавливаться и возобновляться

Подсказка: Подумайте, как корутины взаимодействуют с циклом событий и задачами.

Миссия 5

Сопоставьте примеры корутин с их поведением в бою с Драконом Края:

- `async def attack_dragon()`
- `async def heal_hero()`
- `async def recharge_shield()`
- `async def scan_for_obstacles()`

Определения:

1. Атака по дракону с задержкой между ударами
2. Восстановление здоровья героя после ранения
3. Обновление энергетического щита героя
4. Поиск препятствий на пути героя

Подсказка: Обратите внимание на названия и контекст каждой корутины.

Миссия 6

Сопоставьте функции `asyncio` с их назначением в сражении с Драконом Края:

- `asyncio.create_task()`
- `asyncio.gather()`
- `asyncio.sleep()`
- `asyncio.run()`

Определения:

1. Запускает основную корутину и цикл событий
2. Создаёт задачу для выполнения корутины параллельно
3. Ждёт выполнения сразу нескольких корутин
4. Приостанавливает выполнение корутины на заданное время

Подсказка: Подумайте, как эффективно координировать несколько корутин в бою.

Миссия 7

Расположите действия программиста по подготовке асинхронного сражения с Драконом Края в правильном порядке:

1. Определить корутины для разных задач (атака, лечение, защита)
2. Запустить цикл событий и создать задачи для корутин
3. Обеспечить правильное ожидание выполнения корутин через `await` или `gather`
4. Обработать исключения и ошибки, возникающие в корутинах

5. Завершить цикл событий и получить результаты сражения

Подсказка: Важно сначала определить логику, затем запустить и управлять задачами, а в конце корректно завершить работу.

Миссия 8

Расположите этапы отладки асинхронного кода в ходе битвы с Драконом Края:

1. Проверить, что все корутины запускаются и завершаются
2. Использовать логи для отслеживания выполнения корутин
3. Обрабатывать исключения внутри корутин для предотвращения сбоев
4. Тестировать работу корутин с разными сценариями и задержками

Подсказка: Начинайте с проверки запуска, потом изучайте поведение, а потом улучшайте стабильность и тестируйте.

Миссия 9

Расположите в правильном порядке шаги обработки ошибок в асинхронном сражении с Драконом Края:

1. Добавить try-excerpt блоки в корутины
2. Логировать ошибки для анализа
3. Обеспечить корректное завершение задач при ошибках
4. Перезапустить задачи, если это необходимо

Подсказка: Ошибки нужно сначала поймать, потом изучить, затем корректно обработать и при необходимости повторить задачу.

Миссия 10

Расположите этапы работы с задачами (Tasks) в asyncio при сражении с Драконом Края:

1. Создание задач с помощью `asyncio.create_task()`
2. Запуск цикла событий с `asyncio.run()`
3. Ожидание завершения задач с `await` или `asyncio.gather()`
4. Обработка результатов задач

Подсказка: Важно правильно создавать, запускать, ожидать и затем обрабатывать задачи для эффективной работы.

Миссия 11

Команда Minecraft-программистов запускает корутину для контроля полета стрелы дракона. Что из этого верно про корутины?

Подсказка: Подумайте, как корутины отличаются от обычных функций.

Миссия 12

При создании задачи для слежения за здоровьем дракона команда решила использовать `asyncio`. Что является главным преимуществом `asyncio`?

Подсказка: Подумайте, как можно выполнять несколько задач одновременно, не используя потоки.

Миссия 13

В задаче «Атака дракона» нужно отменить задачу, если игрок покинул игру. Как правильно отменить `asyncio`-задачу?

Подсказка: Подумайте, какой метод позволяет корректно прервать выполнение задачи.

Миссия 14

Сопоставьте термины и их определения, связанные с `asyncio` и корутинами в Minecraft-коде.

- Корутина
- Task

- Event Loop
- await

1. Объект, который запускает и управляет выполнением корутины
2. Ключевое слово для приостановки корутины до завершения другой корутины
3. Цикл, который обрабатывает все события и задачи
4. Функция, которая может быть приостановлена и возобновлена

Подсказка: Обратите внимание, что `await` — это оператор ожидания, а `Task` — это оболочка для корутины.

Миссия 15

Сопоставьте состояния `asyncio`-задачи в сражении с драконом с их описаниями.

- Pending
- Running
- Cancelled
- Done

1. Задача завершена или отменена
2. Задача ожидает запуска или возобновления
3. Задача была прервана до завершения
4. Задача в процессе выполнения

Подсказка: Подумайте, как задачи переходят между состояниями во время выполнения.

Миссия 16

Сопоставьте примеры кода с их описанием из мира Minecraft-программирования на Python `asyncio`.

- `async def fight_dragon(): ...`
- `await asyncio.sleep(1)`
- `task = asyncio.create_task(fight_dragon())`
- `await task`

1. Создает корутину-функцию для боя с драконом
2. Ожидание выполнения задачи (корутины)
3. Запускает задачу в event loop
4. Приостанавливает выполнение на 1 секунду

Подсказка: Внимательно следите за тем, что делают `async`, `await` и `create_task`.

Миссия 17

Расположите этапы создания и запуска корутины для атаки дракона в правильном порядке:

1. Создать корутину-функцию `async def`
2. Вызвать `asyncio.create_task` для запуска задачи
3. Запустить event loop или дождаться завершения задачи
4. Использовать `await` для ожидания результата

Подсказка: Помните, что корутины создаются, запускаются, а затем ожидаются.

Миссия 18

Расположите действия для правильной отмены `asyncio`-задачи, если игрок сбежал из боя с драконом:

1. Вызвать `cancel()` у задачи
2. Обработать `asyncio.CancelledError` внутри корутины

3. Дождаться завершения задачи с `await`
4. Очистить ресурсы и продолжить логику игры

Подсказка: Отмена — это не просто остановка, нужно корректно обработать исключения и завершить задачи.

Миссия 19

Расположите этапы отладки и оптимизации асинхронного кода, когда команда улучшает сражение с драконом:

1. Добавить логирование состояний задач
2. Использовать профилировщик для поиска узких мест
3. Рефакторить корутины для повышения читаемости
4. Проверить корректность работы `cancel` и обработки исключений

Подсказка: Сначала нужно понять, где проблема, потом улучшить код и проверить надежность.

Миссия 20

Расположите этапы взаимодействия между программой и gRPC-сервером, который управляет поведением дракона в Minecraft:

1. Установить соединение с gRPC-сервером
2. Создать корутину для отправки запросов и получения ответов
3. Запустить event loop и создать задачи для обмена сообщениями
4. Обработать ответы от сервера и обновить состояние игры

Подсказка: gRPC требует установления соединения, асинхронного обмена сообщениями и обработки результатов.

Миссия 1

В команде Minecraft-программистов решили добыть элитры, используя API сервера. Какой HTTP-метод чаще всего используют для получения информации через API?

Подсказка: Этот метод предназначен для запроса данных без их изменения.

Миссия 2

Программисты Minecraft пытаются подключиться к API, но сервер требует передать ключ API в заголовке запроса. Какой заголовок обычно используется для передачи токена авторизации?

Подсказка: Часто используется тип авторизации Bearer.

Миссия 3

Для получения данных об элитрах с сервера Minecraft-программисты используют библиотеку `requests` в Python. Какой из вариантов правильно отправит GET-запрос к API по адресу `https://api.minecraftcraft.com/elytra`?

Подсказка: В `requests` для GET-запроса используется метод `requests.get`.

Миссия 4

Сопоставьте ключевые части HTTP-запроса с их назначением при работе с API для добычи элитр:

- URL
- Метод
- Заголовки
- Тело запроса

Назначения:

1. Адрес ресурса на сервере
2. Тип операции (получение, отправка и др.)
3. Дополнительные параметры, например, авторизация или тип содержимого
4. Данные, отправляемые на сервер (часто для POST или PUT)

Подсказка: Подумайте, что нужно указать, чтобы запрос был понят серверу и выполнил нужное действие.

Миссия 5

Команда Minecraft-программистов получает ответ от API. Сопоставьте элементы HTTP-ответа с их описанием:

- Status Code
- Headers
- Body

Описание:

1. Код состояния, показывающий результат запроса (например, 200, 404)
2. Метаданные, такие как тип содержимого и дата
3. Основные данные ответа, например, JSON с информацией об элитрах

Подсказка: Ответ всегда содержит код, данные и дополнительные параметры.

Миссия 6

Программисты Minecraft определяют формат передачи данных при работе с API. Сопоставьте форматы с их описаниями:

- JSON
- XML
- CSV
- YAML

Описание:

1. Легковесный формат обмена данными, часто используемый с API
2. Текстовый формат с тегами, похожий на HTML
3. Табличный формат, удобный для данных в строках и столбцах
4. Читаемый формат с отступами для структурирования данных

Подсказка: Подумайте, какой формат самый популярный для веб-API и какой более человекочитаемый.

Миссия 7

Вопрос: Расположите шаги запроса к API для получения данных об элитрах в правильном порядке:

1. Обработка ответа сервера (парсинг JSON)
2. Отправка GET-запроса на сервер
3. Подготовка URL и заголовков с авторизацией
4. Использование полученных данных в программе

Подсказка: Подумайте, сначала надо подготовиться, потом отправить, потом обработать, и только после использовать.

Миссия 8

Вопрос: Расположите этапы обработки ошибки при работе с API:

1. Логирование ошибки для анализа
2. Проверка кода состояния ответа
3. Вывод сообщения пользователю
4. Повторный запрос или переход к альтернативному сценарию

Подсказка: Сначала нужно проверить ответ, затем сообщить, сохранить данные об ошибке, и потом решить, что делать дальше.

Миссия 9

Вопрос: Расположите действия при обновлении кода, который взаимодействует с API добычи элитр, чтобы избежать сбоев:

1. Написание и запуск тестов для проверки новой логики
2. Анализ документации API на предмет изменений
3. Разработка и интеграция обновленного кода
4. Деплой изменений в продакшен

Подсказка: Необходимо сначала понять изменения, потом написать код, проверить и только потом внедрить.

Миссия 10

Вопрос: Расположите этапы успешного взаимодействия Minecraft-программиста с API для добычи элитр в правильном порядке:

1. Подключение к API с использованием ключа авторизации
2. Отправка запроса с правильными параметрами
3. Получение и проверка ответа
4. Обработка и использование данных в игре

Подсказка: Сначала надо авторизоваться, затем отправить запрос, проверить ответ и использовать данные.

- **Слишком много запросов** — нужно подождать
- Ресурс не найден
- Ошибка сервера
- Неверный запрос

Урок 4 - Ферма опыта — базы данных (SQLite)

Шаг 01:

Миссия 1

Программисты-герои Minecraft собрались автоматизировать ферму опыта. Что из перечисленного — ****основное преимущество**** использования SQLite в небольшом проекте фермы опыта?

Подсказка: Подумайте о лёгкости развёртывания и отсутствии зависимости от сервера.

Ответ:

- **Минимальные требования к инфраструктуре** — одна библиотека, один файл.
- Масштабируемость до сотен серверов без любой настройки.
- Полная поддержка распределённых транзакций между узлами сети.
- Нативная поддержка шифрования на уровне файлов без настроек.

Шаг 02:

Миссия 2

Команда grps хочет хранить информацию об игроках, которые добывают опыт. Какой тип данных в SQLite лучше всего подходит для хранения полей вроде "уровень игрока"?

Подсказка: Уровень — целое число, не дробное.

Ответ:

- **INTEGER**
- TEXT

- REAL
- BLOB

🔗 Шаг 03:

Миссия 3

Ферма опыта написана с помощью grpc-сервиса, который логирует события "игрок получил опыт" в SQLite. Какой подход обеспечивает ****атомарность**** операции добавления опыта?

Подсказка: Подумайте о транзакциях и rollback.

Ответ:

- **Обёрнуть INSERT и UPDATE в BEGIN TRANSACTION ... COMMIT.**
- Выполнить два отдельных SQL-запроса подряд без BEGIN/COMMIT.
- Сначала COMMIT, потом BEGIN TRANSACTION для другого запроса.
- Записать в таблицу логов, потом обновить опыт без контроля ошибок.

🔗 Шаг 04:

Миссия 4

Задача: Сопоставьте термины с их определениями, которые используют программисты Minecraft-фермы опыта, работая с SQLite.

Термины:

Ответ:

- VACUUM
- PRAGMA journal_mode
- PRIMARY KEY
- FOREIGN KEY

Определения:

1. Ограничение, обеспечивающее уникальность и идентификацию строки.
2. Устанавливает, как SQLite управляет журналом транзакций.
3. Команда для очищения и реорганизации файла базы данных, уменьшения фрагментации.
4. Связь между таблицами для поддержания целостности.

Подсказка:

PRAGMA — это команда управления настройками. VACUUM освобождает место внутри файла.

- **VACUUM** — Команда для очищения и реорганизации файла базы данных, уменьшения фрагментации.
- **PRAGMA journal_mode** — Устанавливает, как SQLite управляет журналом транзакций.
- **PRIMARY KEY** — Ограничение, обеспечивающее уникальность и идентификацию строки.
- **FOREIGN KEY** — Связь между таблицами для поддержания целостности.

🔗 Шаг 05:

Миссия 5

Задача: Сопоставьте операции SQLite с тем, что они делают на ферме опыта.

Операции:

Ответ:

- INSERT
- UPDATE

- DELETE
- SELECT

Действия на ферме:

1. Добавить нового игрока в базу.
2. Изменить количество опыта игрока.
3. Убрать запись об игроке, который покинул сервер.
4. Получить список всех игроков с уровнем выше 10.

Подсказка:

INSERT = добавление, UPDATE — изменение, DELETE — удаление, SELECT — чтение.

- **INSERT** — Добавить нового игрока в базу.
- **UPDATE** — Изменить количество опыта игрока.
- **DELETE** — Убрать запись об игроке, который покинул сервер.
- **SELECT** — Получить список всех игроков с уровнем выше 10.

 Шаг 06:

Миссия 6

Задача: Сопоставьте понятия безопасности и целостности данных с мерами в SQLite, важными для грс-сервиса фермы опыта.

Понятия:

Ответ:

- Атомарность
- Изоляция
- Целостность ссылок
- Устойчивость (durability)

Меры:

1. Использование транзакций, rollback при ошибках.
2. Конфигурация journal_mode и WAL для надёжного журнала.
3. FOREIGN KEY ограничения между таблицами.
4. Гарантия, что после COMMIT данные сохранены даже при сбое сервера.

Подсказка:

ACID — важна каждая буква: атомарность, консистентность, изоляция, устойчивость.

- **Атомарность** — Использование транзакций, rollback при ошибках.
- **Изоляция** — Конфигурация journal_mode и WAL для надёжного журнала.
- **Целостность ссылок** — FOREIGN KEY ограничения между таблицами.
- **Устойчивость (durability)** — Гарантия, что после COMMIT данные сохранены даже при сбое сервера.

 Шаг 07:

Миссия 7

Задача: Сопоставьте элементы sqlite-схемы с их примерами, которые могут использовать герои Minecraft для фермы опыта.

Элементы схемы:

Ответ:

- Таблица опыта (experience)
- Таблица игроков (players)
- Связующая таблица задания-игрок (quest_player)
- Индекс по уровню (idx_level)

Примеры:

1. Хранит уникальные данные об игроках: имя, UUID, дата присоединения.
2. Определяет, какие игроки выполнили какие задания на опыт.
3. Быстрый поиск всех игроков выше определённого уровня.
4. Отдельная таблица для уровня опыта, часов игры и т.п.

Подсказка:

INDEX даёт ускорение поиска, связующая таблица реализует многие-ко-многим.

- **Таблица опыта (experience)** — Отдельная таблица для уровня опыта, часов игры и т.п.
- **Таблица игроков (players)** — Хранит уникальные данные об игроках: имя, UUID, дата присоединения.
- **Связующая таблица задания-игрок (quest_player)** — Определяет, какие игроки выполнили какие задания на опыт.
- **Индекс по уровню (idx_level)** — Быстрый поиск всех игроков выше определённого уровня.

 Шаг 08:

Вопрос: Отсортируйте шаги миграции схемы SQLite для фермы опыта, когда добавляется новая таблица для логов опыта.

1. Применить ALTER TABLE или создать новую таблицу.
2. Обновить код grps-сервиса, чтобы писать логи в новую таблицу.
3. Создать индекс для ускорения запросов по времени.
4. Написать миграцию, проверить на тестовой базе.

Подсказка:

Сначала схема, потом код, затем оптимизация, и не забывайте тестирование.

Ответ:

- Написать миграцию, проверить на тестовой базе.
- Применить ALTER TABLE или создать новую таблицу.
- Обновить код grps-сервиса, чтобы писать логи в новую таблицу.
- Создать индекс для ускорения запросов по времени.

 Шаг 09:

Вопрос: Отсортируйте этапы восстановления после сбоя сервера grps, чтобы минимизировать потерю данных базы SQLite на ферме опыта.

1. Переключиться на backup-файл, если WAL повреждён.
2. Остановить grps-сервис, чтобы прекратить новые записи.
3. Выполнить проверку integrity базы с PRAGMA integrity_check.
4. Восстановить транзакции из WAL (если используется WAL-режим).

Подсказка:

Безопасность данных важна: сначала остановка, проверка, восстановление, резерв.

Ответ:

- Остановить grpc-сервис, чтобы прекратить новые записи.
- Выполнить проверку integrity базы с PRAGMA integrity_check.
- Восстановить транзакции из WAL (если используется WAL-режим).
- Переключиться на backup-файл, если WAL повреждён.

🔮 Шаг 10:

Миссия 10

Вопрос: Расположите по порядку этапы оптимизации производительности чтения из SQLite базы фермы опыта при большом числе SELECT-запросов.

1. Добавить необходимые индексы (по уровню, по времени).
2. Профилировать запросы, найти медленные SELECT-запросы.
3. Использовать prepare statements / параметризованные запросы в grpc-сервисе.
4. **Пересмотреть схему таблиц** — нормализовать, денормализовать, при необходимости разделить таблицы.

Подсказка:

Понимание узких мест — ключ: схема, запросы, индексы, исполнение.

Ответ:

- Профилировать запросы, найти медленные SELECT-запросы.
- **Пересмотреть схему таблиц** — нормализовать, денормализовать, при необходимости разделить таблицы.
- Добавить необходимые индексы (по уровню, по времени).
- Использовать prepare statements / параметризованные запросы в grpc-сервисе.

📖 Урок 5 - Автоматизация деревни — ORM (SQLAlchemy)

🔮 Шаг 01:

Миссия 1

Разработчики Minecraft-деревни хотят хранить данные о жителях (NPC) в базе данных с помощью SQLAlchemy. Какой класс SQLAlchemy отвечает за представление таблицы в виде модели?

Подсказка: Это класс, от которого наследуются все модели.

Ответ:

- **declarative_base().Base / Base**
- Session
- engine
- Query

🔮 Шаг 02:

Миссия 2

При автоматизации деревни программная команда хочет изменить поведение сессии SQLAlchemy: чтобы изменения сохранялись только при явном вызове commit(). Какой параметр контролирует это?

Подсказка: Это касается autoflush и autoflush автоматически? Подумайте об autoflush и expire_on_commit.

Ответ:

- **Session(autocommit=False, autoflush=False)**
- Session(autoflush=True)
- Session(commit_on_close=True)
- Session(expire_on_commit=False)

🔮 Шаг 03:

Миссия 3

Во время работы с геймерами-NPC команда хочет быстро получить всех жителей, у которых есть хотя бы одно здание. Какой метод SQLAlchemy поможет отфильтровать таких жителей?

Подсказка: Используйте оператор .join() или .exists().

Ответ:

- **session.query(Villager).join(Building).distinct()**
- session.query(Villager).filter(Villager.buildings==None)
- session.query(Building).all()
- session.query(Villager).filter(Villager.id > 0)

🔮 Шаг 04:

Миссия 4

Сопоставьте функции SQLAlchemy с их назначением.

Функции/атрибуты:

Ответ:

- declarative_base()
- session.add(obj)
- query.filter()
- relationship(...)

Назначение:

1. Добавляет объект в сессию для дальнейшей записи в базу
2. Создает базовый класс для моделей
3. Позволяет указать связи между моделями
4. Фильтрует результаты запроса

Подсказка: Посмотрите, что делает каждый метод — добавление, фильтрация, связь.

- **declarative_base()** — Создает базовый класс для моделей
- **session.add(obj)** — Добавляет объект в сессию для дальнейшей записи в базу
- **query.filter()** — Фильтрует результаты запроса
- **relationship(...)** — Позволяет указать связи между моделями

🔮 Шаг 05:

Миссия 5

Сопоставьте термины с их описаниями в контексте ORM деревни.

Термины:

Ответ:

- lazy loading

- eager loading
- back_populates
- ForeignKey

Описания:

1. Связь обратного свойства между отношениями моделей
2. Доступ к связанным объектам производится при первом обращении
3. Указывает внешний ключ в таблице, связывающий с другой моделью
4. Загружать все связанные объекты сразу при запросе

Подсказка: Подумайте, когда данные загружаются сразу, а когда — отложено.

- **lazy loading** — Доступ к связанным объектам производится при первом обращении
- **eager loading** — Загружать все связанные объекты сразу при запросе
- **back_populates** — Связь обратного свойства между отношениями моделей
- **ForeignKey** — Указывает внешний ключ в таблице, связывающий с другой моделью

Шаг 06:

Миссия 6

Вопрос: Упорядочите этапы работы с транзакциями в SQLAlchemy во время автоматизации деревни, чтобы обеспечить атомарность.

1. session.commit()
2. session.begin()
3. выполнение операций insert/update/delete
4. в случае ошибки выполнить rollback()

Подсказка: Транзакция — это блок: начать, сделать, сохранить или отменить.

Ответ:

- session.begin()
- выполнение операций insert/update/delete
- session.commit()
- в случае ошибки выполнить rollback()

Шаг 07:

Миссия 7

Вопрос: Расположите шаги оптимизации ORM-запросов в деревне Minecraft-команды, когда они хотят ускорить загрузку связанных данных.

1. Обнаружить N+1 проблему
2. Измерить время запроса
3. Применить eager loading (например, joinedload или subqueryload)
4. Профилировать снова, убедиться, что ускорение произошло

Подсказка: Сначала найти проблему, затем оптимизировать и проверить изменения.

Ответ:

- Обнаружить N+1 проблему
- Измерить время запроса
- Применить eager loading (например, joinedload или subqueryload)
- Профилировать снова, убедиться, что ускорение произошло

🔧 Шаг 08:

Миссия 8

Команда Minecraft готовится автоматизировать управление жителями деревни с помощью ORM SQLAlchemy. Какой класс в SQLAlchemy используется для создания модели таблицы?

Подсказка: Этот класс является основой для создания всех моделей в SQLAlchemy.

Ответ:

- **declarative_base().Base**
- Session
- engine
- Query

🔧 Шаг 09:

Миссия 9

В процессе автоматизации Minecraft-деревни, необходимо наладить связь между таблицами "villagers" и "buildings". Как правильно организовать связь один ко многим, где один житель может управлять многими зданиями?

Подсказка: Используйте внешний ключ и настройку отношений в SQLAlchemy.

Ответ:

- **relationship('Building', back_populates='villager')** и **ForeignKey** на стороне buildings
- только Column(Integer) в таблице buildings
- relationship('Villager', many_to_many='buildings_villagers')
- использовать чистый SQL без relationship

🔧 Шаг 10:

Миссия 10

Автоматизация деревни требует, чтобы изменения в базе данных сохранялись только после явного вызова commit(). Как настроить сессию SQLAlchemy для выполнения этого?

Подсказка: Рассмотрите параметры сессии для автокоммита и автофлуша.

Ответ:

- **Session(autocommit=False, autoflush=False)**
- Session(autoflush=True)
- Session(commit_on_close=True)
- Session(expire_on_commit=False)

🔧 Шаг 11:

Миссия 11

При автоматизации Minecraft-деревни необходимо получить всех жителей, у которых есть хотя бы одно здание. Какой запрос в SQLAlchemy поможет выполнить эту задачу?

Подсказка: Используйте метод join() или exists() для фильтрации данных.

Ответ:

- **session.query(Villager).join(Building).distinct()**
- session.query(Villager).filter(Villager.buildings==None)
- session.query(Building).all()
- session.query(Villager).filter(Villager.id > 0)

🔮 Шаг 12:

Миссия 12

Сопоставьте функции SQLAlchemy с их назначением:

Подсказка: Подумайте, что делает каждая функция в контексте работы с базой данных.

Ответ:

- **declarative_base()** — Создает базовый класс для моделей
- **session.add(obj)** — Добавляет объект в сессию для дальнейшей записи в базу
- **query.filter()** — Фильтрует результаты запроса
- **relationship(...)** — Позволяет указать связи между моделями

🔮 Шаг 13:

Миссия 13

Сопоставьте термины SQLAlchemy с их описаниями:

Подсказка: Обратите внимание на термины, связанные с загрузкой данных и связями между моделями.

Ответ:

- **lazy loading** — Доступ к связанным объектам производится при первом обращении
- **eager loading** — Загружать все связанные объекты сразу при запросе
- **back_populates** — Связь обратного свойства между отношениями моделей
- **ForeignKey** — Указывает внешний ключ в таблице, связывающий с другой моделью

🔮 Шаг 14:

Миссия 14

Сопоставьте события SQLAlchemy с их типичным применением:

Подсказка: События 'before_*' выполняются до изменения данных, 'after_*' — после.

Ответ:

- **before_insert** — Валидация данных перед сохранением нового NPC
- **after_commit** — Логирование после фиксации транзакции
- **before_delete** — Очистка связанных записей перед удалением здания
- **after_update** — Уведомление жителей об изменении здания

📖 Урок 6 - Хранилище ресурсов 2.0 — PostgreSQL

🔮 Шаг 01:

Миссия 1

Команда Minecraft-программистов разрабатывает ресурсное хранилище на PostgreSQL через gprc-сервис. Какой тип данных лучше всего использовать для хранения больших двоичных файлов (например, текстур), если они хранятся прямо в базе?

Подсказка: подумайте, что PostgreSQL предлагает специально для двоичных данных.

Ответ:

- **BYTEA**
- TEXT
- VARCHAR
- JSONB

🔧 Шаг 02:

Миссия 2

В ходе масштабирования ресурсного хранилища Minecraft команда решила, что некоторые таблицы будут сильно переиспользоваться и часто обновляться. Какой параметр конфигурации WAL влияет на то, когда транзакции считаются надёжно зафиксированными?

Подсказка: это касается параметра, управляющего тем, дожидается ли commit записи в WAL на диск.

Ответ:

- **synchronous_commit**
- fsync_connections
- wal_level
- autovacuum

🔧 Шаг 03:

Миссия 3

Для ресурса «золотые руды Minecraft» команда хочет обеспечить, что при попытке вставить запись с уже существующим уникальным ключом происходит обновление, а не ошибка. Как называется конструкция PostgreSQL, позволяющая это сделать?

Подсказка: она появилась в версии ~9.5 и часто называется «upsert».

Ответ:

- **INSERT ... ON CONFLICT DO UPDATE**
- MERGE TABLE
- REPLACE INTO
- UPSERT keyword

🔧 Шаг 04:

Миссия 4

Команда Minecraft-сервиса хочет защитить ресурсный склад от ошибок, если два процесса одновременно пытаются обновить один и тот же ряд. Какой уровень изоляции транзакций в PostgreSQL гарантирует, что такие конфликты будут обнаружены, и в случае нарушения произойдёт ошибка сериализации?

Подсказка: не Read Committed, не слабый уровень; нужно максимальное усилие по сохраняемости и согласованности.

Ответ:

- **SERIALIZABLE**
- Read Committed
- Repeatable Read
- Read Uncommitted

🔧 Шаг 05:

Миссия 5

Сопоставьте настройки PostgreSQL с их назначением:

Подсказка: одни параметры отвечают за надёжность, другие — за производительность или поведение при сбоях.

Ответ:

- **autocommit off** — все операции внутри транзакции до явного COMMIT

- **synchronous_commit = off** — повышение скорости за счёт возможной потери последних транзакций при краше
- **fsync = on** — гарантия, что данные WAL записаны на диск перед подтверждением транзакции
- **wal_level = logical** — поддержка логической репликации и потоковой передачи изменений

🔧 Шаг 06:

Миссия 6

Сопоставьте типы индексов PostgreSQL с их применением:

Подсказка: разные индексы полезны для разных задач: полнотекстового поиска, равенства, диапазонов.

Ответ:

- **B-Tree** — наилучший выбор для равенства и диапазонных запросов
- **Hash** — быстрое равенство, но не для диапазонов
- **GIN** — для массивов и полнотекстового поиска
- **BRIN** — для очень больших таблиц, упорядоченных по физическому виду

🔧 Шаг 07:

Миссия 7

Сопоставьте уровни изоляции транзакций с их свойствами в PostgreSQL:

Подсказка: подумайте, какие аномалии разрешены, а какие — нет.

Ответ:

- **READ COMMITTED** — каждый запрос видит данные, зафиксированные до начала запроса
- **REPEATABLE READ** — снимок в начале транзакции, все запросы видят одинаковую картину
- **SERIALIZABLE** — самый строгий уровень, предотвращает фантомные чтения и требует обработки ошибок сериализации
- **READ UNCOMMITTED** — в PostgreSQL ведёт себя как READ COMMITTED, грязных чтений нет

🔧 Шаг 08:

Миссия 8

Упорядочите шаги настройки логической репликации для ресурсного склада Minecraft-сервиса:

Подсказка: сначала подготовка, потом публикация, потом подписка, затем проверка данных.

1. Определить публикацию на первичном сервере (CREATE PUBLICATION ...)
2. Настроить слейв-сервер как подписчика (CREATE SUBSCRIPTION ...)
3. Убедиться, что все нужные таблицы имеют уникальные первичные ключи и индексы
4. Настроить права доступа и сетевые настройки между узлами
5. Проверить, что данные из первичного сервера реплицируются корректно

Ответ:

- Убедиться, что все нужные таблицы имеют уникальные первичные ключи и индексы
- Определить публикацию на первичном сервере (CREATE PUBLICATION ...)
- Настроить слейв-сервер как подписчика (CREATE SUBSCRIPTION ...)
- Настроить права доступа и сетевые настройки между узлами
- Проверить, что данные из первичного сервера реплицируются корректно

Шаг 09:

Миссия 9

Упорядочите шаги планирования миграции схемы базы данных с большими таблицами, когда требуется минимизировать простой Minecraft-сервиса:

Подсказка: сначала подготовка схемы и индексов, потом саму миграцию, затем переключение, затем проверка.

1. Добавить новые столбцы и индексы параллельно с существующими
2. Подготовить скрипты миграции и тестировать на копии базы
3. Запустить саму миграцию (ALTER TABLE ...)
4. Переключить сервис на новую схему / сделать релиз с мигрированным кодом
5. Проверить, что данные и запросы работают корректно под нагрузкой

Ответ:

- Добавить новые столбцы и индексы параллельно с существующими
- Подготовить скрипты миграции и тестировать на копии базы
- Запустить саму миграцию (ALTER TABLE ...)
- Переключить сервис на новую схему / сделать релиз с мигрированным кодом
- Проверить, что данные и запросы работают корректно под нагрузкой

Шаг 10:

Миссия 10

Упорядочите действия, которые должна предпринять команда Minecraft, чтобы настроить резервное копирование (backup) + восстановление (restore) в PostgreSQL с минимальной потерей данных:

Подсказка: сначала установить snapshot, потом WAL-архивирование, тестировать восстановление.

1. Настроить архивирование WAL (archive_mode и archive_command)
2. Сделать полную резервную копию базы (base backup)
3. Настроить права доступа и хранение бэкапа в безопасном месте
4. Тестировать восстановление из архива и WAL до момента сбоя
5. Регулярно проверять целостность бэкапов

Ответ:

- Настроить архивирование WAL (archive_mode и archive_command)
- Сделать полную резервную копию базы (base backup)
- Настроить права доступа и хранение бэкапа в безопасном месте
- Тестировать восстановление из архива и WAL до момента сбоя
- Регулярно проверять целостность бэкапов

Шаг 11:

Миссия 11

Minecraft-команда строит grps-сервис, который хранит метаданные ресурсов (имя, размеры, хеш) в PostgreSQL. Какой тип данных лучше всего использовать для хранения хеша SHA-256 (фиксированная длина 64 hex-символа)?

Подсказка: подумайте про экономию места и проверку на валидность.

Ответ:

- **CHAR(64)**

- TEXT
- VARCHAR(128)
- BYTEA

🔧 Шаг 12:

Миссия 12

Сотрудники Minecraft-сервиса решают, что таблица хранения логов будет расти очень быстро. Какое свойство следует указать при объявлении таблицы, чтобы автоматически удалять старые записи (например, старше 30 дней)?

Подсказка: есть встроенная возможность Postgres через механизм, связанный с триггерами и планировщиком.

Ответ:

- **Использовать pg_cron или сторонний job + триггер DELETE старых записей**
- CHECK constraint
- FOREIGN KEY с каскадным удалением
- UNIQUE index

🔧 Шаг 13:

Миссия 13

Для грпс-сервиса Minecraft требуется хранить географические координаты расположения ресурсов на карте. Какой тип данных PostgreSQL наиболее подходит для таких точных decimal координат (широта/долгота) с сохранением точности?

Подсказка: float может вносить ошибки, подумайте про decimal или numeric.

Ответ:

- **NUMERIC(9,6)**
- REAL
- DOUBLE PRECISION
- TEXT

🔧 Шаг 14:

Миссия 14

Представьте, что грпс-сервис Minecraft хочет масштабироваться горизонтально, и будет использовать логическую репликацию. Какой параметр в postgresql.conf должен быть установлен для поддержки логической репликации?

Подсказка: это параметр уровня WAL.

Ответ:

- **wal_level = logical**
- wal_level = minimal
- synchronous_commit = remote_write
- archive_mode = on

🔧 Шаг 15:

Миссия 15

Сопоставьте механизмы PostgreSQL с их свойствами в контексте хранения ресурсов Minecraft:

Подсказка: один механизм для масштабирования на чтение, другой для сохранения параллелизма, ещё для защиты данных.

Ответ:

- **Streaming Replication** — горизонтальное масштабирование чтения, основная + реплики
- **Logical Replication** — репликация на уровне логических изменений, таблиц
- **Point-in-Time Recovery (PITR)** — восстановление базы к определённом моменту используя WAL архивацию
- **Hot Standby** — реальные чтения с реплики во время работы мастера

🔮 **Шаг 16:**

Миссия 16

Сопоставьте виды индексов PostgreSQL с примерами использования в ресурсном складе Minecraft:

Подсказка: подумайте, где нужны быстрые поиска по префиксу, где — полнотекст, где — диапазон.

Ответ:

- **B-Tree** — поиск по ID, диапазон дат
- **GIN** — фильтрация по тегам или JSONB полю
- **GIN (pg_trgm)** — поиск по части строки, автодополнение

📖 Урок 7 - Создание портала между мирами — многопоточность (threading)

🔮 **Шаг 01:**

Миссия 1

Аликс из команды Minecraft-программистов пишет grpc-сервис, который генерирует порталы. Что делает метод `start()` у экземпляра `threading.Thread` ?

Подсказка: этот метод не запускает целевой код напрямую, а активирует процесс выполнения в другом потоке.

Ответ:

- **Запускает новый поток, в котором выполняется функция target**
- Вызывает функцию `target` немедленно в текущем потоке
- Создаёт новый процесс
- Останавливает выполнение основного потока

🔮 **Шаг 02:**

Миссия 2

При создании порталов через grpc-вызовы важно синхронизировать ресурсы. Какой инструмент из модуля `threading` позволяет блокировать доступ к ресурсу несколькими потоками?

Подсказка: подумай о механизме захвата и освобождения.

Ответ:

- **`threading.Lock`**
- `threading.Timer`
- `threading.Barrier`
- `time.sleep`

🔮 **Шаг 03:**

Миссия 3

Тесса из команды Minecraft-инженеров использует `join()` после запуска нескольких потоков. Что делает этот метод?

Подсказка: он связан с ожиданием завершения.

Ответ:

- **Блокирует основной поток до завершения указанного потока**
- Объединяет потоки в пул
- Создает копию потока
- Уничтожает поток вручную

🔗 Шаг 04:

Миссия 4

В одном из gRPC-порталов поток А ждёт ресурс, занятый потоком В, а поток В ждёт, пока поток А освободит другой ресурс. Что происходит?

Подсказка: вспомни классическую ситуацию взаимной блокировки.

Ответ:

- **Deadlock (взаимная блокировка)**
- Race condition
- Starvation
- Thread leak

🔗 Шаг 05:

Миссия 5

Сопоставьте элементы из модуля `threading` с их назначением:

Подсказка: обращай внимание на поведение — что управляет доступом, а что таймингом.

Ответ:

- **`threading.Thread`** — Создание и запуск нового потока
- **`threading.Lock`** — Синхронизация доступа к общим данным
- **`threading.Event`** — Сигнализация между потоками
- **`threading.Timer`** — Отложенный запуск функции

🔗 Шаг 06:

Миссия 6

Сопоставьте проблемы многопоточности с их описаниями:

Подсказка: ключевые слова: гонка, взаимная блокировка, ресурсы, `starvation`.

Ответ:

- **Race Condition** — Результат зависит от порядка выполнения потоков
- **Deadlock** — Потоки блокируют друг друга, ожидая ресурсы
- **Starvation** — Один поток постоянно недополучает ресурсы
- **Thread Leak** — Потоки создаются, но не завершаются должным образом

🔗 Шаг 07:

Миссия 7

Сопоставьте методы объекта `Thread` с их действиями:

Подсказка: смотри, что запускает, что проверяет, что завершает.

Ответ:

- **start()** — Запускает поток
- **join()** — Ожидает завершения потока
- **is_alive()** — Проверяет, работает ли поток
- **run()** — Выполняет целевую функцию (обычно не вызывается напрямую)

 **Шаг 08:**

Миссия 8

В Minecraft-команде программистов герой-Херобрин создал ggrc-портал, и нужно обрабатывать события в фоновом потоке. Что из следующего создаёт новый поток для выполнения функции target?

Подсказка: посмотри на класс, который принимает аргумент target.

Ответ:

- **threading.Thread(target=some_function)**
- threading.Lock()
- threading.Event()
- threading.Timer(0, some_function) without starting

 **Шаг 09:**

Миссия 9

Когда портал начинает обрабатывать несколько ggrc-запросов одновременно, герой-Стив замечает, что данные ресурса портала иногда повреждаются. Какой механизм threading помогает предотвратить одновременную запись в один и тот же ресурс?

Подсказка: подумай про взаимное исключение (mutual exclusion).

Ответ:

- **использовать threading.Lock()**
- использовать threading.ThreadPoolExecutor без lock
- использовать time.sleep между операциями
- использовать глобальную переменную без защиты

 **Шаг 10:**

Миссия 10

Герой-Моника запускает поток и хочет убедиться, что портал завершил все задачи до закрытия основного ggrc-сервиса. Какой метод следует вызвать для каждого потока?

Подсказка: метод, который заставляет основной поток ждать.

Ответ:

- **thread.join()**
- thread.stop()
- thread.kill()
- thread.is_running()

 **Шаг 11:**

Миссия 11

Герой-Алекс наблюдает, что ggrc-портал отвечает просто медленно, когда много потоков одновременно ждут того же ресурса. Какая из ситуаций может быть причиной?

Подсказка: ищи блокировки, ожидания, когда поток не получает ресурсы длительное время.

Ответ:

- **Hit to Lock contention**
- Too many threads with no I/O work
- Garbage collection overhead
- Lack of target function

🔮 **Шаг 12:**

Миссия 12

Сопоставьте объекты из модуля threading с их описанием:

Подсказка: Некоторые объекты дают управление потоком, другие — синхронизацию или сигнал.

Ответ:

- **threading.Condition** — сигнал между потоками + возможность ожидать на условии
- **threading.Event** — простое уведомление между потоками
- **threading.Semaphore** — ограничение количества потоков, одновременно получает доступ
- **threading.Lock** — взаимное исключение для одного ресурса

🔮 **Шаг 13:**

Миссия 13

Сопоставьте виды проблем многопоточности с их симптомами в gprc-портале:

Подсказка: обратите внимание, когда обработка спотыкается, когда ресурс недоступен, когда поток застревает.

Ответ:

- **Deadlock** — две задачи ждут друг друга и ни одна не продолжает
- **Race Condition** — результат зависит от порядка, в котором потоки выполняются
- **Starvation** — один поток почти никогда не получает доступ к ресурсу
- **Livelock** — потоки активно меняют состояние, но никакого прогресса нет

🔮 **Шаг 14:**

Миссия 14

Сопоставьте методы потоков с тем, что они возвращают или проверяют:

Подсказка: один из методов проверяет, жив ли поток; другой может быть переопределён.

Ответ:

- **is_alive()** — возвращает True если поток ещё выполняется
- **daemon** — свойство, показывающее, является ли поток демоном
- **run()** — метод, который выполняет target (обычно не вызывается вручную)
- **setDaemon(True)** — устанавливает поток как демон, чтобы не блокировал завершение программы

🔮 **Шаг 15:**

Миссия 15

Упорядочите шаги для безопасного завершения всех потоков перед закрытием gprc-портала:

Подсказка: сначала сигнал, потом join, потом очистка.

1. Установить флаг остановки (stop flag) или событие для всех потоков
2. Дать всем потокам процедуру обработки флага и выхода
3. Вызвать join() для каждого потока, чтобы дождаться завершения
4. Освободить ресурсы (сокеты, файлы и т.п.) после того, как потоки завершились
5. Закрыть основной grpc-сервис

Ответ:

- Установить флаг остановки (stop flag) или событие для всех потоков
- Дать всем потокам процедуру обработки флага и выхода
- Вызвать join() для каждого потока, чтобы дождаться завершения
- Освободить ресурсы (сокеты, файлы и т.п.) после того, как потоки завершились
- Закрыть основной grpc-сервис

Шаг 16:

Миссия 16

Упорядочите этапы обработки нескольких grpc-запросов параллельно с использованием threading:

Подсказка: сначала подготовка, потом запуск, потом синхронизация, потом завершение.

1. Подготовить список задач / функций, которые будут обрабатываться параллельно
2. Создать потоки для каждой задачи и запустить их через start()
3. Синхронизировать доступ к общим ресурсам (locks, events и т.д.) внутри задач
4. Ожидать завершения потоков с помощью join()
5. Собрать результаты и вернуть ответ через grpc

Ответ:

- Подготовить список задач / функций, которые будут обрабатываться параллельно
- Создать потоки для каждой задачи и запустить их через start()
- Синхронизировать доступ к общим ресурсам (locks, events и т.д.) внутри задач
- Ожидать завершения потоков с помощью join()
- Собрать результаты и вернуть ответ через grpc